REVISIÓN DE ARQUITECTURA

smp-device-integrity-service

Integridad de dispositivo de la app Personas
Propuesta AppSec v2 — hallazgos, posturas y diagramas

Documento	Revisión de arquitectura de software / AppSec
Revisión	José Amadeo Díaz
Origen	**propuesta_arquitectura_appsec_v2** y sus diagramas
Audiencia	Equipos de AppSec, arquitectura, APIM, móvil y backends de negocio
Fecha	5 de noviembre de 2026
Uso	Interno. Base para la siguiente versión del documento de arquitectura
Veredicto	La dirección es correcta. El diseño todavía no está listo para producción ni para aprobarse como está.

Cómo leer este documento

Cada hallazgo se presenta bajo una estructura homogénea que permite evaluar su impacto y definir las acciones correspondientes:

1. Situación actual: descripción de lo establecido en la propuesta de solución
2. Análisis e impacto: implicancias técnicas, arquitectónicas y funcionales del hallazgo
3. Escenario de aplicación: descripción de cómo se manifiesta o materializa la situación identificada.
4. Recomendación arquitectónica: acciones propuestas para mitigar riesgos, resolver brechas o mejorar la solución.
5. Propuesta de incorporación: texto sugerido para su inclusión en el documento de arquitectura.

Contenido

Contenido 1
1. Resumen ejecutivo 2
1.1 Hallazgos y decisiones arquitectónicas pendientes 3
1.2 Aspectos de la propuesta que están bien encaminados 5
1.3 Organización del Documento 5
2. Vista general de la solución propuesta 5

2.1 Niveles de validación de integridad 6
2.2 Modos de operación y controles de contingencia 6
2.3 Android — Goole Play Integrity 7
2.4 Huawei — SysIntegrity 9
2.5 iOS — Apple App Attest 11
2.6 Decisión y aplicación de políticas de seguridad en APIM 14
3. Hallazgos de seguridad 16
3.1 S1 — Posibilidad de falsificar el header X-Integrity-Decision 16
3.2 S2 — El cliente puede seleccionar la plataforma con el control de seguridad más débil 21
3.3 S3 — Tier 2 no garantiza la vinculación de la evidencia con la operación 26
4. Hallazgos de capacidad y operación 43
4.1 O1 — Cuota de Google Play Integrity 43
La habilitación de Google Play Integrity debe realizarse de manera progresiva. Primero se debe asegurar la cuota necesaria, después validar el consumo y comportamiento en MONITOR y, finalmente, activar ENFORCE cuando exista capacidad suficiente para atender la demanda proyectada. 45
4.2 O2 — Counter iOS en Cosmos DB 45
4.3 O3 — Dispositivos sin App Attest 51
4.4 O4 — Versiones antiguas de la app 55
4.5 O5 — Rate limit de /deviceintegrity/initiate 60
4.6 O6 — Huawei: validación local del JWS, no una API de descifrado 64
5. Contradicciones internas del documento 68
6. Pendientes de la sección 4.4 73
6.1 Caída del servicio o del proveedor 73
6.2 Named Value ausente o inválido 75
6.3 Tratamiento de SKIPPED en el backend 76
6.4 Responsable del switch, aprobación y caducidad 77
7. Referencias usadas en la revisión 79

1. Resumen ejecutivo

Se realizó una revisión de la propuesta para centralizar la validación de integridad de la aplicación Personas mediante el servicio **smp-device-integrity-service**, utilizando APIM como punto de orquestación e integrando los mecanismos de seguridad de Google Play Integrity, Huawei SysIntegrity y Apple App Attest.

La propuesta sigue un enfoque arquitectónico adecuado: validar la integridad de la aplicación desde el servidor, utilizar desafíos de un solo uso (*challenge*), diferenciar los modos de monitoreo (MONITOR) y bloqueo (ENFORCE), y realizar estas validaciones antes de permitir el acceso a los servicios de negocio.

También es importante mantener una separación clara entre la validación de integridad del dispositivo, los controles de autorización y los mecanismos de idempotencia. Cada uno cumple una función de seguridad diferente.

Sin embargo, la propuesta todavía requiere definir algunos aspectos importantes antes de obtener la aprobación arquitectónica y avanzar a producción. Principalmente, se necesita precisar cómo se vincula la evidencia de integridad con cada operación, qué componente será responsable de aplicar las políticas, cómo responderá la solución ante fallas de los servicios de validación y cómo se gestionará el consumo de las cuotas disponibles de Google.

Este documento presenta los puntos identificados, el análisis de sus implicancias y las recomendaciones arquitectónicas correspondientes. El objetivo es que el equipo pueda resolver estas definiciones y consolidar una propuesta consistente, segura y preparada para su implementación en producción.

1.1 Hallazgos y decisiones arquitectónicas pendientes

Se presentan los principales hallazgos identificados durante la revisión arquitectónica, ordenados según su impacto y prioridad. Para cada uno se describe la situación encontrada y la recomendación propuesta para resolverla. Los puntos clasificados como bloqueantes requieren atención antes de aprobar la solución y autorizar su paso a producción.

#	Prioridad	Hallazgo	Postura en una línea	Estado
S1	Bloqueante	El cliente puede manipular el header X-Integrity-Decision y enviar una decisión falsa al backend.	APIM debe eliminar cualquier valor recibido del cliente y establecer la decisión validada por el servidor. El backend debe verificar que provenga de una fuente confiable.	Pendiente de definición
S2	Bloqueante	El cliente podría seleccionar la plataforma de validación más permisiva.	La plataforma de validación debe estar asociada al dispositivo registrado y no depender de un valor enviado por el cliente.	Pendiente de definición
S3	Bloqueante	En Tier 2 la evidencia no queda ligada a la petición: Android sin requestHash, Huawei con nonce local reutilizable e iOS con clientDataHash enviado por el cliente.	Recalcular en el servidor el dato que liga la prueba a esta petición (requestHash, nonce derivado o clientDataHash) y registrar lo ya usado. No usar challenge de /initiate en Tier 2.	Pendiente de definición

O1	Bloqueante	La cuota de Google Play Integrity podría ser insuficiente para la demanda de producción (10 000 solicitudes diarias por defecto).	Gestionar la ampliación de cuota antes de producción. Sin capacidad suficiente, la validación de dispositivos Android podría verse interrumpida.	Pendiente de definición
O2	Alto	El contador de seguridad de iOS en Cosmos DB no contempla control de concurrencia mediante ETag.	Implementar actualizaciones condicionales mediante ETag para evitar que dos solicitudes concurrentes acepten la misma evidencia.	Pendiente de definición
O3	Alto	No se ha definido el comportamiento para dispositivos que no soportan App Attest.	Definir una política explícita para estos dispositivos: DENY en Tier 1 y DEGRADED en Tier 2, según el nivel de riesgo.	Pendiente de definición
O4	Alto	No se ha definido el tratamiento de versiones antiguas de la aplicación.	Establecer una versión mínima soportada y una transición controlada entre MONITOR y ENFORCE.	Pendiente de definición
O5	Alto	El endpoint /initiate no tiene controles suficientes contra solicitudes masivas.	Implementar límites de solicitudes por sesión, dirección IP y volumen global, utilizando JWT y controles de frecuencia en Redis.	Pendiente de definición
O6	Alto	No está definido si la validación de Huawei SysIntegrity se realiza localmente o mediante una API externa.	Definir la validación local de la firma JWS y la cadena de confianza correspondiente. Ajustar el diagrama para reflejar el flujo definitivo.	Pendiente de definición
C1	Medio	Existen diferencias entre los contratos y flujos descritos (BFF, headers, hashes y payloads).	Unificar el contrato de integración: definir un único orquestador, un contrato de headers, reglas de generación de hashes y un catálogo de operaciones.	Pendiente de definición
P1	Bloqueante	Las políticas de seguridad del apartado 4.4 todavía tienen definiciones pendientes.	Formalizar las reglas de decisión, el comportamiento ante fallas y el responsable de activar o desactivar las políticas, incluyendo su vigencia.	Pendiente de definición

- Bloqueante: debe resolverse antes de aprobar la arquitectura o autorizar la salida a producción.
- Alto: requiere definición e implementación antes de completar la solución.
- Medio: requiere alineamiento para garantizar consistencia entre los componentes.

Estados:

- Pendiente de definición.
- Pendiente de implementación.
- Validado.

- Cerrado.

1.2 Aspectos de la propuesta que están bien encaminados

La revisión identifica decisiones arquitectónicas acertadas que conviene mantener en la siguiente versión de la solución:

- Centralizar la validación de las tres plataformas (Android, Huawei e iOS) en un servicio interno, evitando que cada backend implemente su propia lógica de validación.
- Realizar la verificación de las evidencias de integridad en el servidor, sin delegar esta responsabilidad al cliente.
- Utilizar un *challenge* de un solo uso, con un tiempo de vigencia (TTL) de 120 segundos para las operaciones Tier 1.
- Diferenciar los modos MONITOR y ENFORCE, evitando registrar tokens o evidencias de seguridad en los logs.
- Aplicar las decisiones de seguridad desde APIM, antes de permitir el acceso a los servicios de negocio.
- Mantener separados los controles de integridad del dispositivo, los mecanismos de autorización y la idempotencia de las operaciones de negocio.

Estos principios constituyen una buena base arquitectónica y deben conservarse en la versión ajustada de la propuesta.

1.3 Organización del Documento

El documento está organizado en las siguientes secciones para facilitar la revisión de los hallazgos y la toma de decisiones:

1. **Vista de la solución:** presenta los nueve diagramas de secuencia correspondientes a los flujos de Android, Huawei, iOS y la toma de decisiones en APIM.
2. **Hallazgos de seguridad (S1–S3):** analiza los riesgos relacionados con la autenticidad de la decisión y la vinculación de las evidencias de integridad con cada operación.
3. **Hallazgos de capacidad y operación (O1–O6):** identifica los aspectos que deben resolverse para garantizar la disponibilidad, escalabilidad y correcto funcionamiento de la solución.
4. **Consistencia de la propuesta (C1):** identifica contradicciones y diferencias entre los contratos, componentes y flujos descritos en el documento.
5. **Definiciones pendientes de políticas (P1):** detalla las decisiones de seguridad que aún deben formalizarse en la sección 4.4.

2. Vista general de la solución propuesta

La solución plantea centralizar la validación de integridad de las aplicaciones móviles Android, Huawei e iOS en un servicio interno denominado **smp-device-integrity-service**. Este servicio será el responsable de validar las evidencias proporcionadas por Google Play Integrity, Huawei SysIntegrity y Apple App Attest.

La integración se realiza a través de **APIM**, que actúa como punto de orquestación entre la aplicación móvil, el servicio de validación y los servicios de negocio. APIM solicita los desafíos de seguridad (*challenge*) cuando corresponde, envía las evidencias para su validación, recibe el resultado y aplica la decisión antes de permitir que la solicitud continúe hacia el backend de negocio.

La solución contempla los siguientes componentes de soporte:

- **Redis:** almacena temporalmente los desafíos de seguridad, incluyendo su tiempo de vigencia y control de uso.
- **Cosmos DB:** almacena la información necesaria para gestionar las claves de los dispositivos iOS, incluyendo keyId, clave pública y contador de uso (*counter*).

2.1 Niveles de validación de integridad

La propuesta establece tres niveles de seguridad (*tiers*), según la criticidad de las operaciones:

- **Tier 1 – Operaciones críticas:** requiere un desafío generado por el servidor, de un solo uso y con una vigencia de 120 segundos. La evidencia debe estar vinculada a la operación que se desea ejecutar, evitando que pueda reutilizarse en otra solicitud.
- **Tier 2 – Operaciones de riesgo intermedio:** no requiere un desafío por cada solicitud. La evaluación se realiza aplicando las reglas de seguridad configuradas en APIM, según la política definida para este nivel.
- **Tier 3 – Operaciones de menor criticidad:** no requiere validación de integridad del dispositivo, de acuerdo con la clasificación de riesgo establecida.

2.2 Modos de operación y controles de contingencia

La solución contempla dos modos de funcionamiento:

- **MONITOR:** permite evaluar y registrar los resultados de las validaciones sin bloquear las solicitudes, facilitando el análisis del comportamiento antes de aplicar restricciones.
- **ENFORCE:** aplica las políticas de seguridad definidas y permite bloquear las solicitudes que no cumplan los criterios establecidos.

Adicionalmente, se contemplan *Named Values* en APIM como mecanismos de contingencia para habilitar o deshabilitar la aplicación de las validaciones por plataforma, ante situaciones que requieran una intervención operativa.

Estos mecanismos deberán contar con reglas claras de activación, responsables definidos y controles para evitar que una situación de contingencia reduzca indebidamente el nivel de seguridad.

> **Nota sobre los ajustes realizados a los diagramas**
>
> Los diagramas presentados en esta sección incorporan los ajustes identificados durante la revisión arquitectónica. Entre los principales cambios se encuentran:

- **Tier 1:** se precisa que la validación y aplicación de la decisión corresponden a APIM, y no al BFF.
- **Cálculo del hash:** se unifica el criterio de separación utilizado para su generación.
- **Huawei e iOS en Tier 2:** se incorpora la vinculación de la evidencia de integridad con la solicitud correspondiente.

Cuando estos cambios difieren de lo descrito en la propuesta original, se identifican expresamente como ajustes derivados de la revisión arquitectónica.

Por tanto, los diagramas representan el flujo recomendado y no necesariamente el comportamiento descrito en la versión original de la propuesta.

2.3 Android — Google Play Integrity

Preparación del proveedor (Warm-up)

Al iniciar la aplicación, se ejecuta *prepareIntegrityToken* para preparar el proveedor de Play Integrity y reducir la latencia de las solicitudes posteriores. Esta operación consume cuota de solicitudes de tokens (*token requests*), pero no corresponde a las operaciones de descifrado (*decode*).

El proveedor preparado se reutiliza para las validaciones de integridad de Tier 1 y Tier 2, evitando realizar el proceso de preparación en cada solicitud.



*Figura 2.1 — **Preparación del proveedor de Play Integrity al iniciar la aplicación***

Flujo Tier 1 – Validación con challenge

Para las operaciones críticas (Tier 1), la aplicación solicita previamente un *challenge* al servidor. Con este valor y el contenido canónico de la operación, calcula el requestHash mediante SHA-256:

requestHash = SHA256(nonce | payload_canónico)

La aplicación solicita a Google Play Integrity el token de integridad asociado a ese hash y lo envía para su validación. El servidor realiza la verificación de la evidencia mediante decodeIntegrityToken, operación que se ejecuta en la infraestructura de Google.

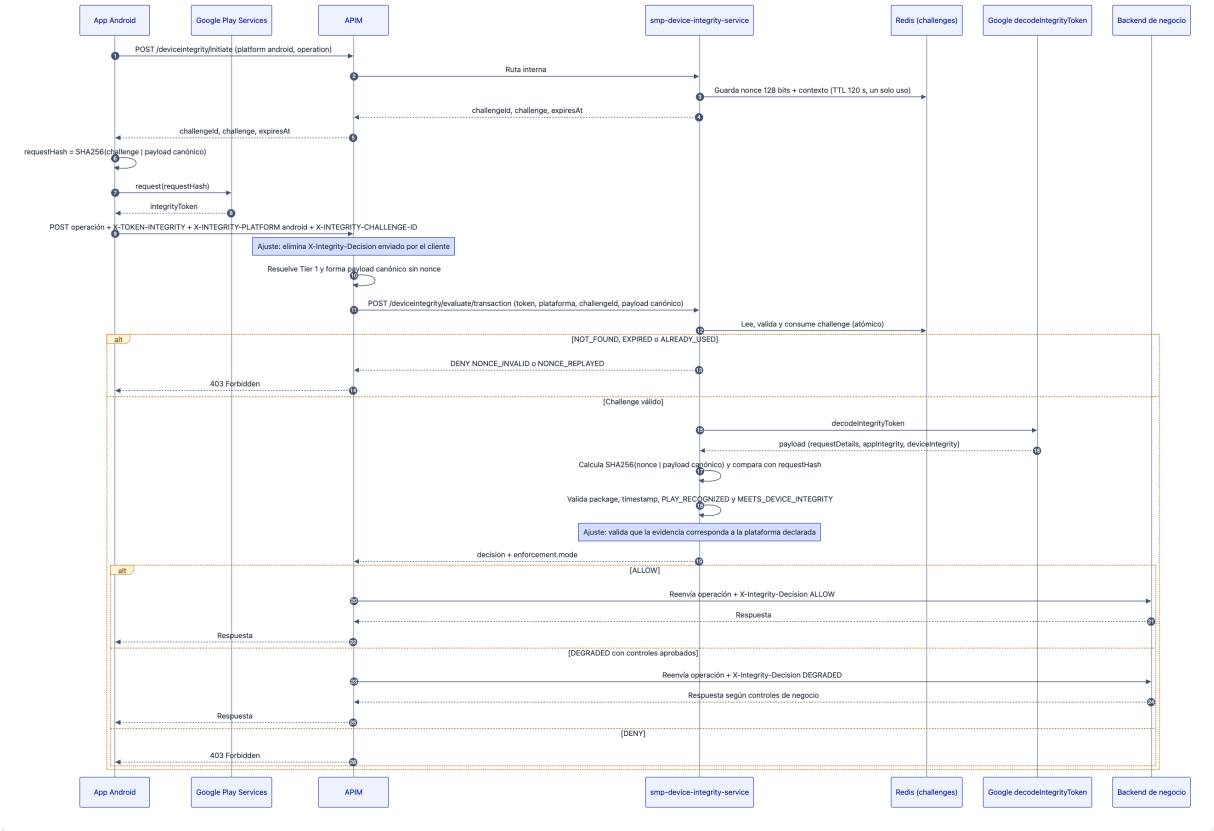


Figura 2.2 — ***Flujo Android Tier 1: generación del challenge y vinculación de la evidencia mediante requestHash.***

Flujo Tier 2 – Validación sin challenge por solicitud

En la propuesta original, el flujo Tier 2 no contemplaba el uso de requestHash, por lo que la evidencia de integridad no quedaba vinculada directamente al contenido de la solicitud.

Como parte de la revisión arquitectónica, el diagrama incorpora este mecanismo: el servidor recalcula el hash a partir de la información recibida y verifica su correspondencia con la evidencia presentada.

Este ajuste permite mantener la vinculación entre la prueba de integridad y la solicitud, incluso cuando no se utiliza un challenge individual para cada operación.

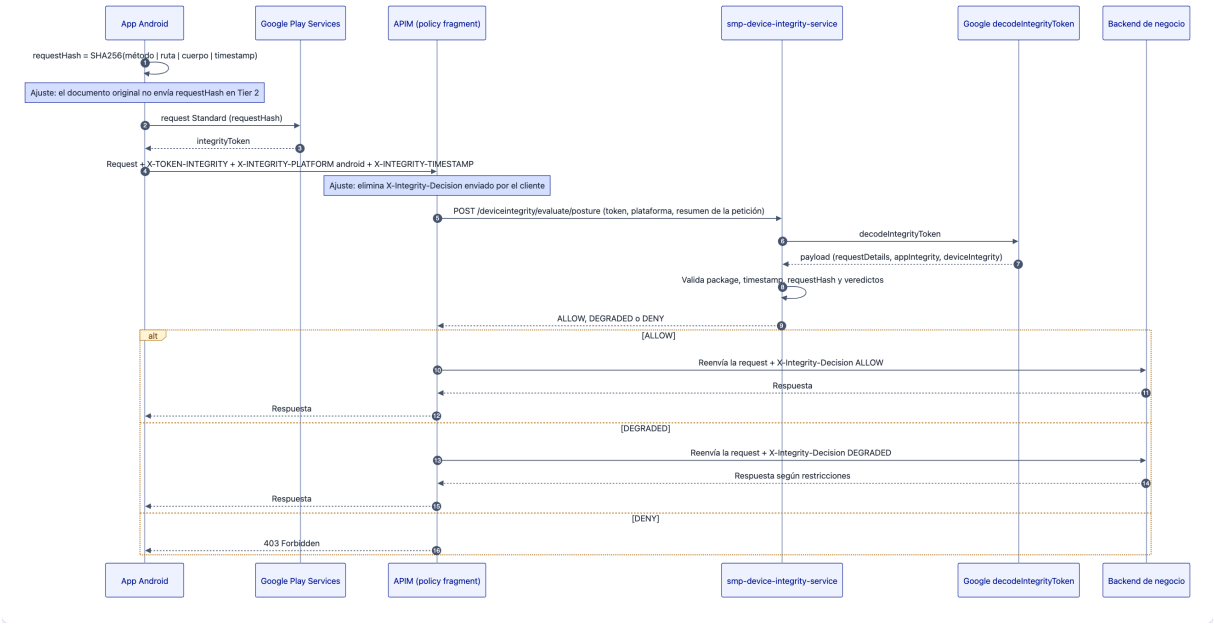


Figura 2.3 Flujo Tier 2 – Validación sin challenge por solicitud

2.4 Huawei — SysIntegrity

La aplicación obtiene la evidencia de integridad en formato JWS mediante HMS Core, que se comunica con los servicios de Huawei. Posteriormente, esta evidencia se envía al banco para su validación, donde se verifica su firma y contenido antes de tomar una decisión de seguridad.

Flujo Tier 1 – Validación con challenge

Para las operaciones críticas (Tier 1), el servidor genera un *challenge* de un solo uso y lo almacena temporalmente en Redis. Este valor se utiliza como nonce en la solicitud de integridad a Huawei.

La evidencia recibida debe contener el nonce esperado. El servidor verifica su correspondencia, valida la firma del JWS y comprueba que el challenge no haya sido utilizado previamente.

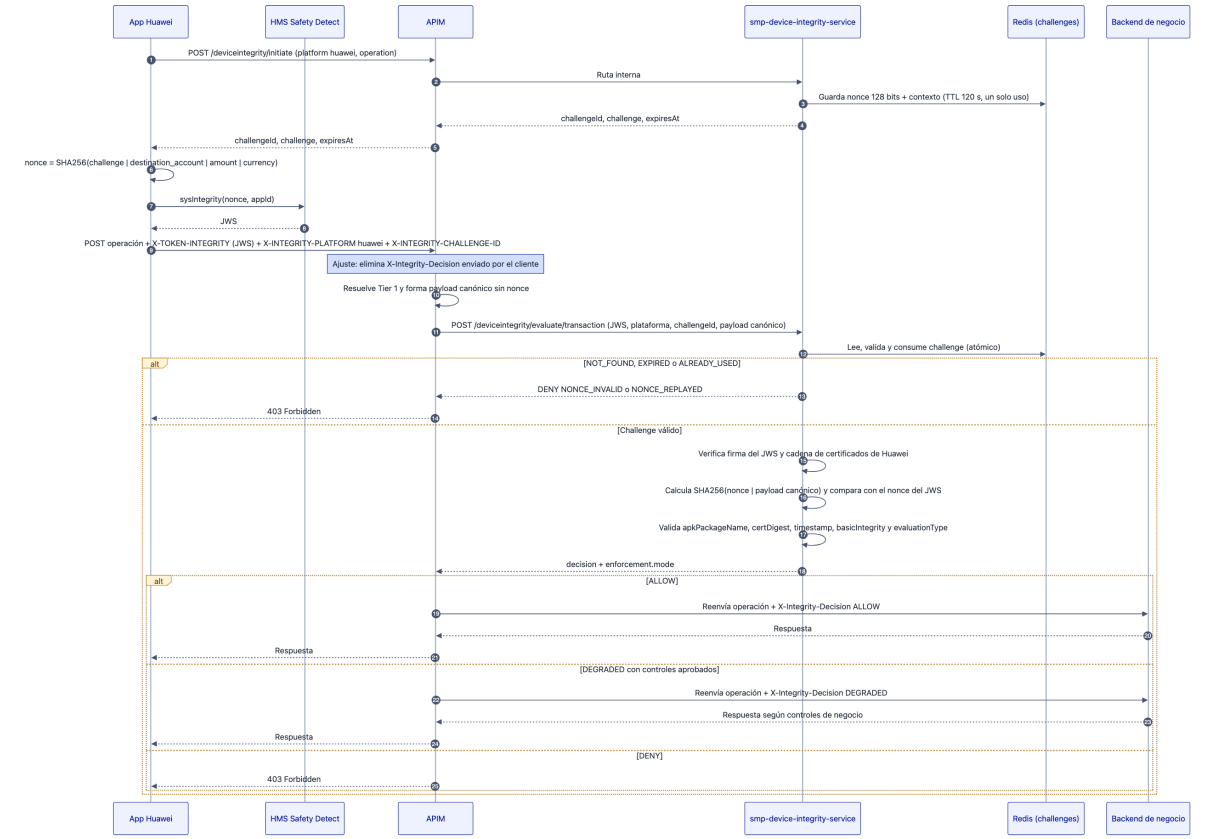


Figura 2.4 — Flujo Huawei Tier 1: validación de integridad mediante challenge generado por el servidor.

Flujo Tier 2 – Validación sin challenge por solicitud

Para las operaciones Tier 2, no se genera un challenge individual por solicitud. En su lugar, el nonce se deriva de la información de la petición y se registra en Redis para controlar su reutilización.

Este mecanismo permite vincular la evidencia con la solicitud correspondiente y establecer un control adicional contra ataques de repetición (*replay attacks*).

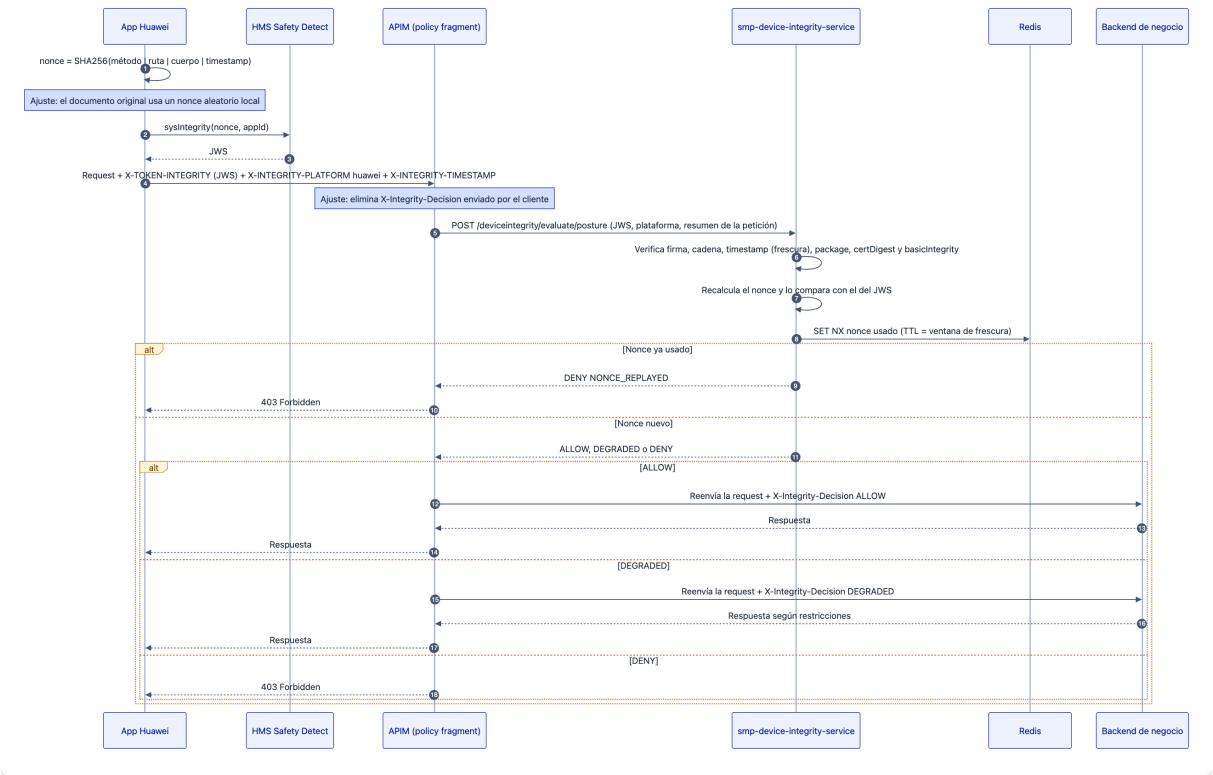


Figura 2.5 — Flujo Huawei Tier 2: generación de nonce derivado de la petición y control anti-replay mediante Redis.

2.5 iOS — Apple App Attest

Enrolamiento y registro del dispositivo

El proceso de enrolamiento es común para las operaciones Tier 1 y Tier 2. La aplicación genera una clave criptográfica mediante App Attest y solicita a Apple la correspondiente evidencia de atestación (*attestation*).

El servidor verifica la evidencia recibida, valida la cadena de certificados hasta la raíz de confianza de Apple y, si el proceso es satisfactorio, registra en Cosmos DB el identificador de la clave (keyId), la clave pública asociada y el contador inicial (counter = 0).

Este registro permite reconocer la clave del dispositivo y verificar las aserciones generadas en las solicitudes posteriores.

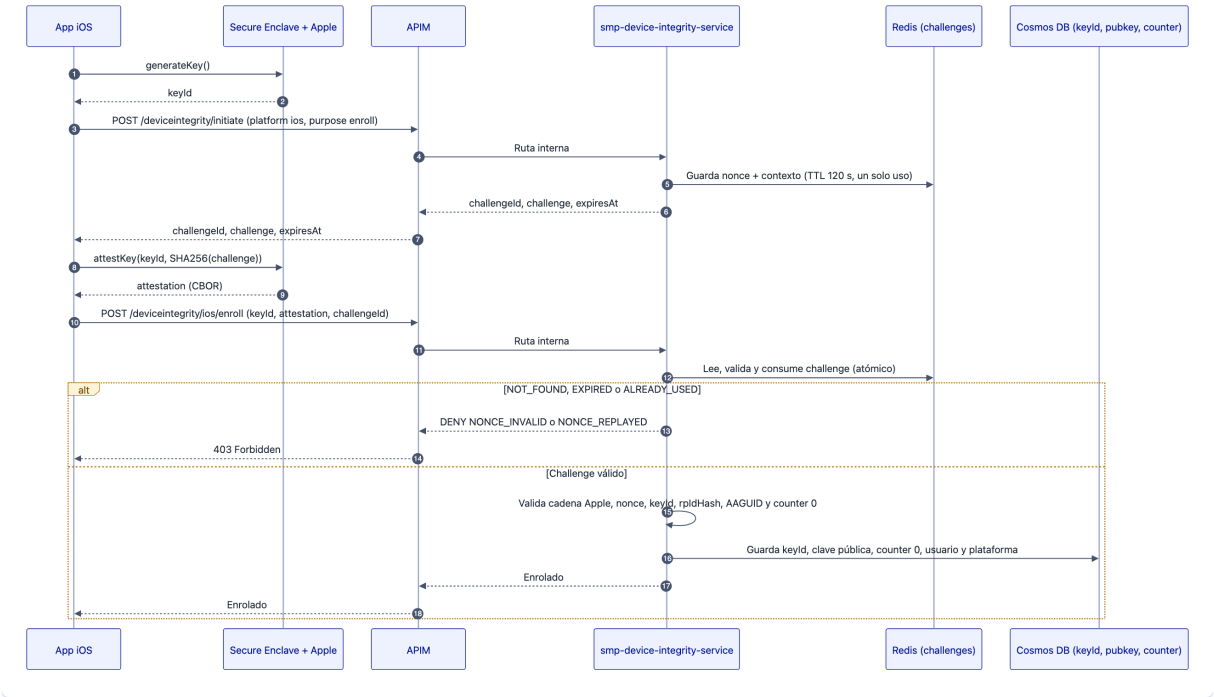


Figura 2.6 — Flujo de Enrolamiento y registro de claves de Apple App Attest.

Flujo Tier 1 – Validación con challenge

Para las operaciones críticas (Tier 1), el servidor genera un *challenge* de un solo uso. La aplicación utiliza este valor junto con la información de la operación para calcular el clientDataHash y generar una aserción (*assertion*) mediante App Attest.

El servidor verifica la aserción utilizando la clave pública previamente registrada, comprueba la vinculación con el challenge y la operación solicitada, y valida que el contador sea consistente con el último valor aceptado.

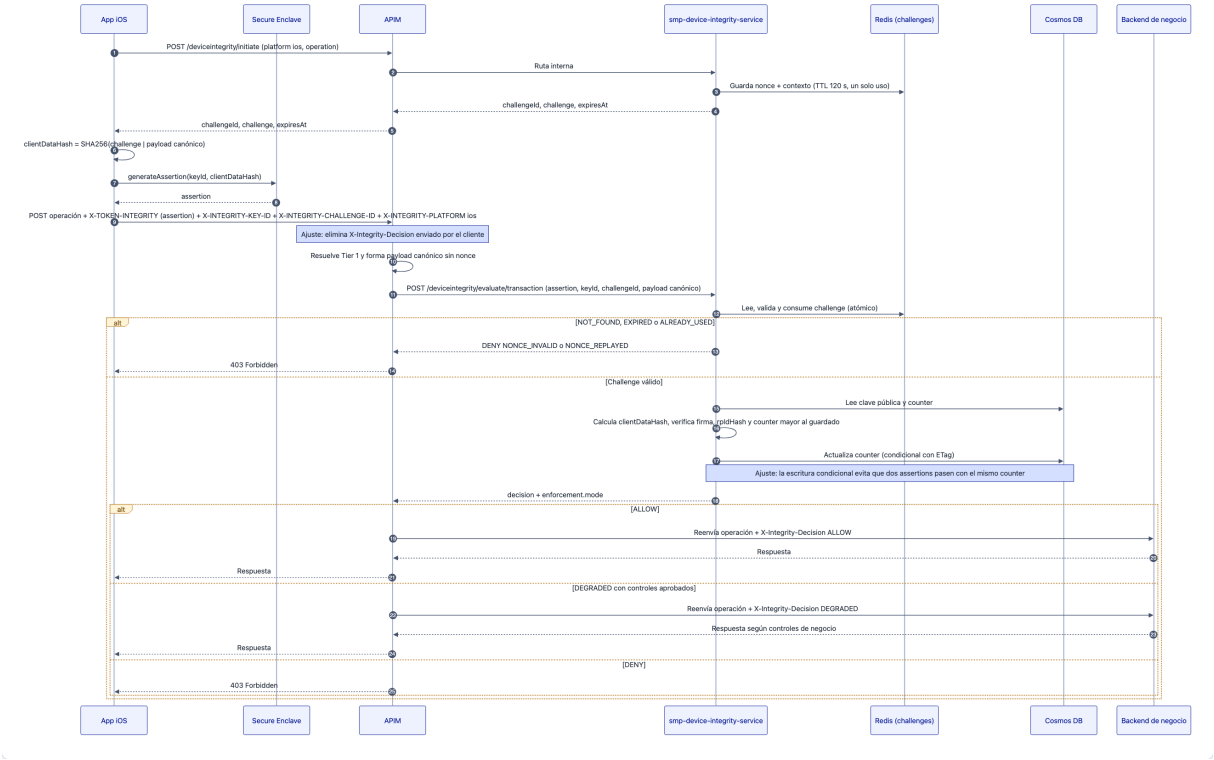


Figura 2.7 — Flujo iOS Tier 1: generación de aserción vinculada al challenge y al contenido de la operación.

Flujo Tier 2 – Validación sin challenge por solicitud

Para las operaciones Tier 2, no se solicita un challenge individual al servidor. La aplicación genera la aserción utilizando un clientDataHash asociado a la petición.

Como parte del ajuste arquitectónico, el servidor recalcula el clientDataHash a partir de la información recibida y verifica que corresponda con el valor utilizado para generar la aserción. También debe validar la firma y controlar la secuencia del contador para evitar la reutilización de evidencias.

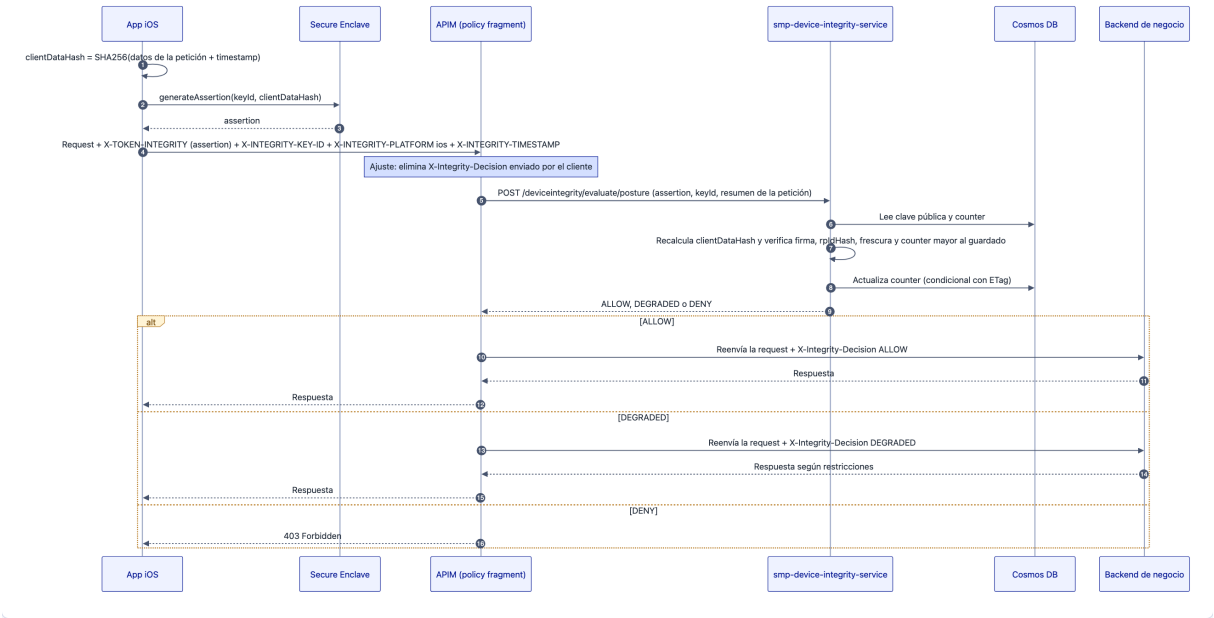


Figura 2.8 — Flujo iOS Tier 2: validación de aserción mediante clientDataHash recalculado en el servidor y control del contador

2.6 Decisión y aplicación de políticas de seguridad en APIM

APIM es el punto responsable de aplicar las decisiones de seguridad antes de permitir que una solicitud llegue al backend de negocio.

La solución contempla dos modos de operación:

- **ENFORCE:** APIM aplica el resultado de la validación de integridad. Las solicitudes que no cumplen las políticas de seguridad son bloqueadas antes de llegar al backend.
- **MONITOR:** APIM registra los resultados de las validaciones para su análisis, pero no bloquea las solicitudes por motivos de integridad. Esto permite evaluar el comportamiento de la solución antes de activar las restricciones.

Adicionalmente, se definen mecanismos de contingencia mediante los siguientes *Named Values* de APIM:

- allowintegrityverify: control global de la validación de integridad.
- allowintegrityverifyandroid: control específico para Android.
- allowintegrityverifyhuawei: control específico para Huawei.
- allowintegrityverifyios: control específico para iOS.

Estos parámetros permiten habilitar o deshabilitar la evaluación de integridad de manera global o por plataforma, según las necesidades operativas.

Criterio de seguridad: el comportamiento predeterminado debe ser validar la integridad. La desactivación de estos controles debe considerarse una medida excepcional de contingencia, con responsables definidos, trazabilidad y condiciones para su restablecimiento.

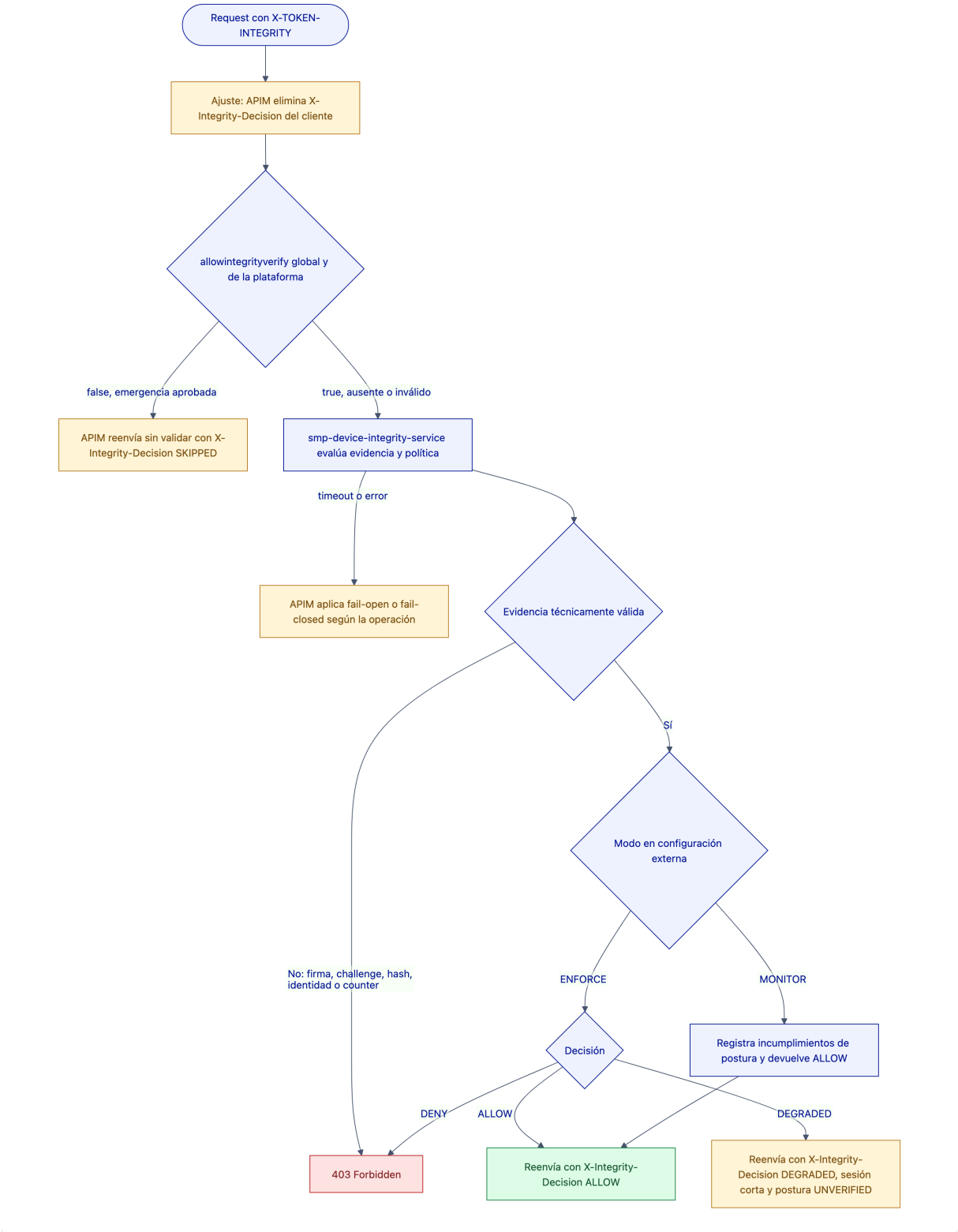


Figura 2.9 — Flujo de decisión con switches de emergencia.

El siguiente diagrama muestra cómo APIM controla la aplicación de las políticas de integridad, considerando la configuración global, los controles específicos por plataforma y el modo operativo (MONITOR o ENFORCE).

También se representan los escenarios de contingencia, incluyendo la desactivación autorizada de las validaciones y el comportamiento ante errores o indisponibilidad del servicio.

Un aspecto importante es que APIM debe eliminar cualquier decisión de integridad enviada por el cliente y establecer sus propios valores de confianza a partir del resultado obtenido del servicio de validación. Los estados SKIPPED y DEGRADED deben conservar su significado y no interpretarse automáticamente como una validación satisfactoria.

3. Hallazgos de seguridad

3.1 S1 — Posibilidad de falsificar el header X-Integrity-Decision

Prioridad: bloqueante.

Descripción del hallazgo

El backend de negocio utiliza el header X-Integrity-Decision para determinar si una solicitud cumple con las condiciones de integridad del dispositivo.

El problema es que este header puede ser enviado o manipulado directamente por el cliente. Por lo tanto, si APIM no elimina el valor recibido y establece uno nuevo a partir del resultado real de la validación, un atacante podría enviar una decisión falsa, por ejemplo ALLOW, y conseguir que el backend interprete la solicitud como validada.

Esto representa un riesgo crítico, ya que permitiría evadir el control de integridad sin necesidad de comprometer el servicio de validación.

Flujo de riesgo identificado

El diseño contempla que el backend reciba una decisión de integridad establecida por APIM. Sin embargo, si no se garantiza el origen de ese valor, el backend podría confiar en un header que realmente fue proporcionado por el cliente.

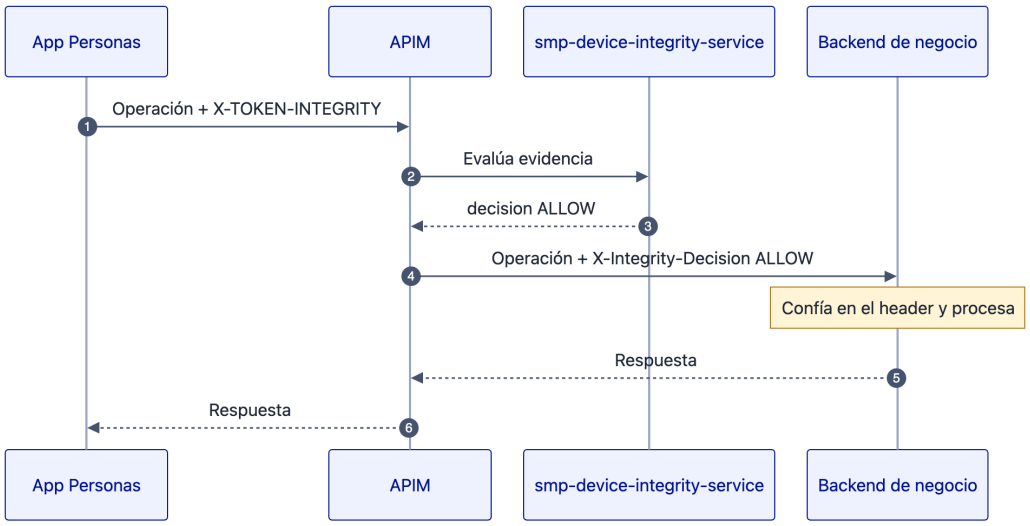


Figura 3.1 — El backend confía en un header de decisión que podría haber sido enviado por el cliente.

Ataque 1: manipulaión del header a través del APIM

Escenario de explotación

APIM, por defecto, puede reenviar los headers recibidos en la solicitud del cliente. Si el flujo de validación no elimina explícitamente el header original y lo reemplaza correctamente, existe la posibilidad de que un valor manipulado llegue al backend.

Esta situación podría producirse, por ejemplo, cuando:

- Una operación no incluye la política de validación de integridad.
- El mecanismo de contingencia permite continuar sin controlar previamente el header recibido.
- La política utiliza append en lugar de reemplazar el valor existente.
- Existe una diferencia en el nombre del header utilizado entre APIM y el backend.
- Una ruta alternativa permite llegar al backend sin aplicar los controles de seguridad previstos.

El riesgo no desaparece por el hecho de que el backend se encuentre en una red interna. El atacante puede utilizar el flujo legítimo de comunicación a través de APIM para introducir un valor manipulado.

Impacto

Una solicitud que no ha superado la validación de integridad podría llegar al backend con un header X-Integrity-Decision: ALLOW, haciendo que el servicio de negocio tome una decisión de seguridad basada en información no confiable.

Recomendación arquitectónica

APIM debe eliminar siempre cualquier valor de X-Integrity-Decision recibido del cliente y establecer uno nuevo exclusivamente a partir del resultado obtenido de smp-device-integrity-service.

La política debe utilizar reemplazo explícito (override) y no agregar valores al header existente (append). Este comportamiento debe mantenerse incluso en los escenarios de contingencia, definiendo expresamente los estados SKIPPED y DEGRADED.

Adicionalmente, el backend debe aceptar decisiones únicamente a través de canales de comunicación confiables y protegidos, garantizando que no pueda ser invocado directamente por clientes externos.

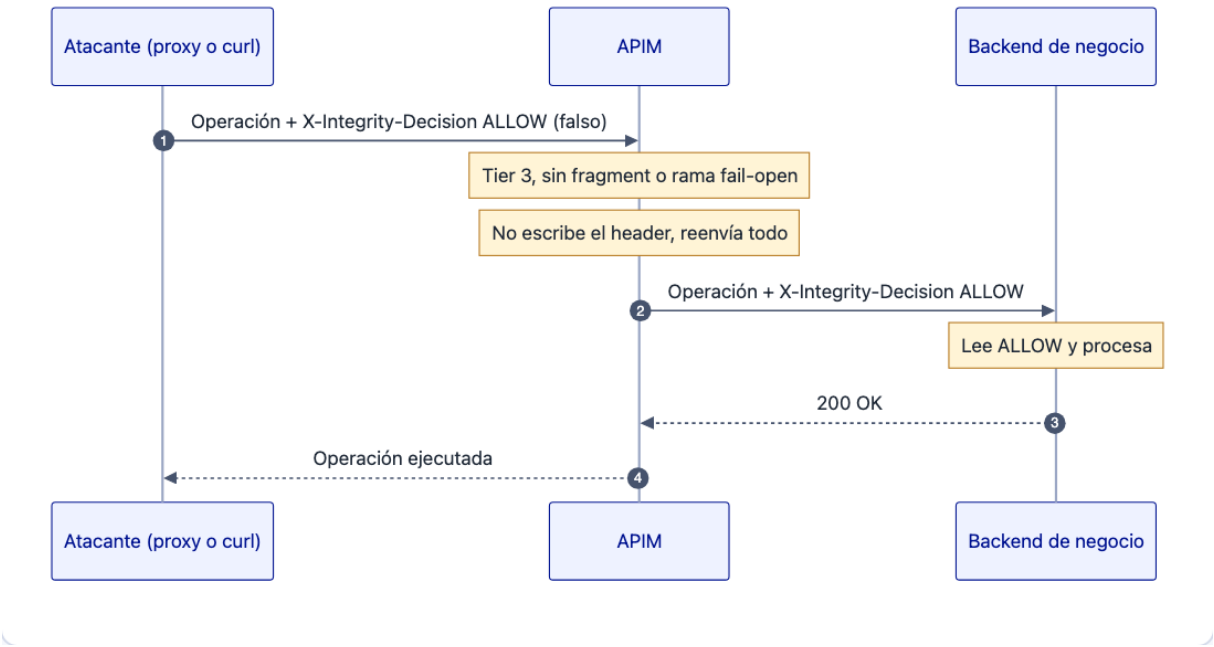


Figura 3.2 — El cliente se manda ALLOW a sí mismo.

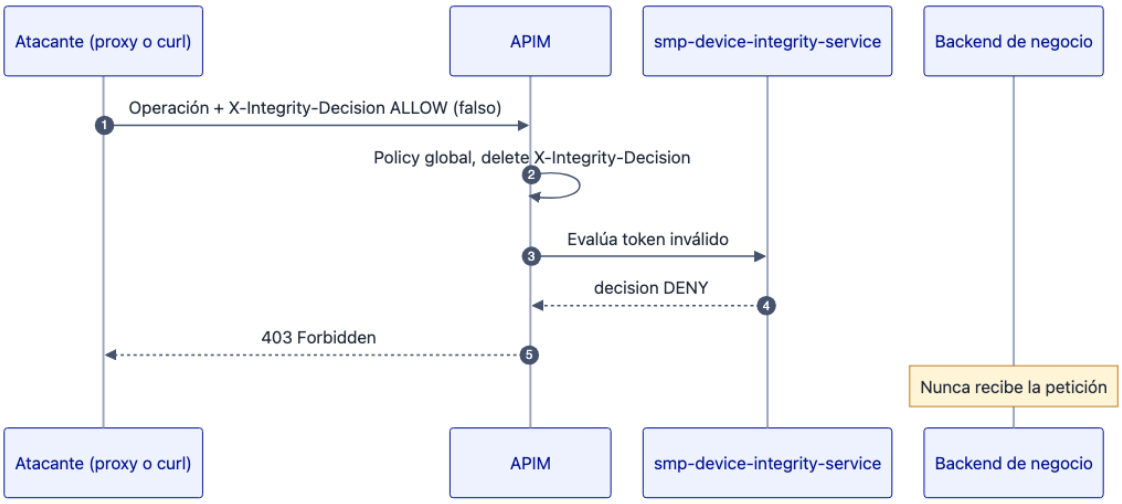


Figura 3.3 — APIM borra el header entrante y escribe el suyo con override.

Ataque 2: Acceso directo al backend sin pasar por APIM

El backend de negocio y el servicio de integridad se encuentran dentro de la red interna. Por ello, este escenario no representa un acceso directo desde Internet, sino un posible acceso desde otros componentes o puntos de la infraestructura, como otro pod, una conexión VPN, un BFF o una exposición accidental del servicio.

Un componente comprometido o una ruta de comunicación no controlada podría invocar directamente al backend y enviar un header X-Integrity-Decision: ALLOW sin haber pasado por APIM ni por el proceso de validación correspondiente.

Aunque este no es el escenario de ataque principal, debe contemplarse como parte de una estrategia de defensa en profundidad.

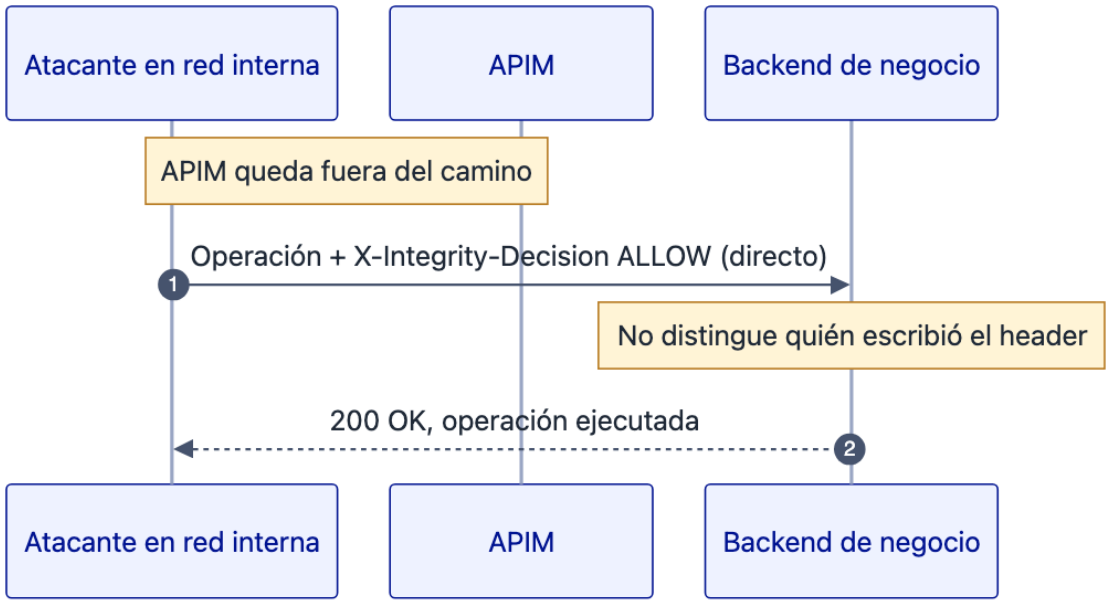


Figura 3.4 - ***Escenario de ataque: acceso directo al backend con un header ALLOW manipulado.***

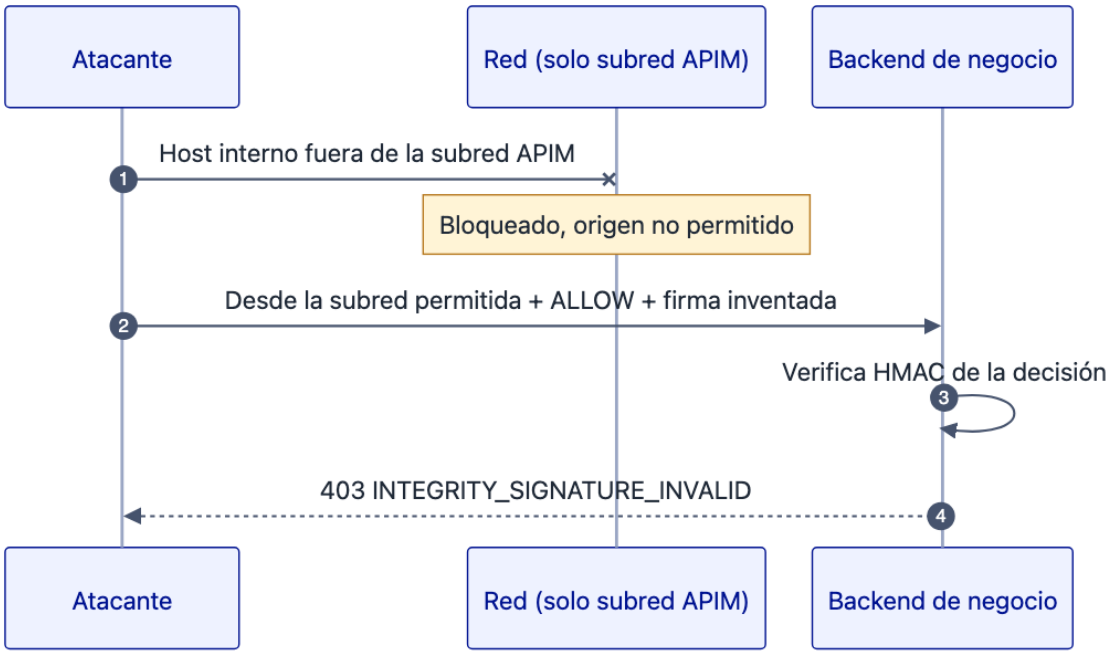


*Figura 3.5 — **Control propuesto: el backend verifica que las solicitudes provengan de APIM mediante un mecanismo de autenticación confiable.***

Postura arquitectónica recomendada

Para evitar que un cliente o componente interno pueda manipular la decisión de integridad, se recomienda implementar los siguientes controles:

1. **Eliminación de headers no confiables:** establecer una política global en APIM (All APIs) que elimine cualquier valor de X-Integrity-Decision, X-Integrity-Signature y X-Integrity-Timestamp recibido desde el cliente.
2. **Establecimiento de headers confiables:** el fragmento de política de integridad debe establecer los headers mediante exists-action="override" en todos los escenarios posibles: ALLOW, DEGRADED, DENY, SKIPPED y error. Ninguna ruta debe conservar valores proporcionados originalmente por el cliente.
3. **Firma de la decisión:** APIM debe generar una firma HMAC-SHA256 sobre una representación canónica de los siguientes datos:

   decisión | método | path | requestId | timestamp

   El secreto utilizado debe administrarse de forma segura mediante Azure Key Vault, evitando su exposición en políticas, configuraciones o logs.

4. **Verificación en el backend:** el backend debe verificar la firma antes de confiar en la decisión de integridad. Si la firma no existe, no es válida, está vencida o contiene una decisión desconocida, la solicitud debe rechazarse y clasificarse como UNVERIFIED o DENY, según corresponda. Nunca debe interpretarse como ALLOW.
5. **Restricción de acceso a nivel de red:** configurar las reglas de red para que el backend únicamente acepte conexiones desde la subred autorizada de APIM, evitando accesos directos desde otros componentes o redes no autorizadas.

Estos controles deben aplicarse de manera consistente a todas las operaciones protegidas, independientemente de su plataforma o nivel de criticidad.

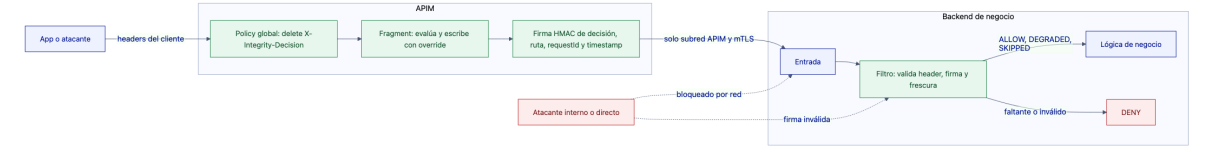


Figura 3.6 — Capas: borrar, override, red, firma.

Nota importante

APIM debe eliminar cualquier header X-Integrity-Decision, X-Integrity-Signature y X-Integrity-Timestamp recibido del cliente. Posteriormente, debe establecer la decisión de integridad mediante override y generar su firma HMAC en todos los escenarios de ejecución. El backend debe verificar la firma y su vigencia antes de aceptar la decisión, confiando únicamente en aquellas generadas por APIM.

3.2 S2 — El cliente puede seleccionar la plataforma con el control de seguridad más débil

Prioridad: bloqueante.

Descripción del hallazgo

APIM utiliza el header X-INTEGRITY-PLATFORM (o X-PLATFORM-ID, según la sección del documento) para determinar qué mecanismo de validación debe utilizar y qué política de seguridad aplicar.

El problema es que este valor es proporcionado por la aplicación móvil y, por tanto, puede ser manipulado por un cliente malicioso.

Por ejemplo, un dispositivo Android comprometido podría identificarse como Huawei para evitar la validación de Google Play Integrity. Si el control allowintegrityverifyhuawei está deshabilitado, APIM podría omitir la validación y continuar con la solicitud en estado SKIPPED.

Esto permitiría que un atacante seleccione indirectamente el mecanismo de seguridad que le resulte más conveniente, debilitando los controles establecidos para su plataforma real.

Flujo esperado y escenario de ataque

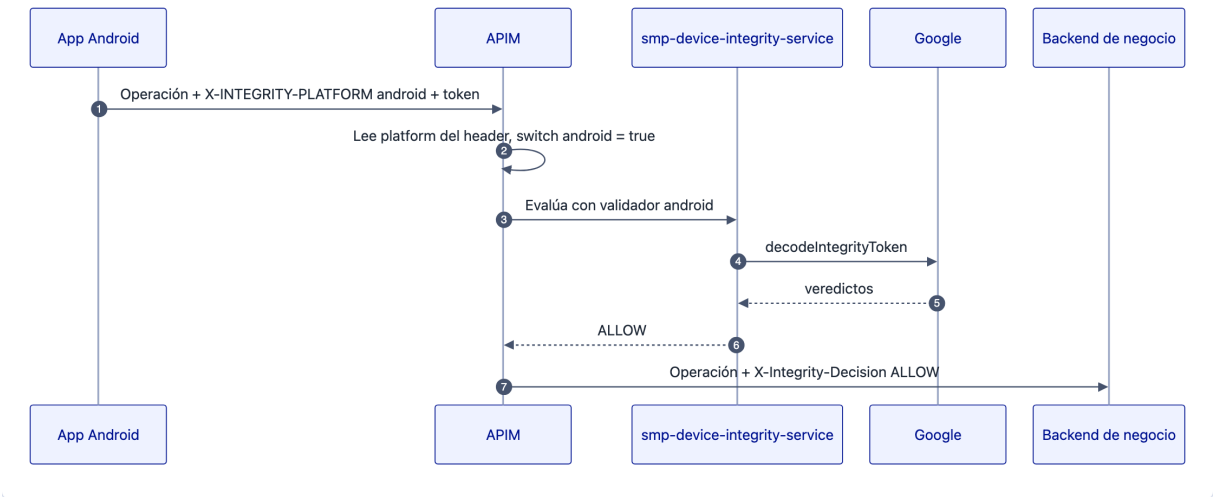


*Figura 3.7 — **Flujo esperado: la plataforma informada por el cliente determina el mecanismo de validación.***

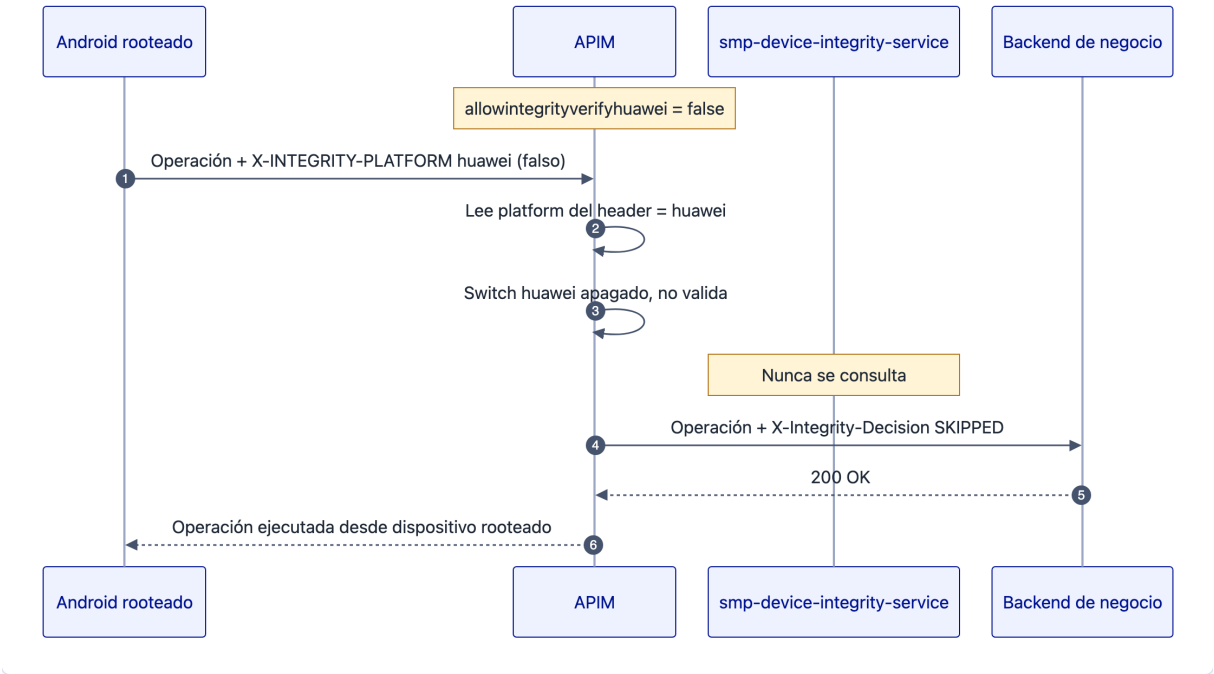


Figura 3.8 — ***Escenario de ataque: el cliente se identifica como Huawei y aprovecha que su validación está deshabilitada para obtener un resultado SKIPPED.***

Dos defensas necesarias

Defensa 1 – Validación de correspondencia entre plataforma y evidencia

El servicio de integridad debe comprobar que la evidencia recibida corresponda al proveedor de la plataforma que se está validando. Por ejemplo, un token de Google Play Integrity no debe aceptarse como evidencia de Huawei SysIntegrity ni de Apple App Attest.

Sin embargo, esta validación por sí sola no resuelve el problema: si APIM deshabilita la evaluación para una plataforma antes de invocar al servicio, nunca se realizará la comprobación de correspondencia.

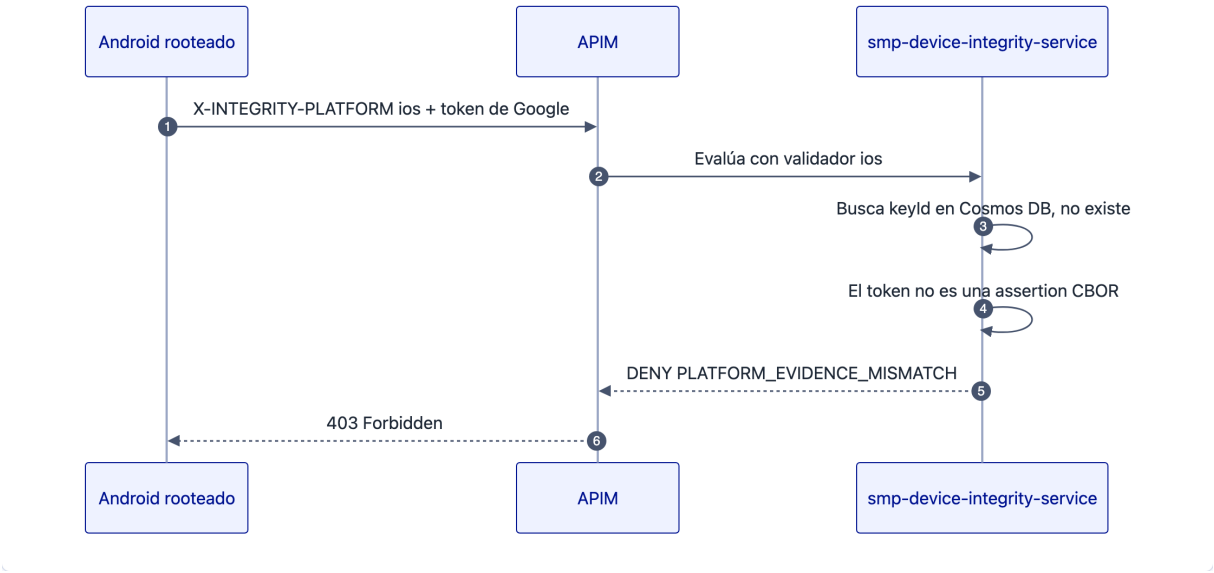


Figura 3.9 — ***Limitación de la primera defensa: la validación de la evidencia no se ejecuta cuando el switch de la plataforma seleccionada está deshabilitado.***

.

Defensa 2 – Vinculación de la plataforma con el dispositivo

La plataforma debe quedar asociada al deviceId durante el proceso de enrolamiento o registro del dispositivo, utilizando un mecanismo confiable de verificación.

APIM debe obtener la plataforma efectiva a partir de este registro o de un atributo confiable incluido en un JWT firmado y validado. La selección del mecanismo de validación y del correspondiente switch debe basarse en esa información, nunca exclusivamente en un header enviado por el cliente.

Si la plataforma informada en la solicitud no coincide con la plataforma registrada, APIM debe rechazar la operación con 403 PLATFORM_MISMATCH.

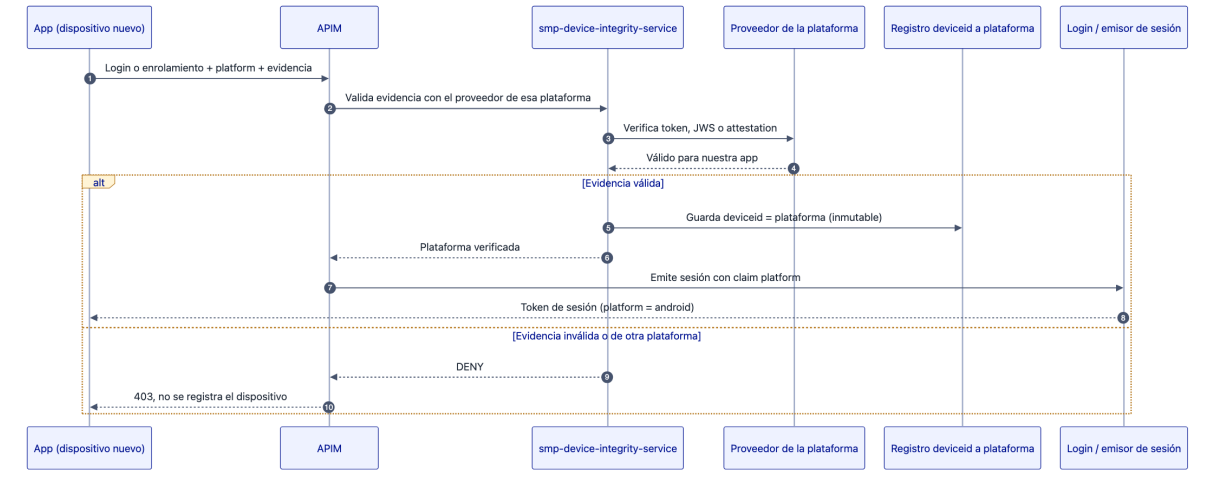


*Figura 3.10 —****Registro de la plataforma asociada al dispositivo durante el enrolamiento.****.*

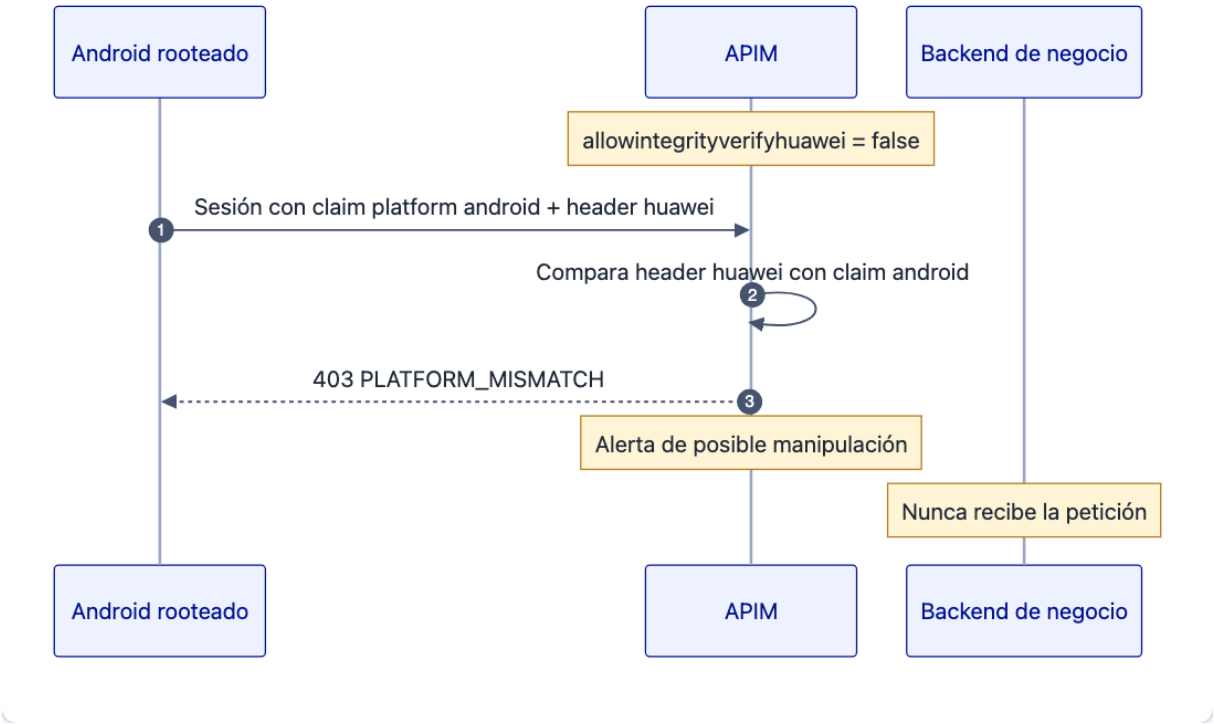


*Figura 3.11 —**Detección de discrepancia entre la plataforma informada y la registrada: rechazo 403 PLATFORM_MISMATCH***

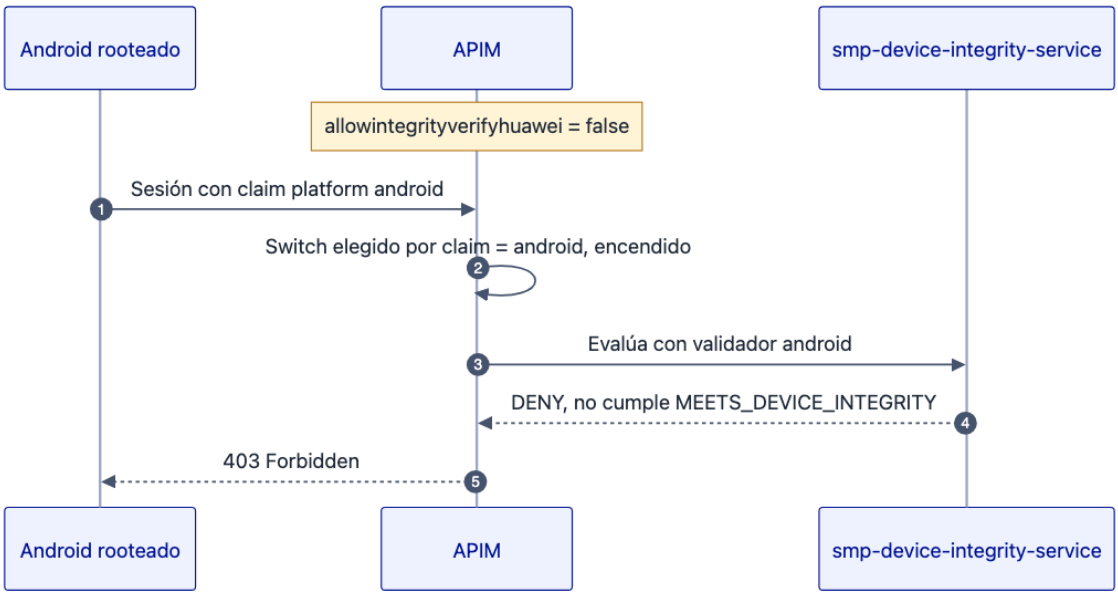


Figura 3.12 —El switch de contingencia de una plataforma únicamente afecta a los dispositivos registrados en esa plataforma.

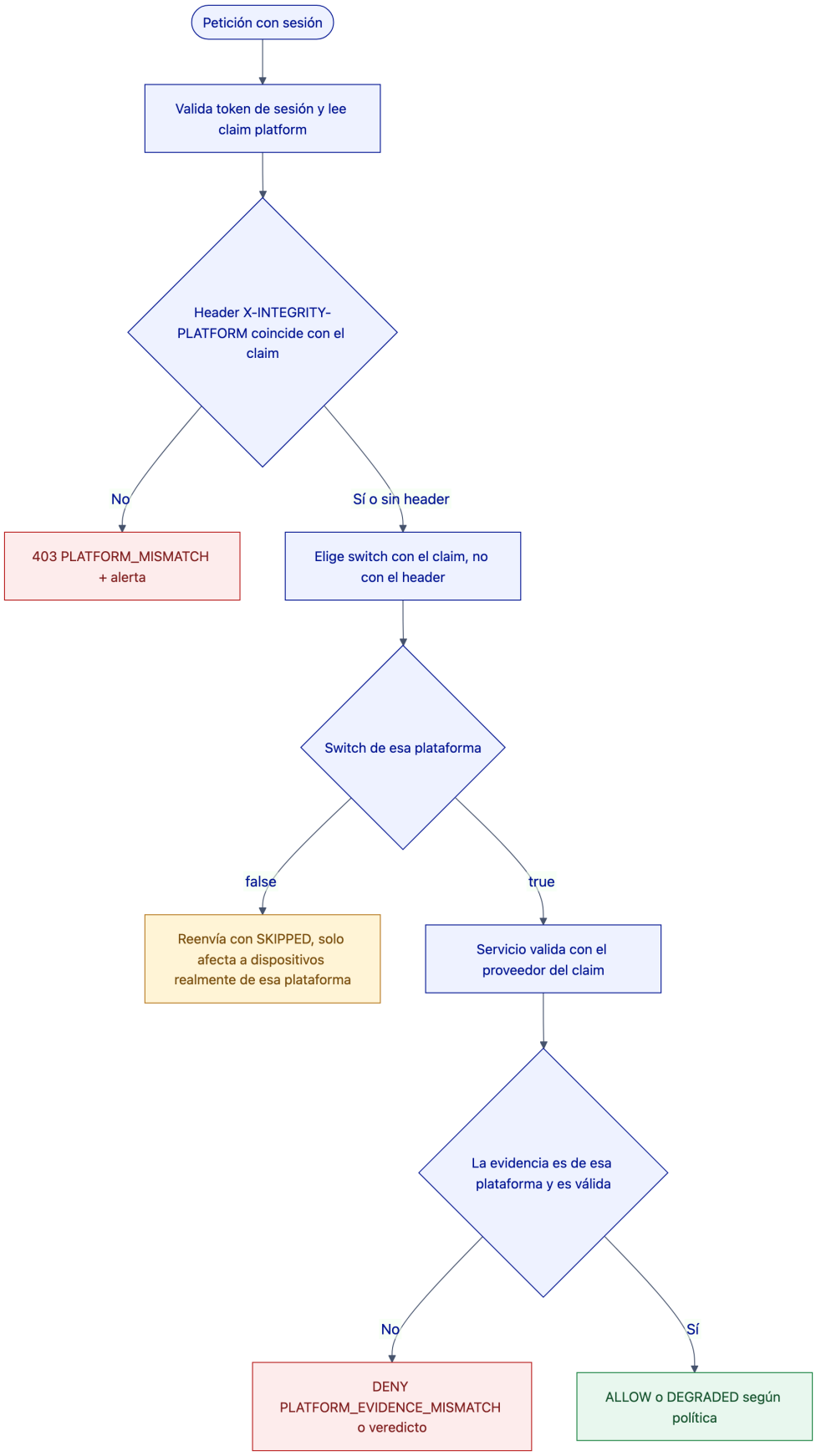


Figura 3.13 — ***Flujo de decisión: determinación de la plataforma efectiva a partir del registro confiable del dispositivo.***

Nota importante

La plataforma utilizada para seleccionar el mecanismo de validación debe obtenerse del registro confiable del dispositivo (deviceId) o de un atributo firmado y validado del JWT, nunca de un header controlado por el cliente.

Si el cliente envía X-INTEGRITY-PLATFORM, su valor debe coincidir con la plataforma registrada. Cualquier discrepancia debe generar un rechazo de la solicitud.

Adicionalmente, cada proveedor debe validar exclusivamente sus propias evidencias de integridad. La desactivación del mecanismo de validación de una plataforma no debe permitir omitir los controles correspondientes a otra.

3.3 S3 — Tier 2 no garantiza la vinculación de la evidencia con la operación

Prioridad: Bloquante

Descripción del hallazgo

La propuesta establece que las operaciones Tier 2 no requieren un *challenge* por cada solicitud. Sin embargo, esto no debe interpretarse como que la evidencia de integridad pueda utilizarse sin controles contra la reutilización (*replay attacks*).

La evidencia debe estar vinculada al contenido de la solicitud que se desea ejecutar. De lo contrario, un atacante podría reutilizar una prueba obtenida previamente para un dispositivo legítimo en otra operación o desde un entorno diferente.

Este riesgo afecta a las tres plataformas (Android, Huawei e iOS), aunque cada una requiere mecanismos específicos para garantizar esa vinculación.

Android: Google Play Integrity

Problema identificado

En el flujo original, la propuesta no contempla el uso de requestHash para las operaciones Tier 2.

Google Play Integrity permite vincular el token con el contenido de una solicitud mediante este campo. Si no se utiliza, la evidencia queda asociada principalmente al contexto de integridad del dispositivo, pero no permite comprobar que corresponde al contenido específico de la operación.

Esto abre dos escenarios de riesgo:

- **Modificación de la solicitud:** un atacante obtiene un token desde un dispositivo legítimo y posteriormente modifica el cuerpo de la petición antes de enviarla al backend.
- **Reutilización de la evidencia:** un token obtenido desde un dispositivo legítimo se reutiliza desde otro entorno, por ejemplo, un bot, mientras la evidencia todavía se encuentra dentro de su periodo de vigencia.

Escenarios representados en los diagramas

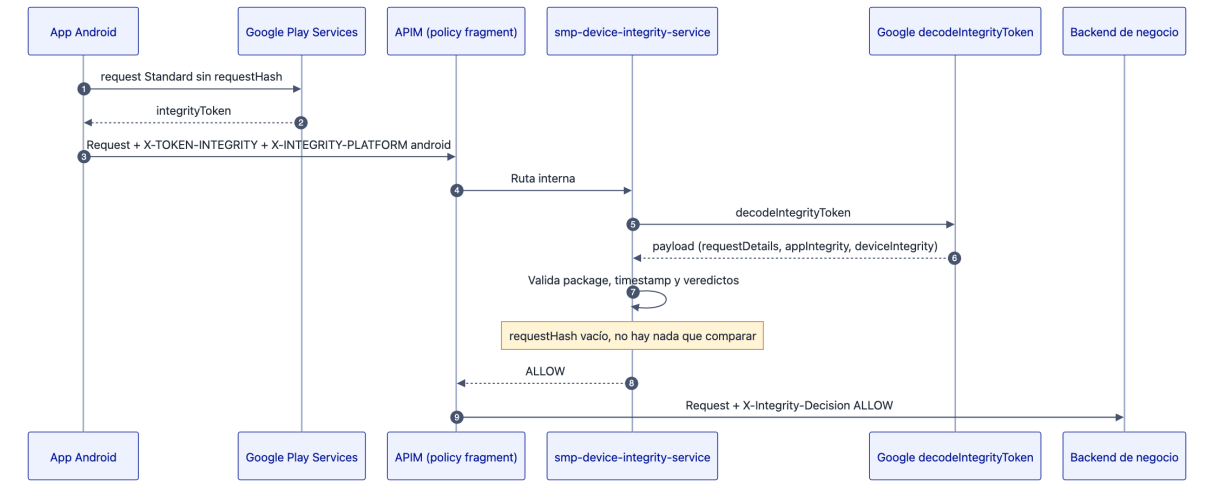


*Figura 3.14 — **Flujo original: generación del token de integridad sin vinculación mediante requestHash.***

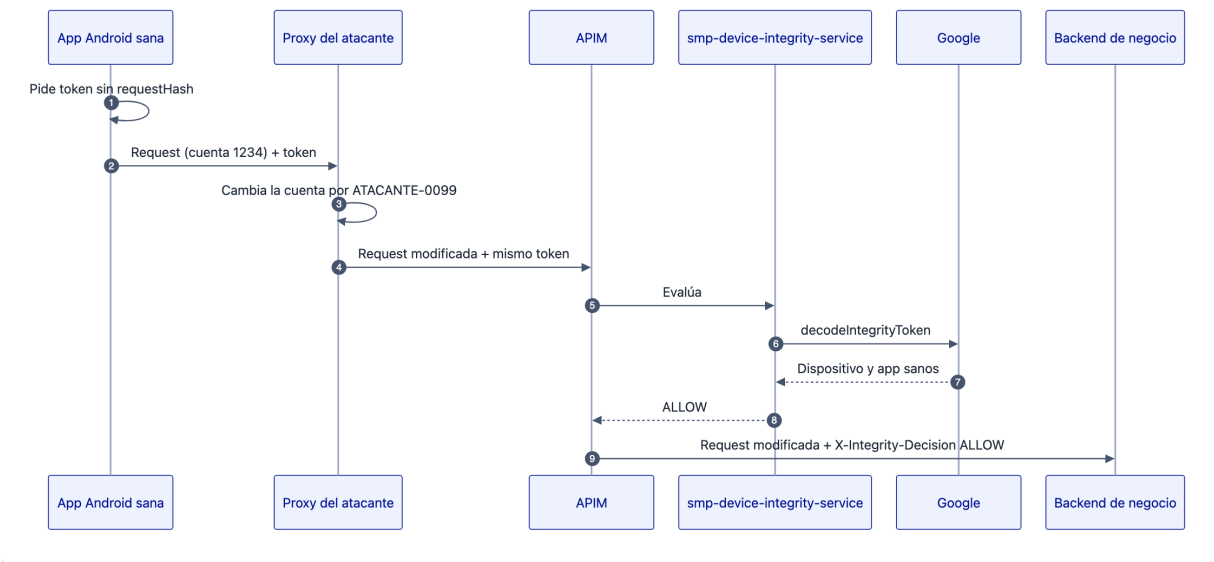


***Figura 3.15 —** Escenario de ataque: modificación del contenido de la solicitud después de obtener el token.*

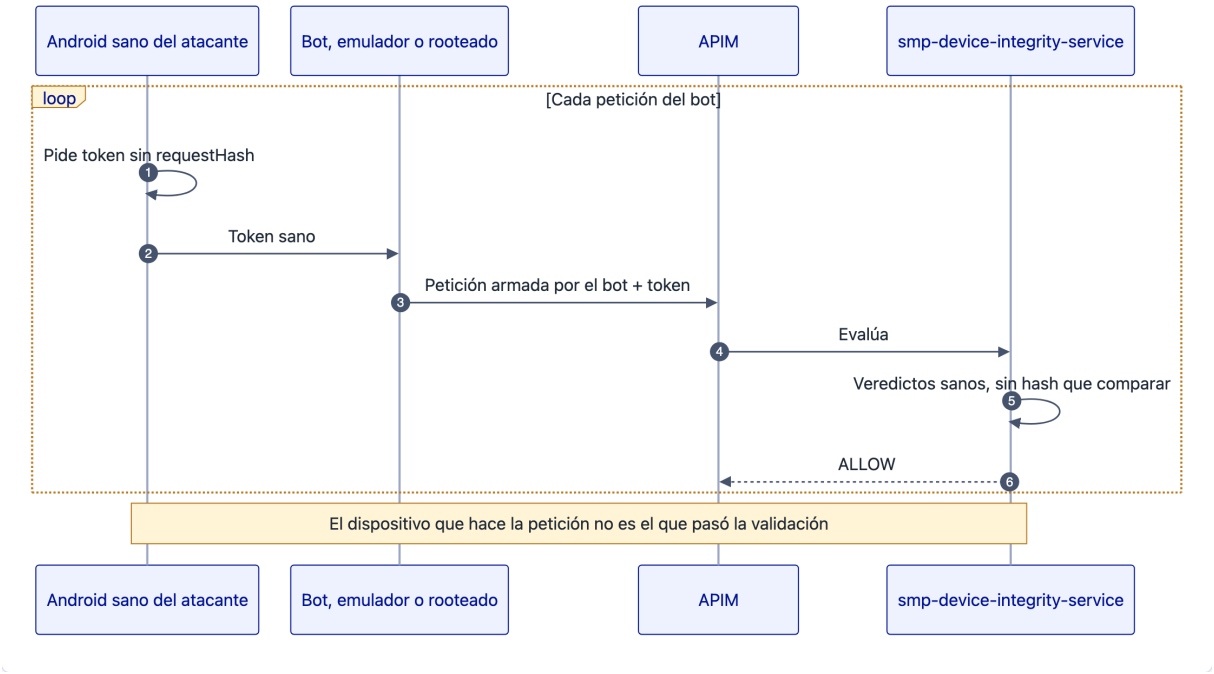


Figura 3.16 — ***Escenario de ataque: reutilización del token obtenido desde un dispositivo legítimo en una solicitud enviada desde un bot.***.

Recomendación arquitectónica

Incorporar requestHash en todas las solicitudes Tier 2 de Android, calculado a partir de una representación canónica de los datos de la operación:

requestHash = SHA256(método | ruta | SHA256(cuerpo) | deviceId | timestamp)

La aplicación debe utilizar este valor al solicitar el token de Play Integrity. Posteriormente, el servicio de integridad debe reconstruir los mismos datos a partir de la solicitud recibida y recalcular el hash para comprobar que coincide con el valor incluido en la evidencia de Google.

Adicionalmente, el servicio debe verificar:

- Que requestPackageName corresponda al paquete autorizado de la aplicación.
- Que timestampMillis se encuentre dentro de la ventana de vigencia permitida.
- Que el requestHash obtenido de la respuesta de Google coincida con el calculado en el servidor.
- Que la evidencia no haya sido utilizada previamente, mediante un mecanismo de control anti-replay.

El hash esperado debe calcularse en el servidor. No debe aceptarse como válido un hash proporcionado directamente por el cliente mediante un header u otro campo de la solicitud.

Diagramas de la solución propuesta

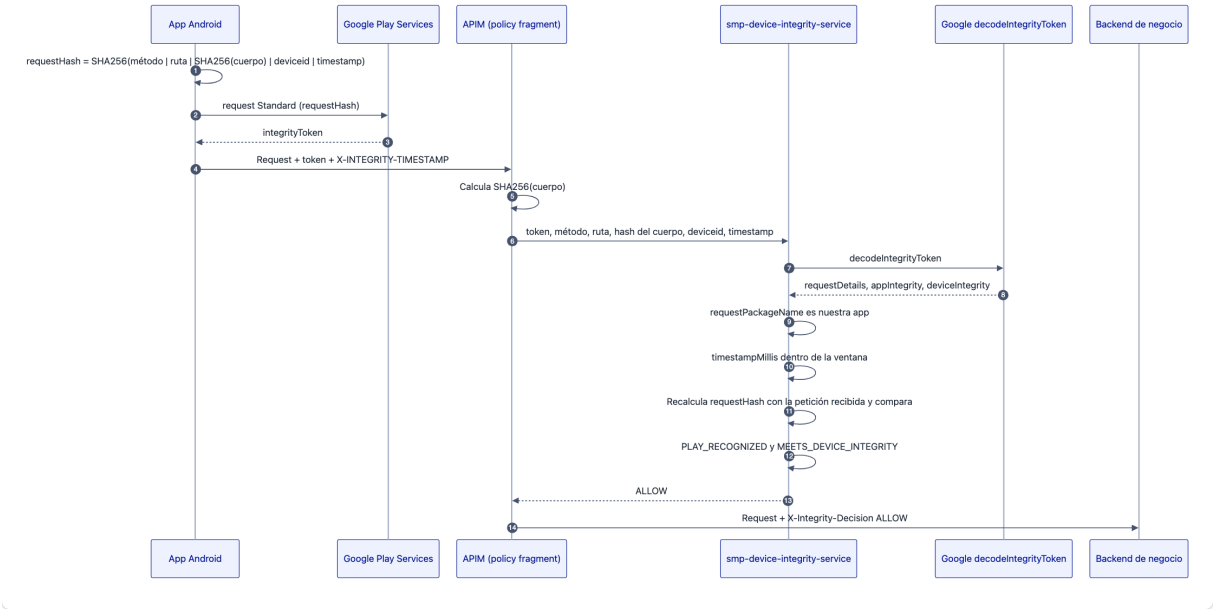


*Figura 3.17 — **Validación de requestHash recalculado en el servidor.***

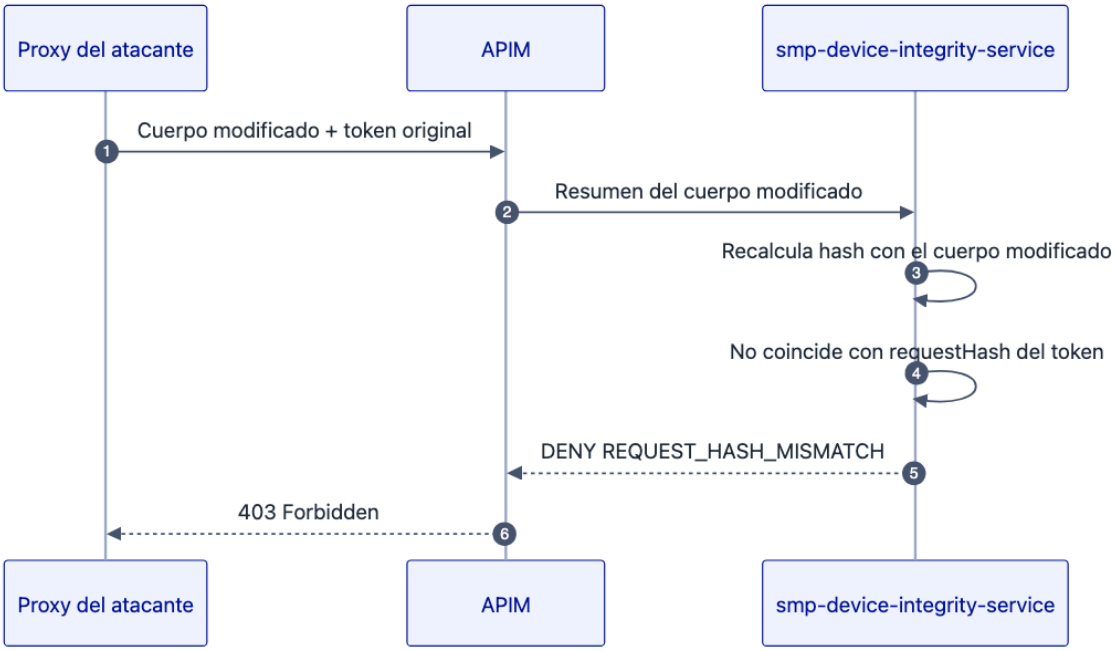


*Figura 3.18 — **Detección de modificación del cuerpo: el hash calculado no coincide con el de la evidencia.***

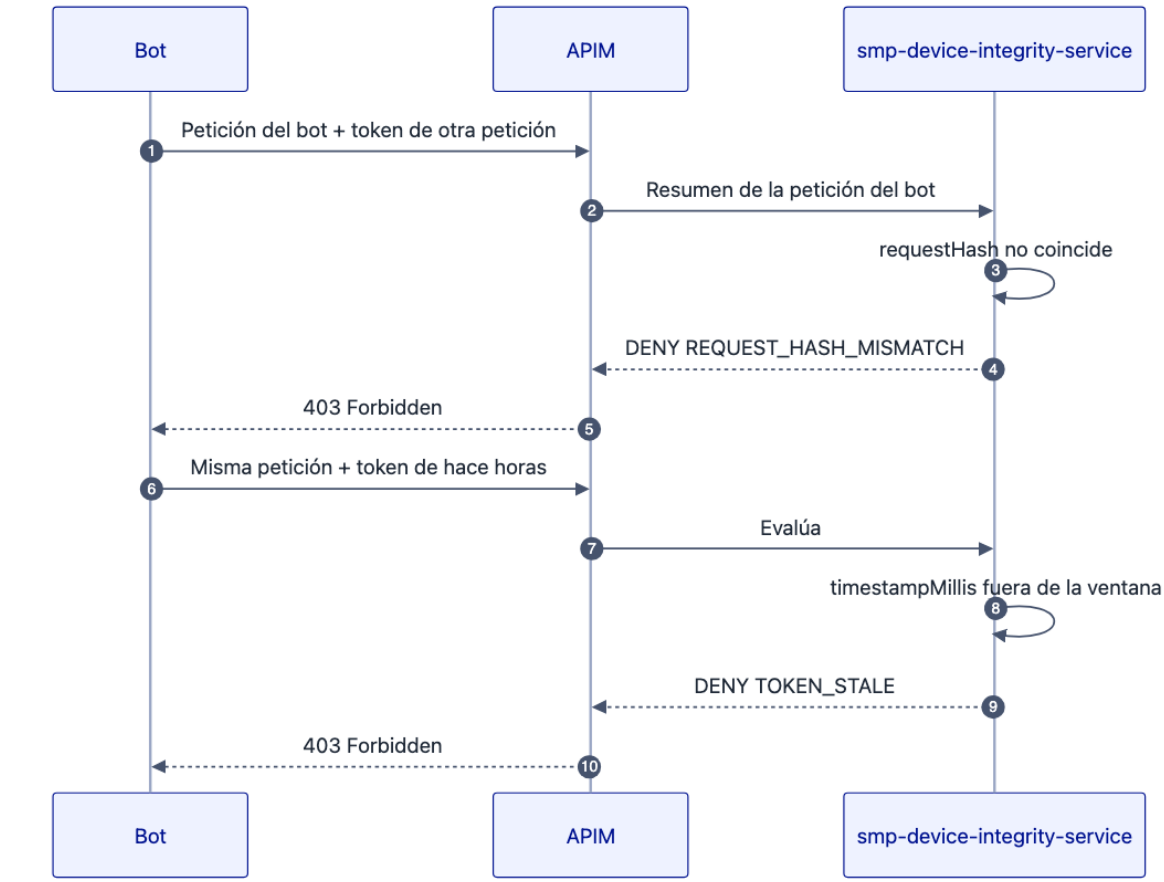


Figura 3.19 — ***Detección de reutilización: el hash de la evidencia no corresponde a la nueva solicitud.***

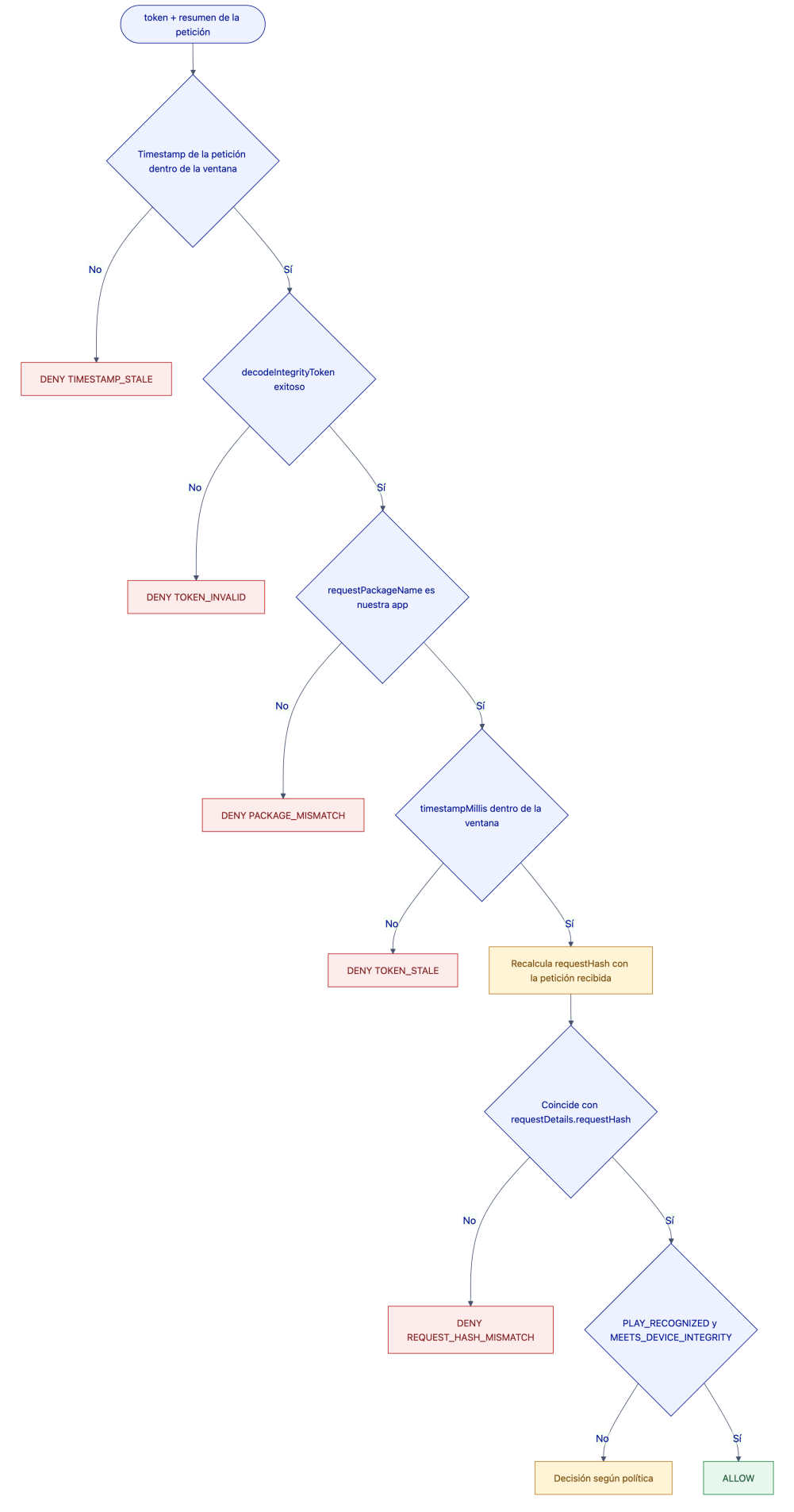

token + resumen de la petición
Timestamp de la petición dentro de la ventana
No
Sí
DENY TIMESTAMP_STALE
decodeIntegrityToken exitoso
No
Sí
DENY TOKEN_INVALID
requestPackageName es nuestra app
No
Sí
DENY PACKAGE_MISMATCH
timestampMillis dentro de la ventana
No
Sí
DENY TOKEN_STALE
Recalcula requestHash con la petición recibida
Coincide con requestDetails.requestHash
No
Sí
DENY REQUEST_HASH_MISMATCH
PLAY_RECOGNIZED y MEETS_DEVICE_INTEGRITY
No
Sí
Decisión según política
ALLOW

*Figura 3.20 — **Flujo Android Tier 2 corregido: vinculación de la evidencia con la solicitud y controles contra replay.***

Huawei: nonce local reutilizable en Tier 2

En el flujo original, el nonce es generado localmente por la aplicación y no está vinculado al contenido de la petición que posteriormente recibe el backend.

Esto permite que un JWS capturado pueda reutilizarse mientras su evidencia siga dentro de la ventana de vigencia aceptada. El servidor puede verificar que la firma es válida y que el resultado de integridad es satisfactorio (basicIntegrity), pero no dispone de un mecanismo suficiente para determinar si ese nonce ya fue utilizado.

Escenario de ataque

1. Un dispositivo obtiene un JWS válido.
2. Un atacante captura esa evidencia.
3. El atacante reutiliza el mismo JWS para enviar múltiples peticiones durante la ventana de vigencia.
4. Si el servidor no mantiene un control de uso único, puede aceptar repetidamente la misma evidencia.

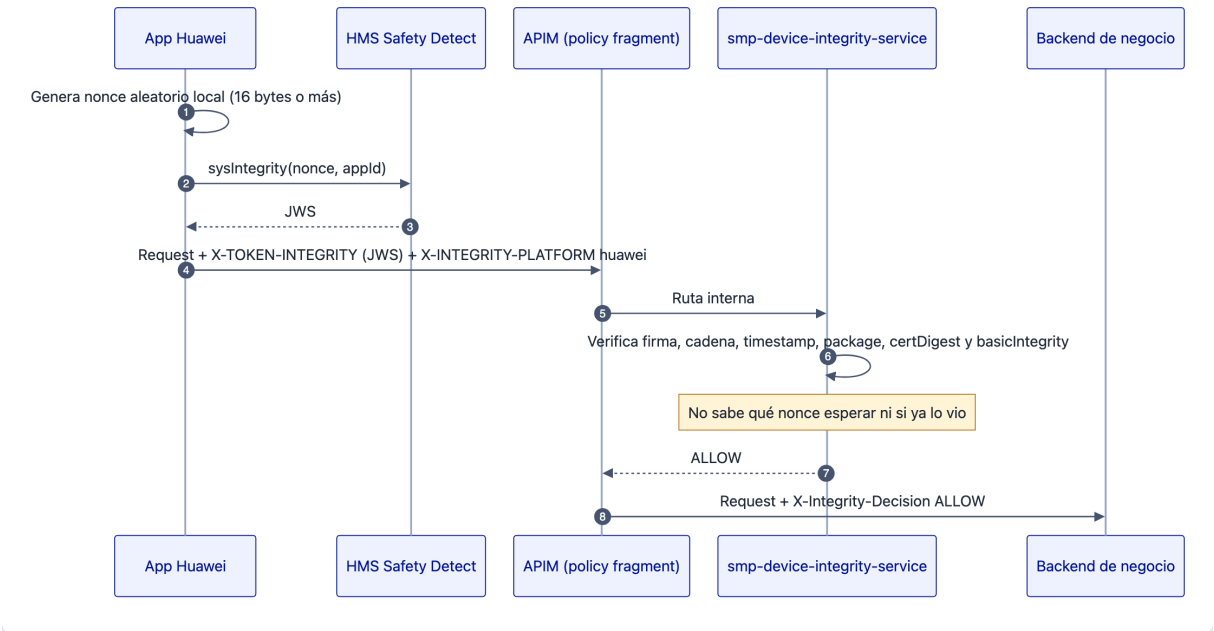


Figura 3.21 — Nonce local, sin vinculación con la petición.

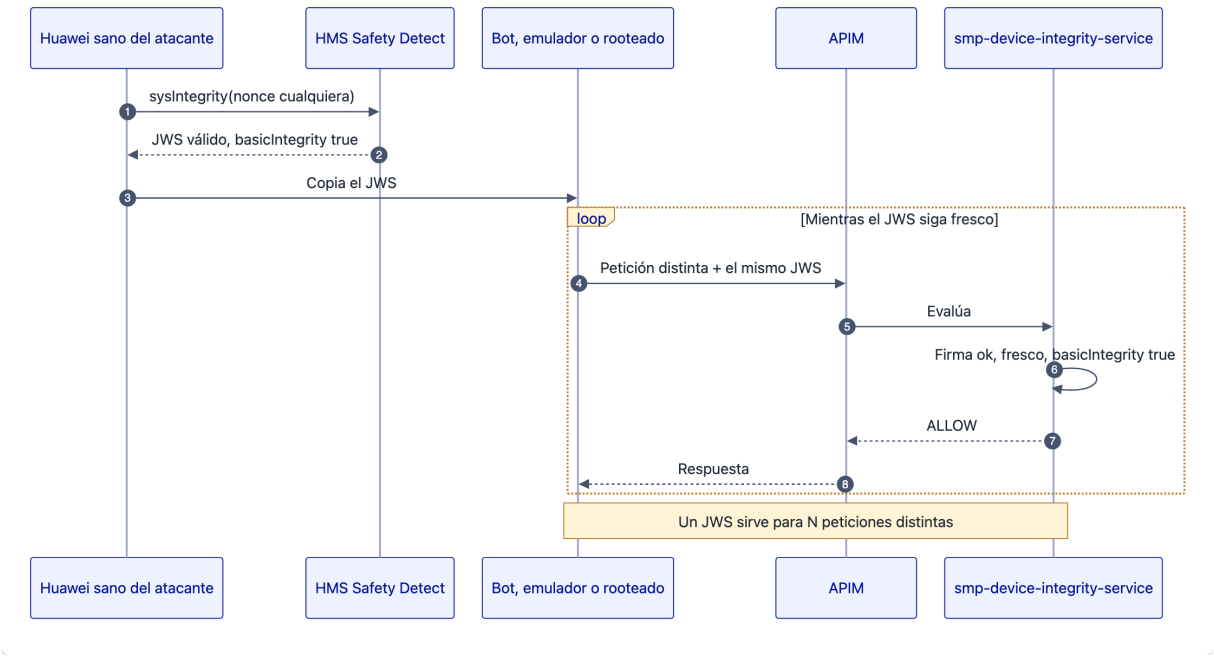


Figura 3.22 — Reutilzación del mismo JWS en múltiples peticiones.

Postura arquitectónica

El nonce debe vincularse al contenido de la petición. Para ello, se propone derivarlo de una representación canónica de la operación:

nonce = SHA256(método | ruta | SHA256(cuerpo) | deviceId | timestamp)

La aplicación calcula este valor y lo incorpora en la solicitud de evidencia a Huawei. Al recibir la petición, smp-device-integrity-service reconstruye el mismo valor a partir de los datos recibidos y verifica que coincida con el nonce incluido en el JWS.

Una vez validada la firma, el contenido de la evidencia y su correspondencia con la petición, el servicio registra el nonce en Redis mediante una operación atómica:

SET integrity:huawei:nonce:{nonce} 1 NX EX {ventana}

- NX: permite registrar el nonce únicamente si todavía no existe.
- EX: establece el tiempo de expiración, alineado con la ventana de vigencia de la evidencia.
- Si Redis confirma el registro, la evidencia puede continuar hacia la evaluación de la política.
- Si la clave ya existe, la evidencia se considera reutilizada y la petición debe rechazarse.

El registro debe realizarse después de verificar la evidencia y su correspondencia con la petición, pero antes de aceptar la decisión de integridad. De esta manera, una evidencia inválida no consume innecesariamente el nonce y dos solicitudes concurrentes no pueden utilizarlo exitosamente.

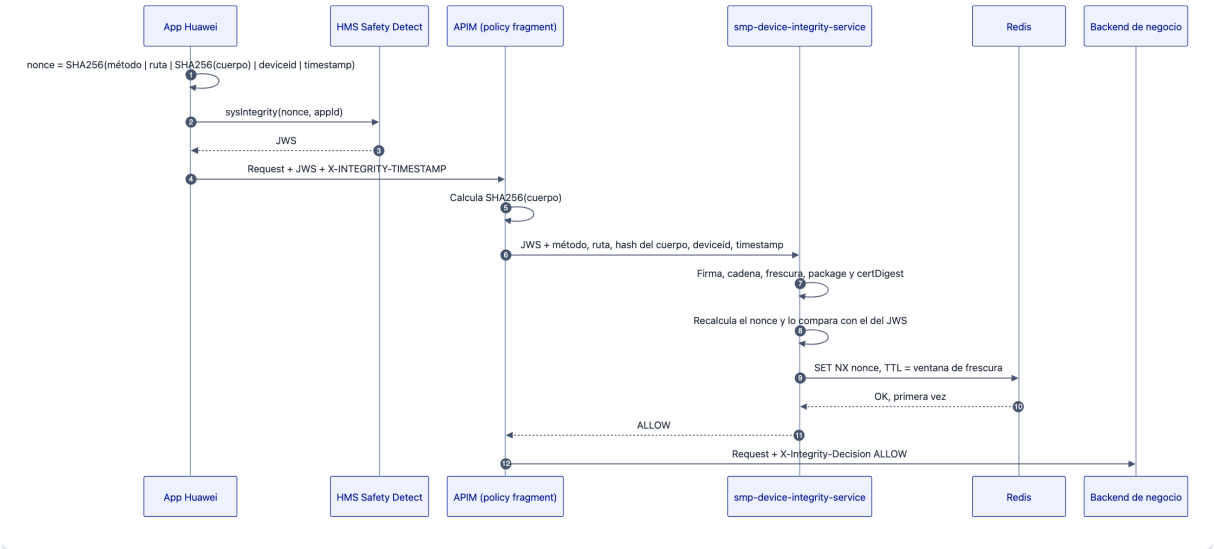


Figura 3.23 — Nonce derivado del contenido de la petición y registro de uso único en Redis.

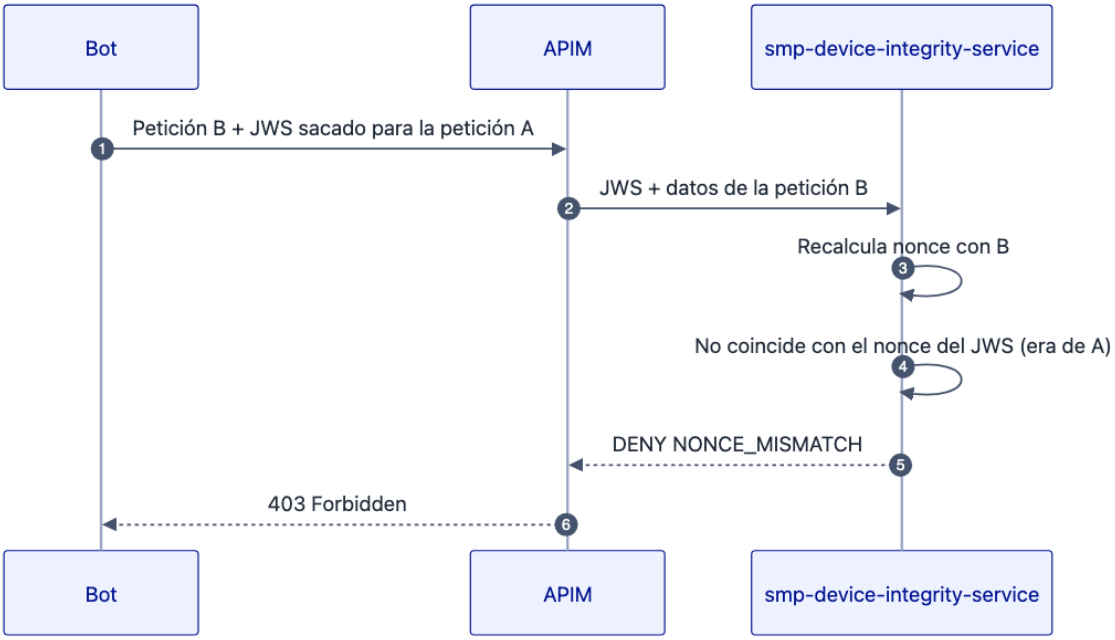


Figura 3.24 — Modificación del cuerpo: el nonce recalculado no coincide con el incluido en la evidencia.

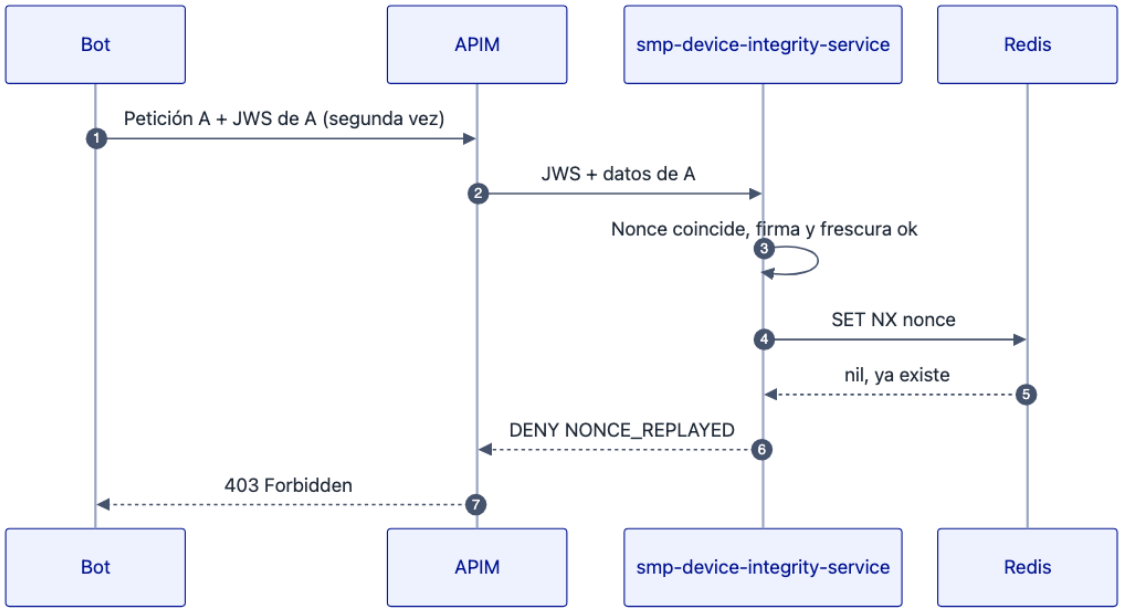


Figura 3.25 — Reutilización de la misma petición y evidencia: Redis rechaza el segundo registro mediante NX.

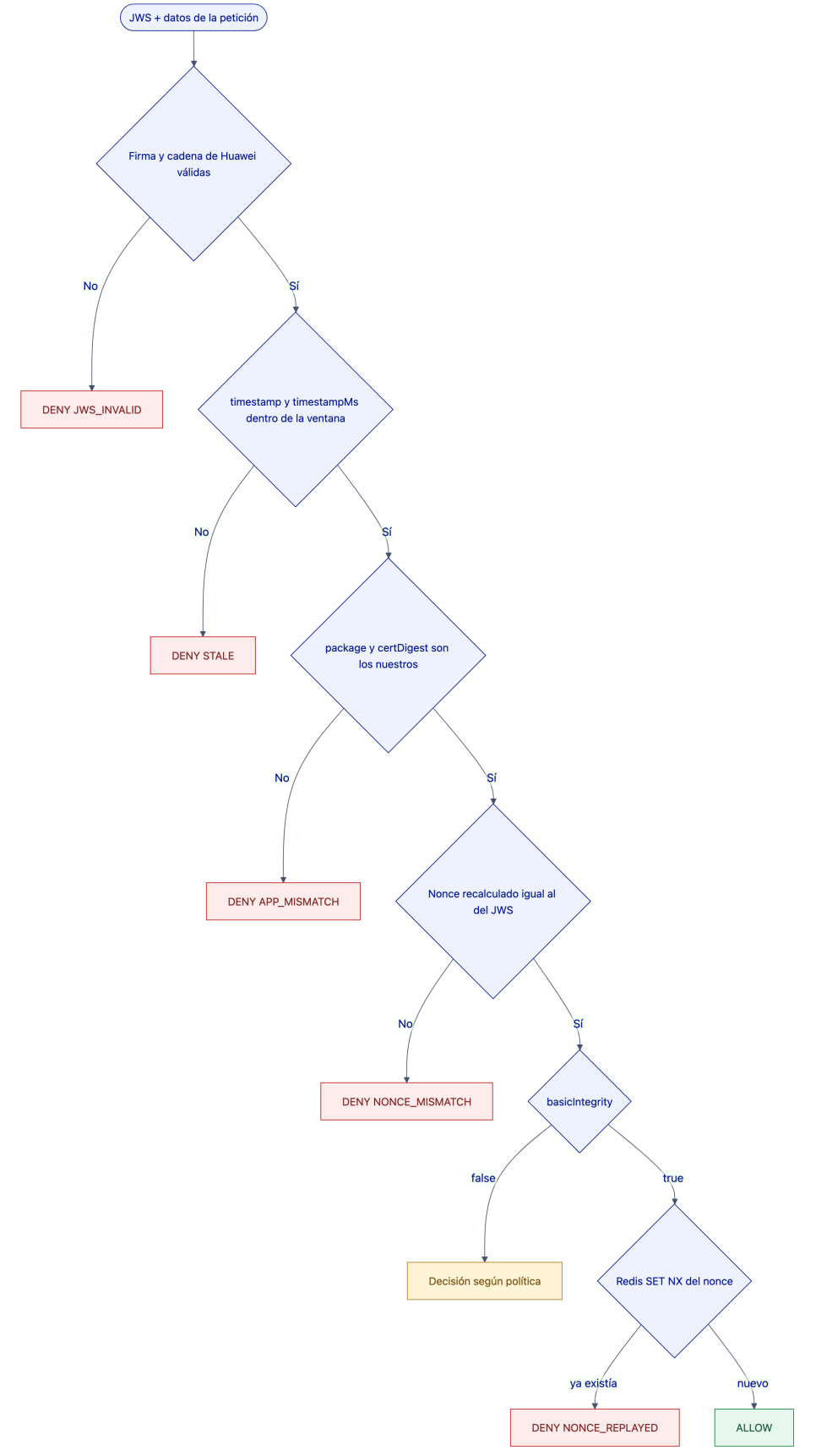


Figura 3.26 — Flujo Huawei Tier 2 corregido.

Riesgo residual

Este mecanismo dificulta la reutilización de una evidencia capturada y evita que una misma prueba se aplique a diferentes contenidos de petición. Sin embargo, no elimina un escenario de relay en tiempo real: un dispositivo legítimo podría generar evidencia para cada petición que le solicite un dispositivo comprometido.

Este riesgo residual debe tratarse mediante controles complementarios, como la vinculación de la sesión con el dispositivo enrolado, la detección de anomalías, los controles antifraude y los límites de frecuencia de solicitudes (*rate limiting*).

iOS: clientDataHash no se vincula con la petición recibida

En el flujo original, la aplicación genera una aserción de App Attest que incorpora un clientDataHash. Este valor participa en el cálculo criptográfico de la firma, junto con authenticatorData.

De acuerdo con el mecanismo de App Attest, la firma verifica el siguiente contenido:

SHA256(authenticatorData || clientDataHash)

El problema arquitectónico aparece cuando el servidor no reconstruye el clientDataHash a partir de la petición que está procesando y, en su lugar, acepta el valor enviado por el cliente, por ejemplo, mediante una cabecera HTTP.

En ese escenario, el servicio puede verificar correctamente la firma, pero no tiene garantía de que el contenido firmado corresponda al cuerpo, método y ruta de la petición recibida. La firma demuestra que la aserción fue generada por una clave válida; no demuestra, por sí sola, que la evidencia esté vinculada a la operación actual.

Escenarios de ataque

Ataque A: modificación de la petición después de generar la aserción.

1. La aplicación genera el clientDataHash y obtiene una aserción válida para una petición.
2. Un intermediario modifica el cuerpo de la solicitud antes de que llegue al servidor.
3. Si el servicio utiliza el hash enviado por el cliente sin reconstruirlo, puede verificar la firma sobre los datos originales, aunque la petición recibida tenga un contenido diferente.

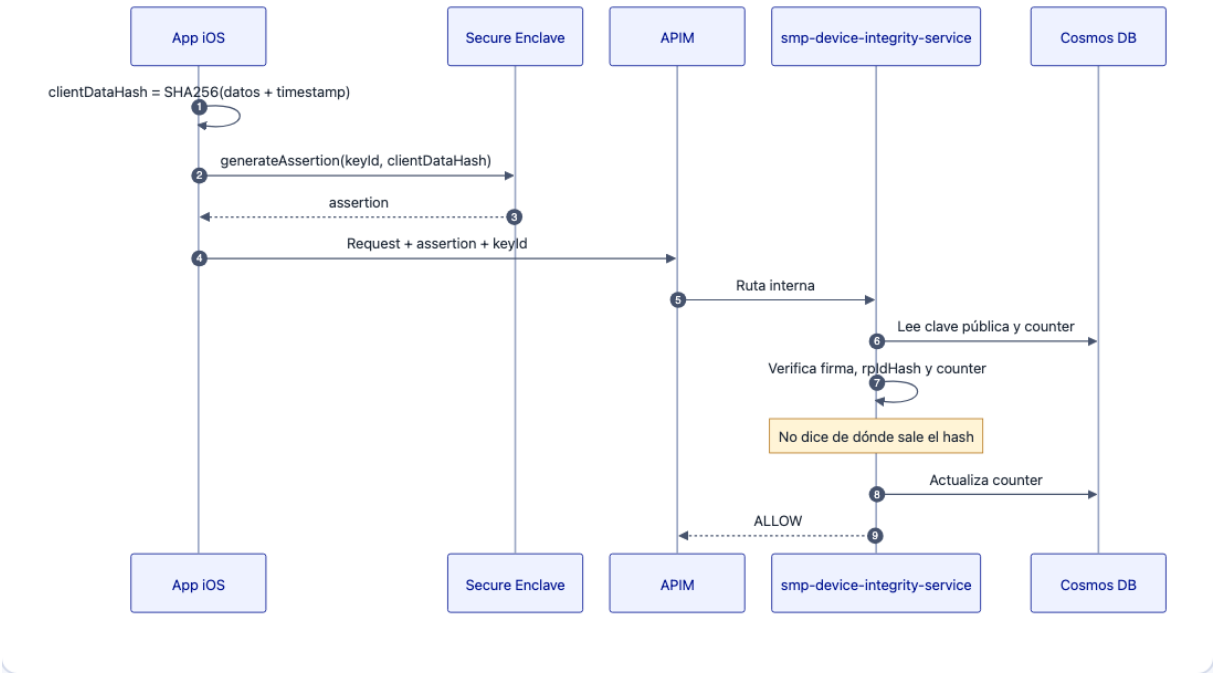


Figura 3.27 — Verificación de firma, rpIdHash y counter sin una vinculación explícita entre el hash firmado y la petición recibida.

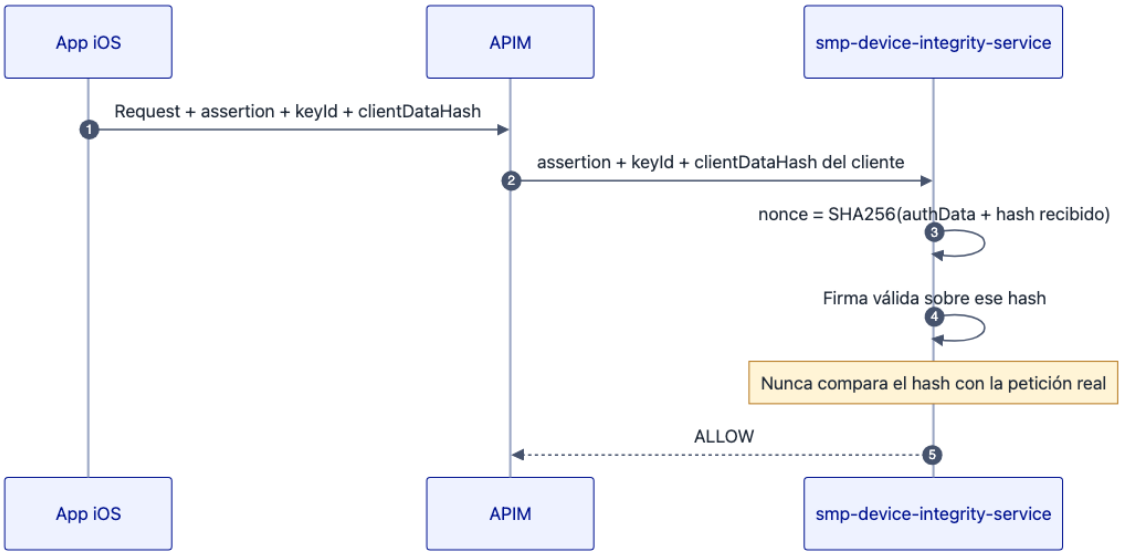


Figura 3.28 — Uso del clientDataHash proporcionado por el cliente.

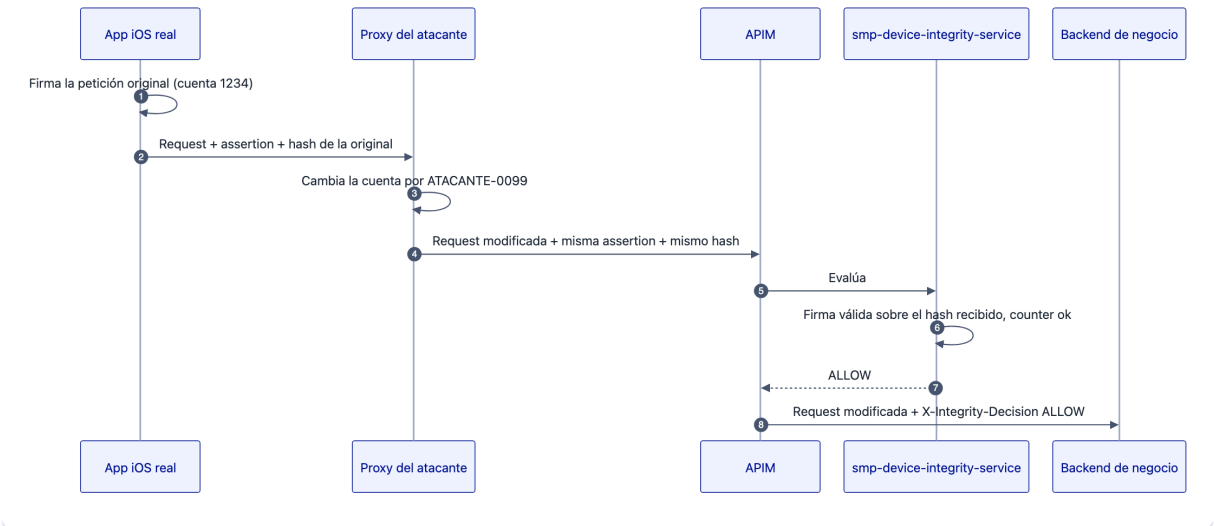


Figura 3.29 — Ataque A: modificación del cuerpo después de generar la aserción.

Ataque B: reutilización o trasplante de una aserción.

Un atacante puede intentar utilizar una aserción generada por un dispositivo legítimo en una solicitud originada desde un entorno comprometido. Si el servicio no comprueba que el hash firmado corresponde a la petición actual, existe el riesgo de aceptar evidencia válida fuera del contexto para el que fue generada.

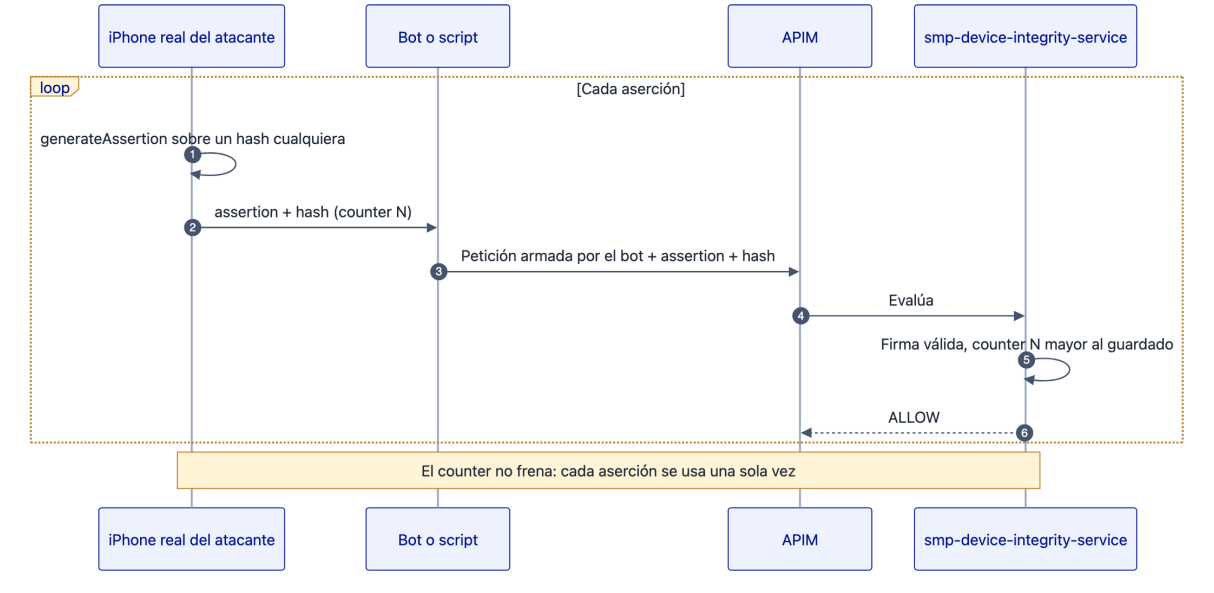


Figura 3.30 — Ataque B: intento de utilizar una aserción de un dispositivo legítimo en una petición originada desde un bot .

Postura arquitectónica

El clientDataHash debe calcularse de manera consistente entre la aplicación y el servidor, pero su validación debe realizarse utilizando exclusivamente los datos de la petición que efectivamente recibió APIM.

Se propone el siguiente flujo:

1. La aplicación construye una representación canónica de la operación y calcula el hash correspondiente.
2. La aplicación utiliza ese valor como clientDataHash al generar la aserción de App Attest.
3. APIM elimina cualquier cabecera X-INTEGRITY-CLIENT-DATA-HASH recibida desde el cliente, evitando que ese valor se convierta en una fuente de confianza.
4. smp-device-integrity-service reconstruye la representación canónica utilizando el método, la ruta y el cuerpo de la petición recibida a través de APIM.
5. El servicio calcula nuevamente el hash y verifica la aserción utilizando el valor reconstruido, junto con la clave pública registrada para el dispositivo.
6. También valida authenticatorData, el rpIdHash y el contador de la aserción. La actualización del contador debe realizarse con control de concurrencia, conforme al mecanismo descrito en la sección 4.2.

Si el hash reconstruido no corresponde al valor firmado en la aserción, la verificación debe rechazarse.

El contador permite detectar la reutilización de aserciones anteriores y contribuye a impedir ataques de repetición (*replay*). Su validación debe ser atómica para evitar que dos solicitudes concurrentes acepten el mismo estado del contador.

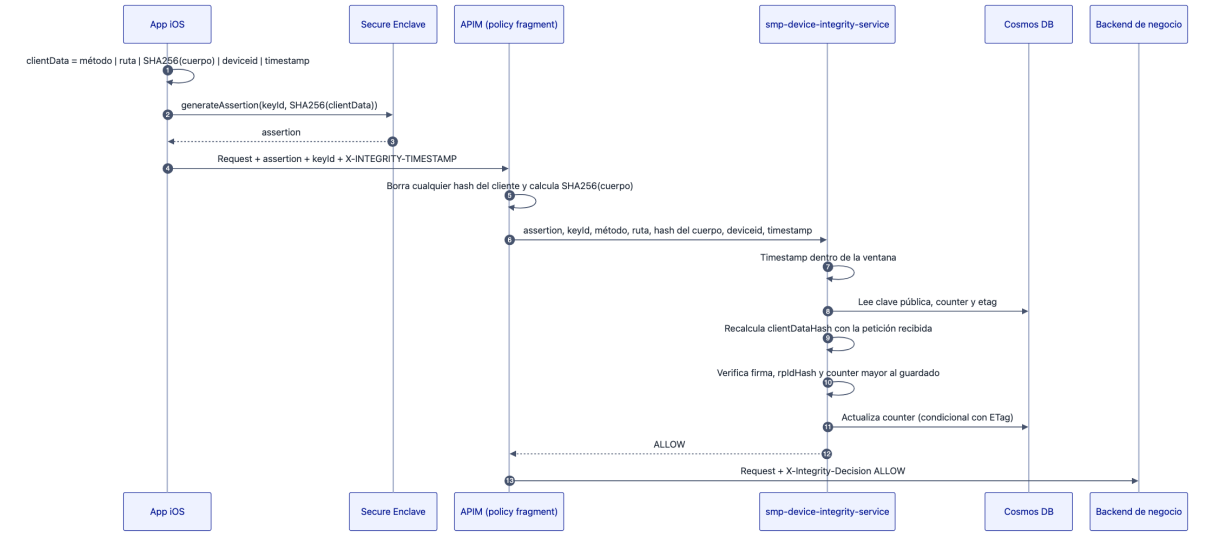


Figura 3.31 — Reconstrucción del clientDataHash en el servidor a partir de la petición recibida

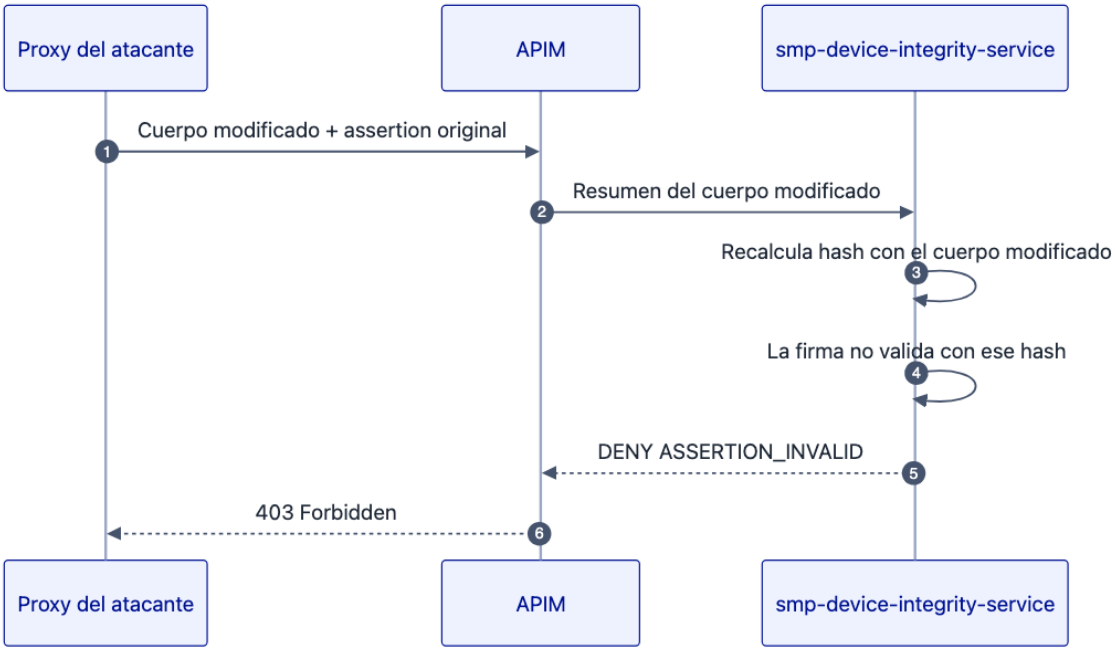


Figura 3.32 — Cuerpo modificado: el hash reconstruido no corresponde al valor firmado y la verificación falla.

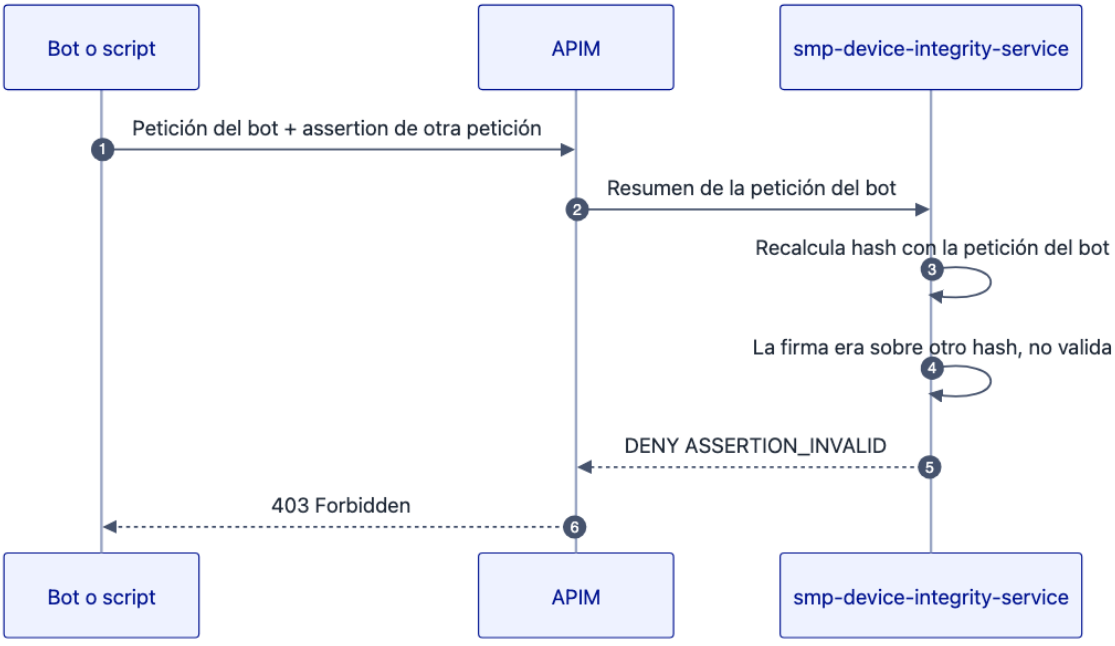


Figura 3.33 — Intento de transplante: la aserción no corresponde al contenido de la nueva petición.

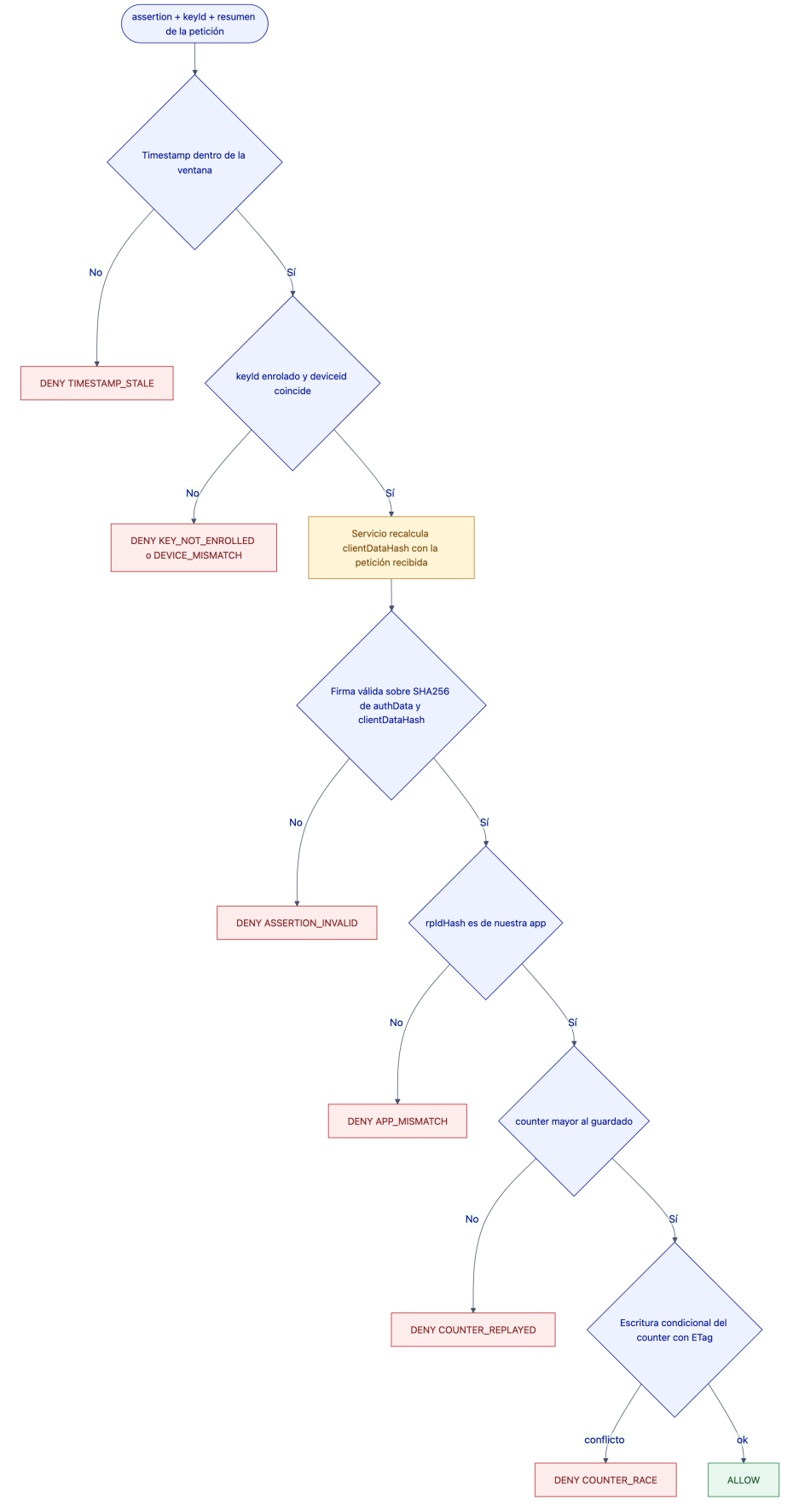


Figura 3.34 — Flujo iOS Tier 2 corregido.

4. Hallazgos de capacidad y operación

4.1 O1 — Cuota de Google Play Integrity

Prioridad: bloqueante para producción Android.

La propuesta utiliza Google Play Integrity Standard API para validar la integridad de las aplicaciones Android. Sin embargo, existe una limitación importante que debe considerarse antes de llevar la solución a producción: Google establece cuotas diarias para la generación de tokens y para su posterior descifrado.

Ambas operaciones tienen cuotas independientes, con un límite predeterminado de 10 000 solicitudes diarias por proyecto de Google Cloud.

Esto tiene un impacto directo en el diseño propuesto:

- Cada preparación del proveedor de tokens (*warm-up*) consume cuota de solicitudes.
- Cada operación que requiere validar un token genera una solicitud de descifrado en Google.
- Las operaciones Tier 1 y Tier 2 incrementan el consumo de descifrados.

Impacto en el negocio

Para una aplicación bancaria con cientos de miles de usuarios, estos límites pueden resultar insuficientes.

Por ejemplo, considerando 500 000 usuarios diarios, de los cuales el 70 % utiliza Android, tres inicios de aplicación al día y un promedio de 4,5 operaciones que requieren validación por usuario, se podrían generar aproximadamente:

- 1,05 millones de preparaciones del proveedor al día.
- 1,575 millones de solicitudes de descifrado al día.

Frente a una cuota predeterminada de 10 000 solicitudes diarias, la diferencia es considerable.

Esto significa que, si no se gestiona anticipadamente la ampliación de cuota, Google podría dejar de procesar las solicitudes de validación mucho antes de finalizar el día. En modo ENFORCE, esta situación podría impedir que los usuarios Android continúen realizando operaciones bancarias, aunque sus dispositivos sean legítimos.

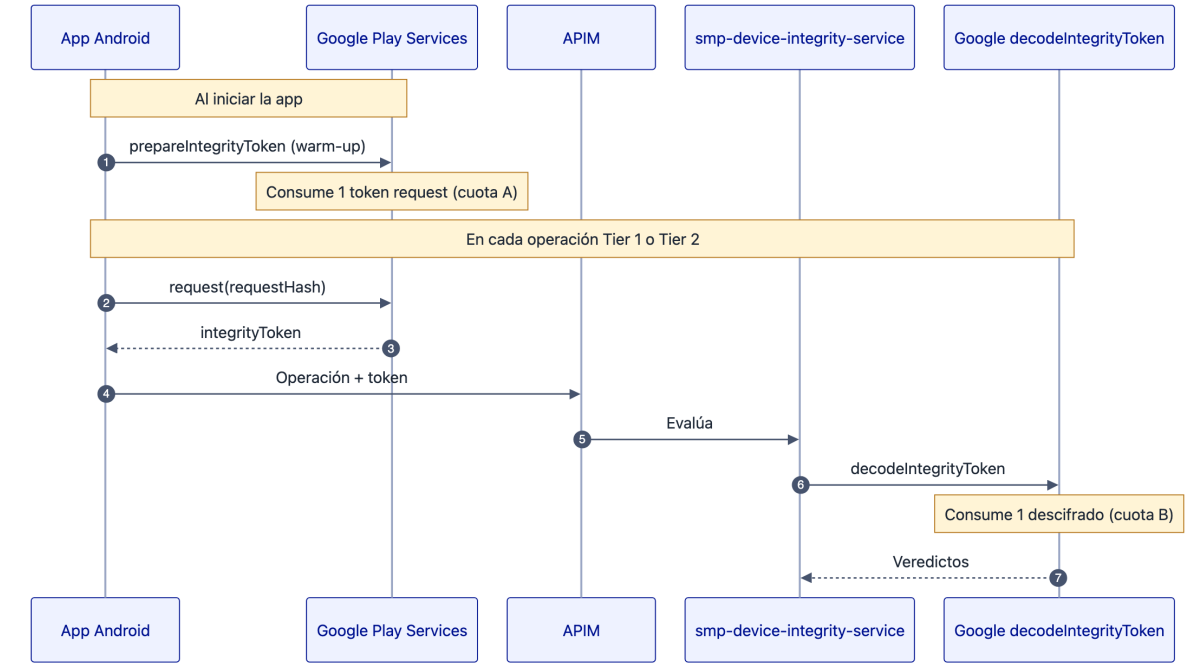


Figura 4.1 — La preparación del proveedor y el descifrado de tokens consumen cuotas diferentes.

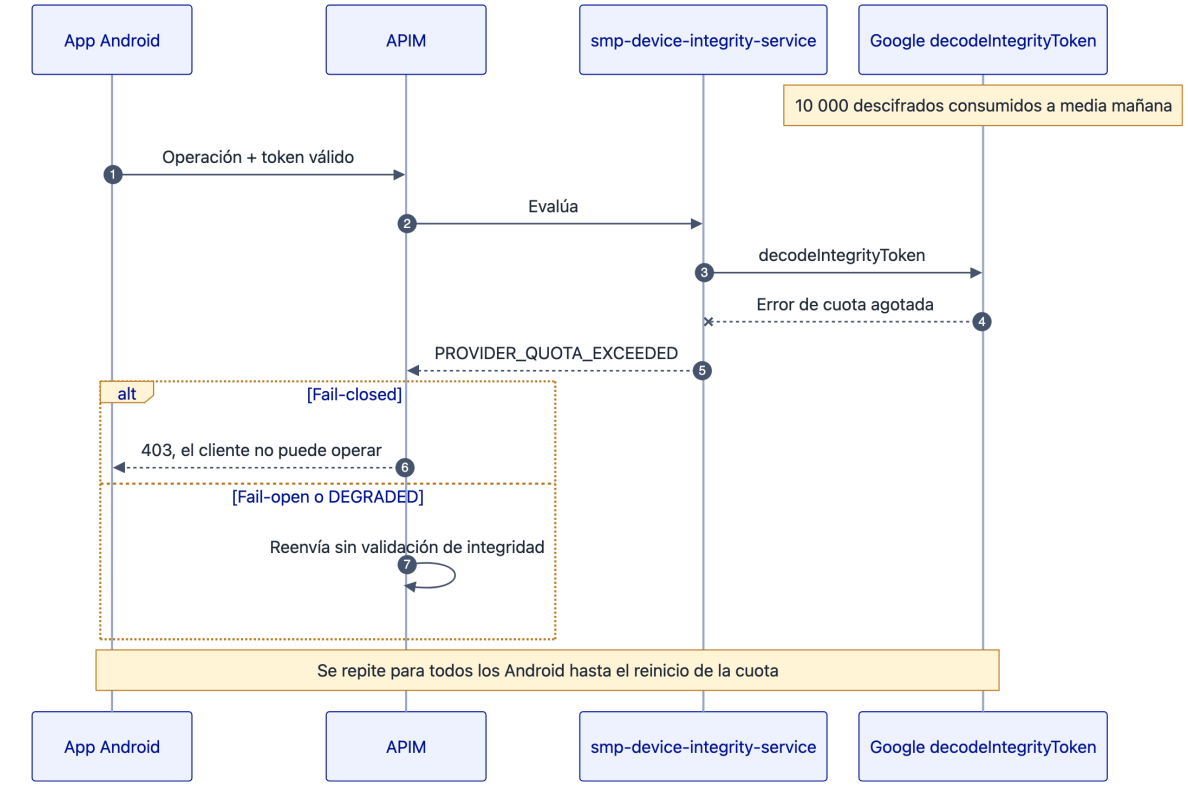


Figura 4.2 — Agotamiento de la cuota de Google y su impacto en las operaciones Android bajo ENFORCE.

Recomendación arquitectónica

La ampliación de cuota debe gestionarse antes de la salida a producción. No debería dejarse como una actividad posterior al despliegue, ya que su aprobación depende de los requisitos establecidos por Google y puede tomar varios días.

Se recomienda:

- Solicitar anticipadamente a Google el incremento de las cuotas de generación y descifrado de tokens.
- Optimizar el uso del proveedor de tokens, evitando preparaciones innecesarias.
- Medir el consumo real durante la etapa MONITOR y contrastarlo con las proyecciones de tráfico.
- Implementar alertas para anticipar el agotamiento de cualquiera de las dos cuotas.
- Habilitar ENFORCE progresivamente, una vez comprobada la capacidad disponible.

La aplicación debe exigir los veredictos MEETS_DEVICE_INTEGRITY y PLAY_RECOGNIZED, además de validar el nombre del paquete, la vigencia de la evidencia y la correspondencia del requestHash con la petición recibida.

Consideración final: el agotamiento de cuota debe tratarse como una indisponibilidad del proveedor de integridad y seguir la política de contingencia definida en la sección 6. Nunca debe interpretarse como una validación satisfactoria.



Figura 4.3 — Gestión de cuota y habilitación progresiva de Google Play Integrity.

La habilitación de Google Play Integrity debe realizarse de manera progresiva. Primero se debe asegurar la cuota necesaria, después validar el consumo y comportamiento en MONITOR y, finalmente, activar ENFORCE cuando exista capacidad suficiente para atender la demanda proyectada.

4.2 O2 — Counter iOS en Cosmos DB

Prioridad: alta.

La solución utiliza el contador (*counter*) de Apple App Attest para controlar la secuencia de aserciones generadas por un dispositivo.

Cada vez que el dispositivo genera una nueva aserción, el contador se incrementa. El servicio debe conservar el último valor aceptado en Cosmos DB y rechazar cualquier aserción cuyo contador sea menor o igual al último registrado.

El problema aparece cuando existen varias solicitudes simultáneas desde un mismo dispositivo.

Si el servicio realiza la lectura, comparación y actualización del contador como operaciones independientes, dos solicitudes concurrentes podrían leer el mismo valor anterior y considerar válida una aserción que ya está siendo procesada por otra solicitud.

Escenario de concurrencia

Por ejemplo, si Cosmos DB tiene registrado el contador 5 y llegan dos solicitudes con el contador 6, ambas réplicas del servicio podrían leer el valor 5 y concluir que la nueva aserción es válida.

Sin un mecanismo de concurrencia, las dos solicitudes podrían ser aceptadas, permitiendo reutilizar una misma aserción.

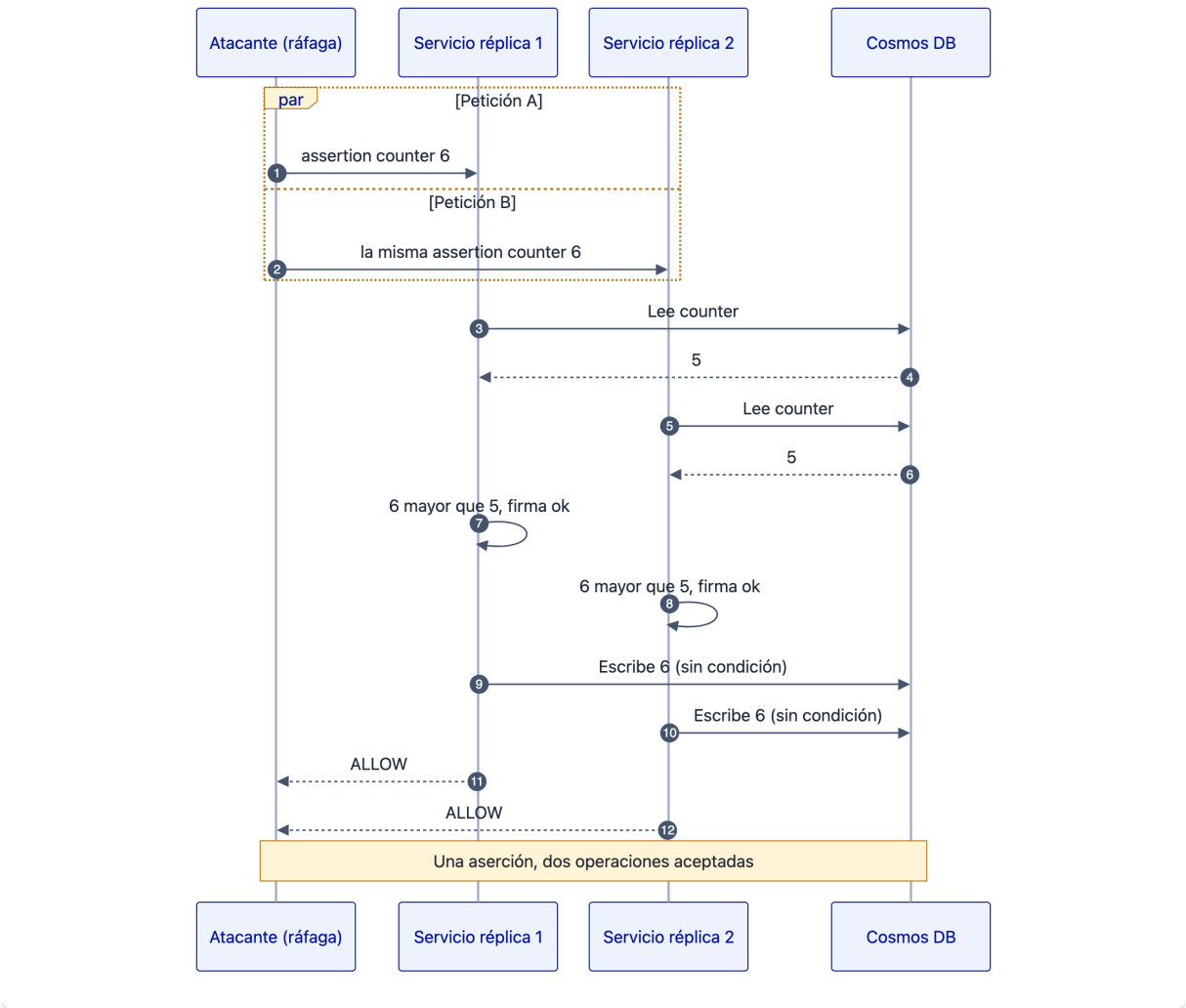


Figura 4.4 — Dos solicitudes concurrentes aceptan la misma aserción con counter 6.

Además, una actualización posterior podría sobrescribir el valor más reciente del contador con uno anterior. Esto genera un retroceso del estado almacenado y abre la posibilidad de que una aserción previamente utilizada vuelva a ser aceptada.

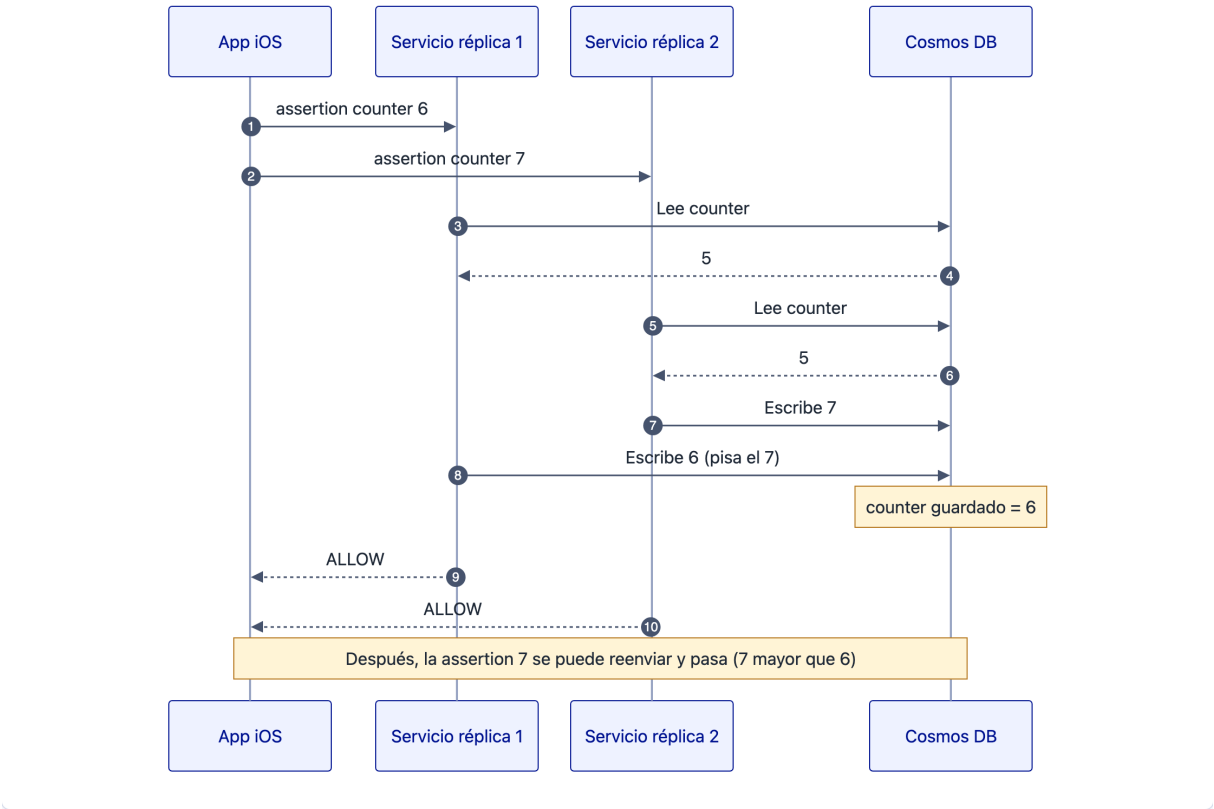


Figura 4.5 — Escritura concurrente que provoca la pérdida de la actualización más reciente del counter.

Recomendación arquitectónica

La actualización del contador debe realizarse mediante un mecanismo de concurrencia optimista que garantice que solamente una solicitud pueda modificar el estado a partir de una misma versión del documento.

Para ello, se recomienda utilizar el _etag de Cosmos DB mediante una operación replaceItem con la condición If-Match, o una actualización condicional que compruebe que el contador almacenado continúa siendo menor que el recibido.

Si otra solicitud modifica previamente el documento, Cosmos DB rechazará la actualización por conflicto de versión (HTTP 412). El servicio debe interpretar este resultado como una posible reutilización de la aserción y responder con COUNTER_REPLAYED, sin aceptar la operación.

Adicionalmente, se recomienda:

- Utilizar keyId como clave de partición para mantener el estado del contador asociado a cada clave enrolada.
- Mantener una única región de escritura para este mecanismo. En configuraciones con múltiples regiones de escritura, el control mediante ETag por sí solo no garantiza la exclusión global requerida.
- Durante el enrolamiento, utilizar createItem en lugar de upsert, para evitar sobrescribir accidentalmente un registro de dispositivo existente. Un conflicto 409 debe tratarse como un enrolamiento duplicado y gestionarse explícitamente.

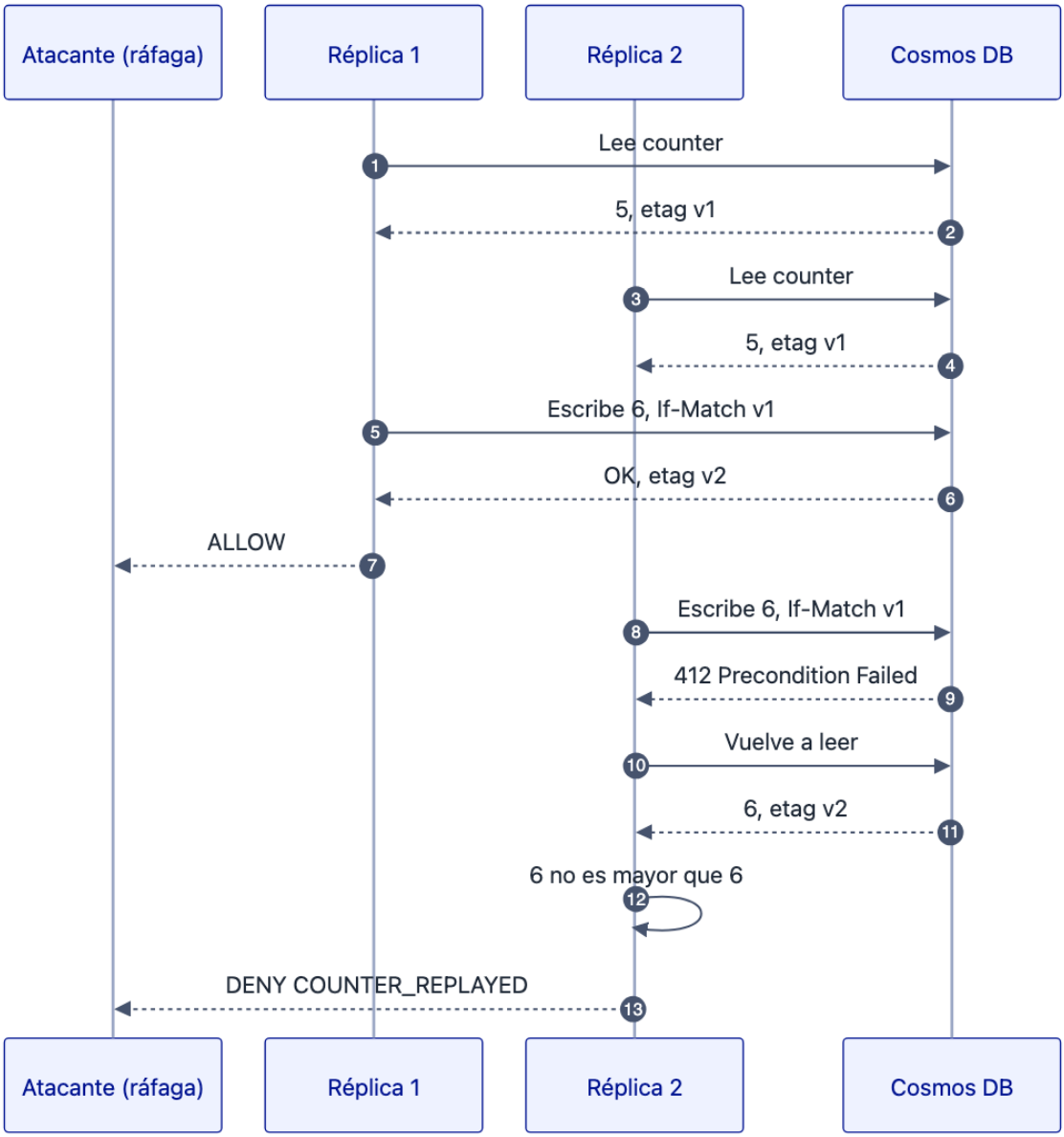


Figura 4.6 — Actualización mediante Etag: solamente una solicitud concurrente consigue modificar el documento.

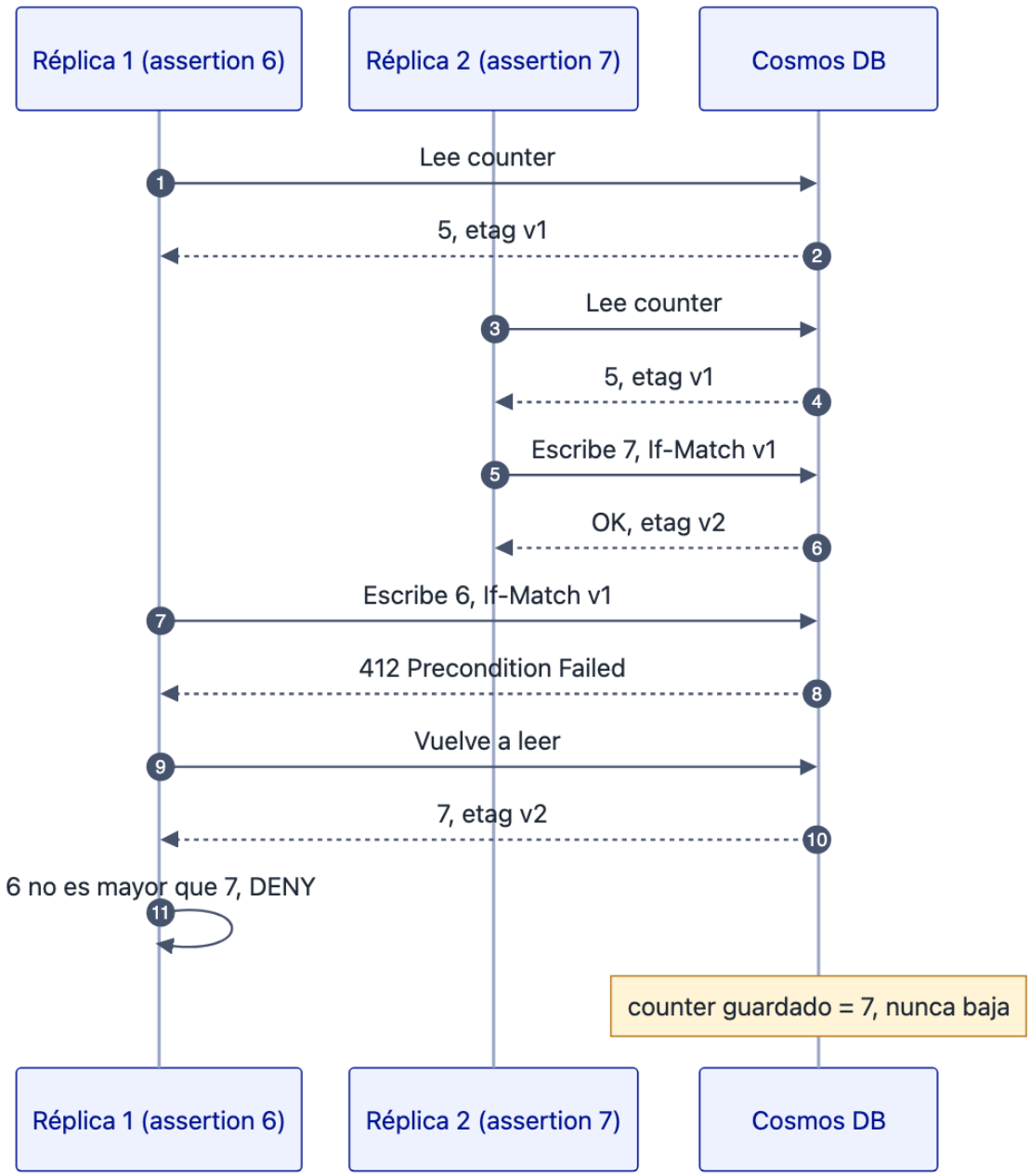


Figura 4.7 — El contador almacenado no retrocede ante solicitudes concurrentes.

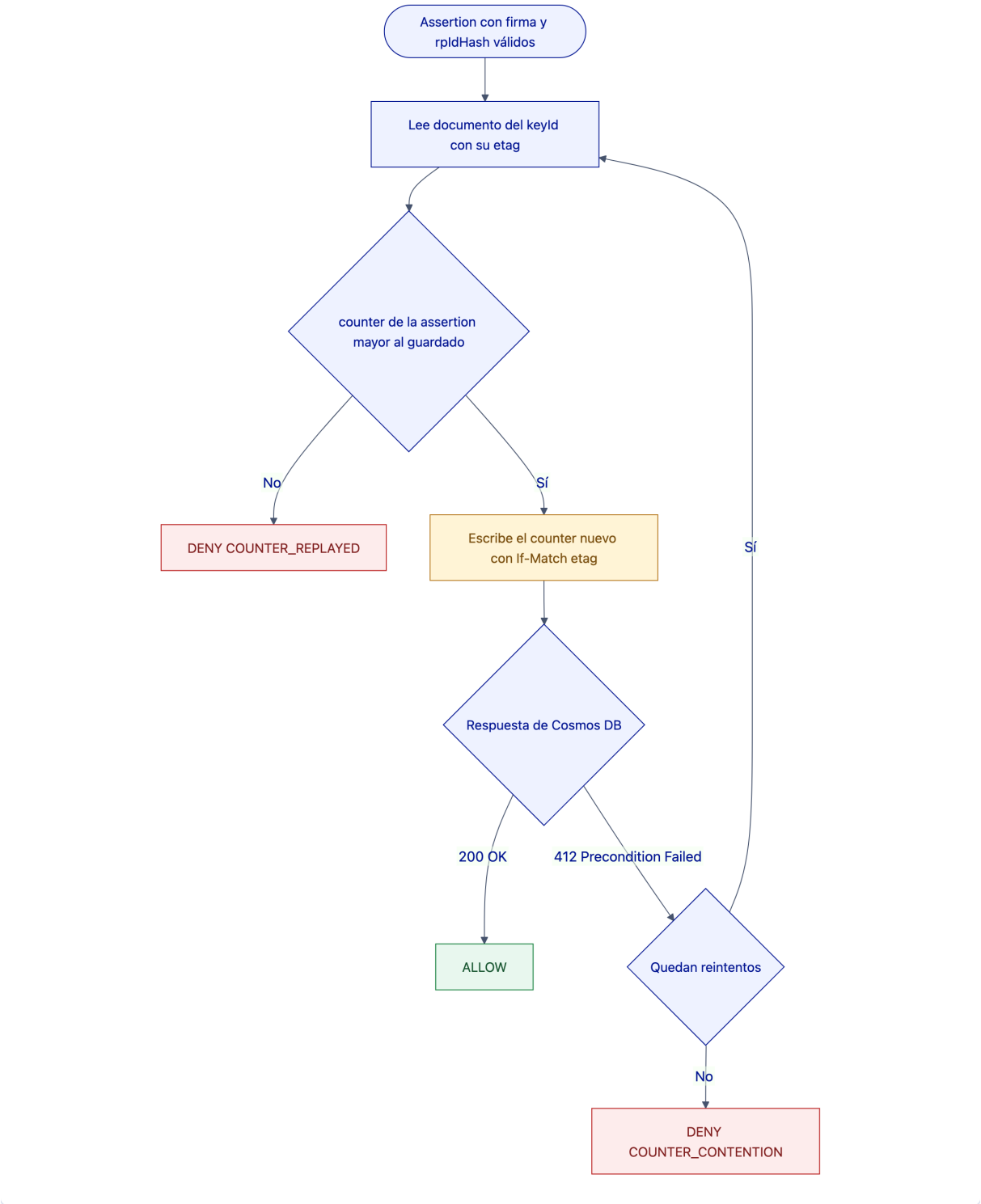


Figura 4.8 — Lógica condicional para validar y actualizar el counter.

Efecto colateral: operaciones legítimas concurrentes

Este control puede generar conflictos incluso cuando las solicitudes provienen de un dispositivo legítimo.

Por ejemplo, si una aplicación envía dos operaciones protegidas en paralelo utilizando la misma clave de App Attest, ambas pueden competir por actualizar el contador. Una de ellas podría ser rechazada

porque la otra actualizó primero el estado en Cosmos DB.

Este comportamiento debe coordinarse con el equipo móvil. Se recomienda evaluar dos alternativas:

- Serializar el envío de operaciones protegidas para evitar que varias aserciones compitan simultáneamente por actualizar el contador.
- Ante una respuesta COUNTER_REPLAYED, permitir que la aplicación genere una nueva aserción y reintente la operación una sola vez, siempre que la operación sea segura para reintentar y se controle su posible duplicidad.

4.3 O3 — Dispositivos sin App Attest

Prioridad: alta.

Apple App Attest no está disponible en todos los entornos de ejecución. La aplicación puede consultar esta capacidad mediante DCAppAttestService.isSupported, que permite determinar si el dispositivo o entorno admite el mecanismo de atestación.

Existen escenarios en los que esta capacidad no está disponible, como el simulador de iOS, determinadas extensiones y aplicaciones ejecutadas en Mac con Apple silicon.

El problema arquitectónico es que la propuesta no define claramente qué decisión debe tomar el servicio cuando no recibe una aserción de App Attest.

No se debe interpretar automáticamente la condición de dispositivo no soportado como una validación satisfactoria (ALLOW). La información de capacidad reportada por la aplicación proviene del cliente y, por tanto, no constituye una evidencia confiable de integridad.

Si el servidor acepta como válida cualquier solicitud que declare App Attest como no soportado, una aplicación modificada podría utilizar ese comportamiento para evitar el mecanismo de protección.

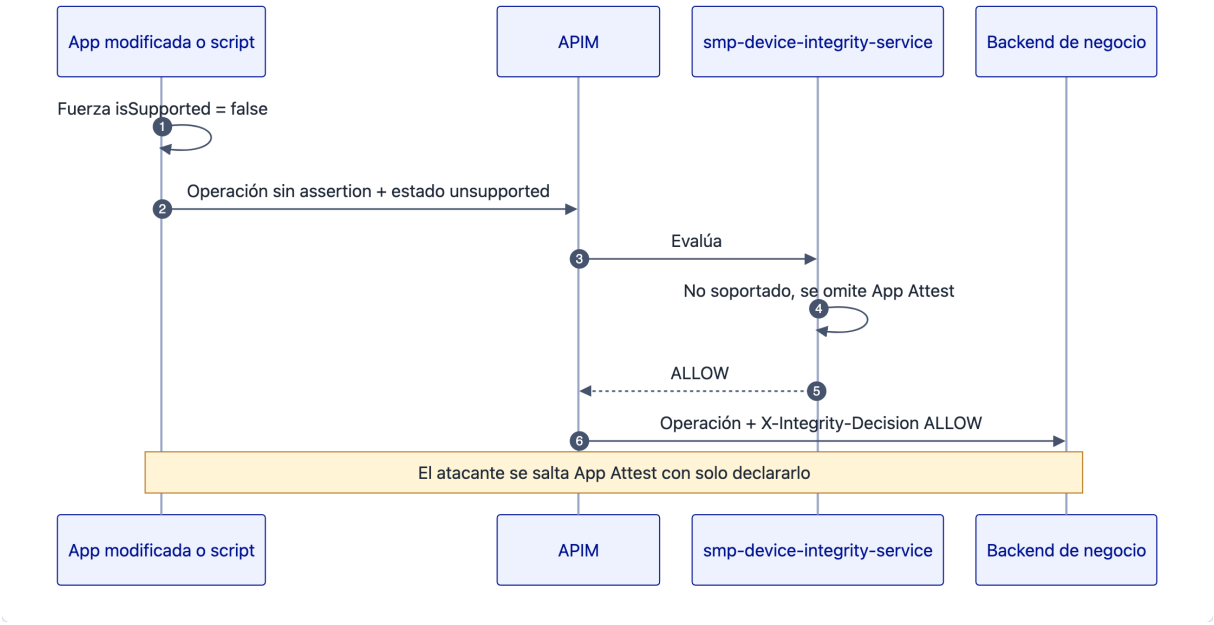


Figura 4.9 — Riesgo de aceptar una solicitud cuando el cliente declara que App Attest no está soportado.

Recomendación arquitectónica

El servicio debe distinguir entre los dispositivos que nunca pudieron utilizar App Attest y aquellos que ya fueron enrolados y posteriormente dejan de presentar una aserción.

Se recomienda establecer las siguientes reglas:

- **Nunca utilizar ALLOW cuando no exista evidencia verificable.** El resultado debe ser UNVERIFIED.
- **Dispositivo previamente enrolado que deja de presentar aserciones:** debe rechazarse con DENY y motivo ATTEST_DOWNGRADE, además de generar una alerta. Un dispositivo que ya completó correctamente el enrolamiento no debería pasar silenciosamente a un estado sin atestación.
- **Clave inválida (invalidKey) o keyId desconocido:** debe iniciarse un proceso de re-enrolamiento. No debe utilizarse esta condición como justificación para degradar los controles de integridad.
- **Eliminar escenarios evitables:** evitar publicar la aplicación para ejecución en Mac con Apple silicon si ese entorno no forma parte de los dispositivos soportados por el negocio, y evitar operaciones protegidas desde extensiones que no puedan proporcionar evidencia de App Attest.

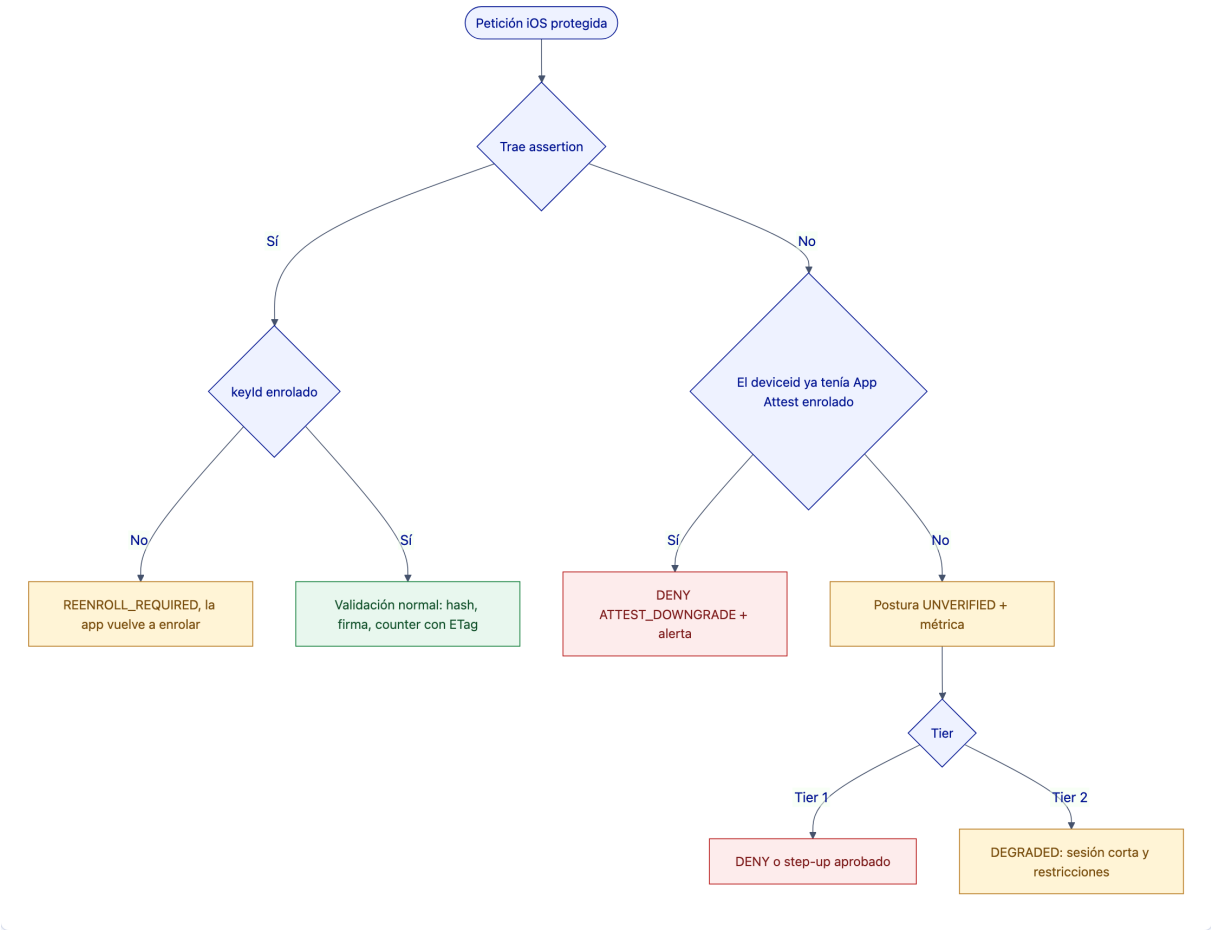


Figura 4.10 — Decisión de integridad iOS según la disponibilidad de una aserción verificable.

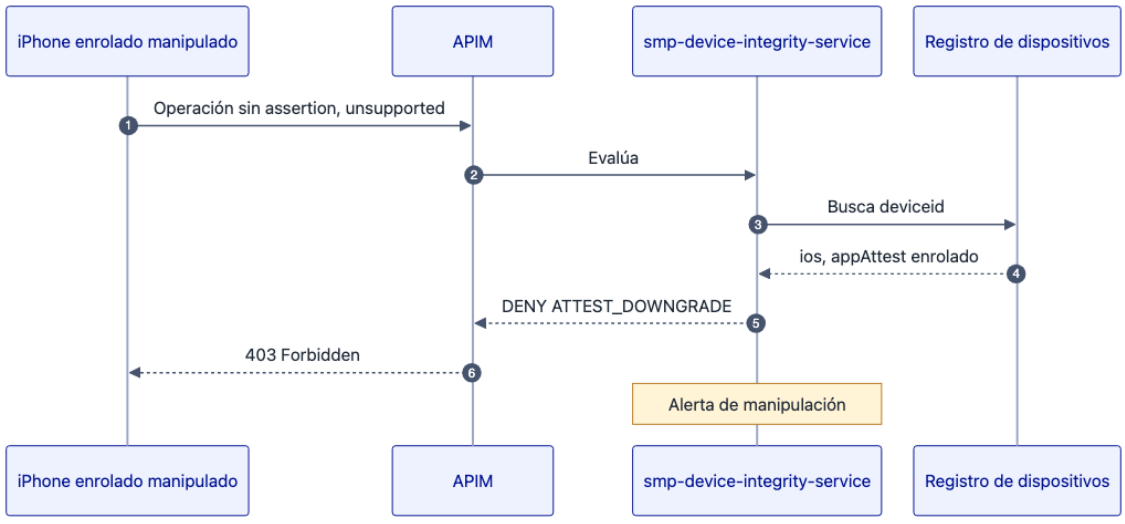


Figura 4.11 — Dispositivo previamente enrolado que declara no soportar App Attest: rechazo y generación de alerta.

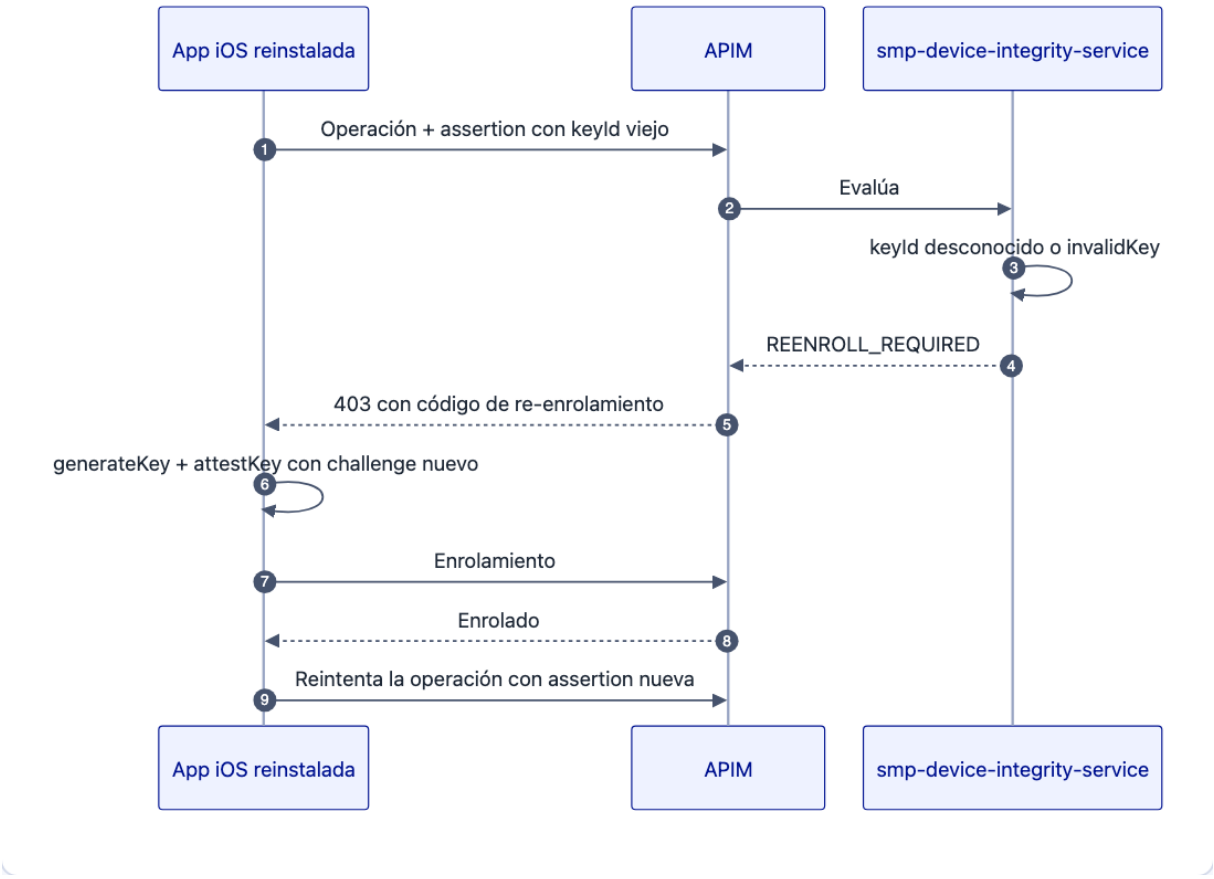


Figura 4.12 — Pérdida o invalidez de la clave: proceso de re-enrolamiento

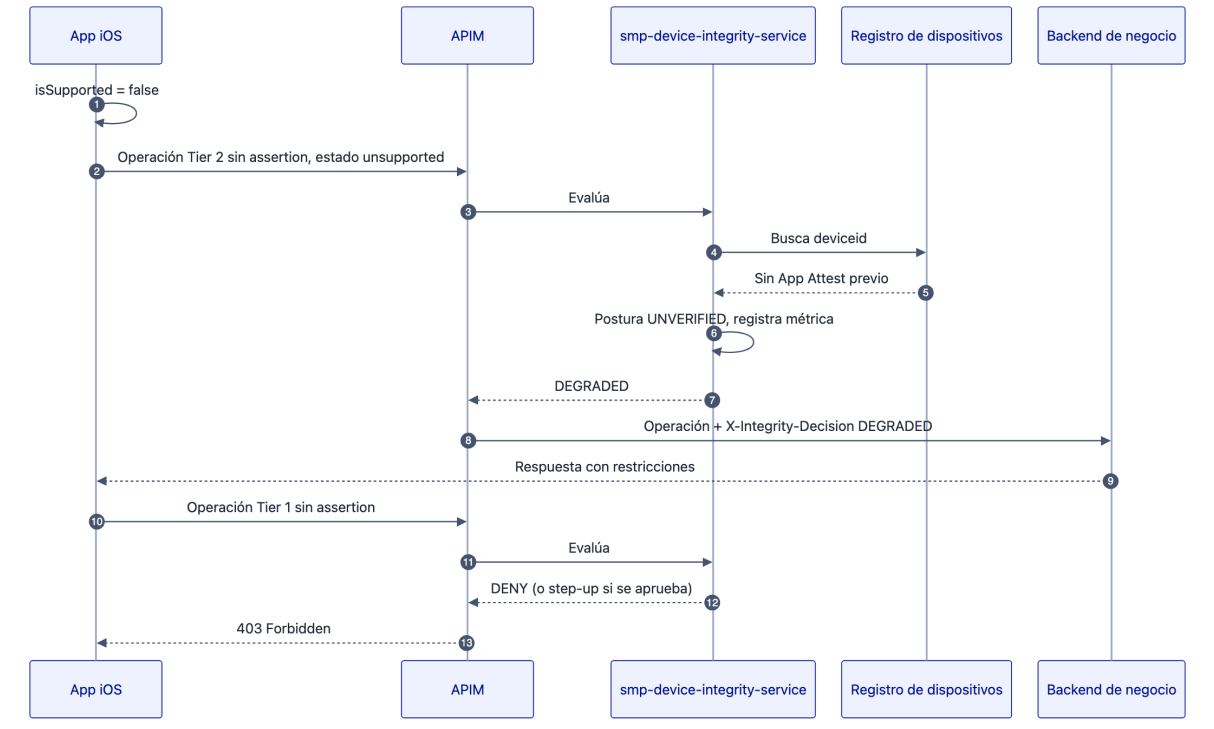


Figura 4.13 — Política de decisión para dispositivos iOS según su estado de enrolamiento y nivel de proteción.

Estado de Integridad	Tier 1	Tier 2

VERIFIED (aserción válida)	ALLOW	ALLOW
UNVERIFIED (Primer uso sin soporte o indisponibilidad de Apple)	DENY o step-up	DEGRADED
ATTEST_DOWNGRADE — Dispositivo previamente enrolado que deja de presentar aserción	DENY + alerta	DENY + alerta
REENROLL_REQUIRED — Clave inválida o desconocida	Re-enrolamiento obligatorio	Re-enrolamiento obligatorio

Consideración final

La ausencia de soporte para App Attest debe tratarse como una condición explícita de la política de seguridad, no como una excepción que permita continuar automáticamente.

La decisión debe considerar el nivel de protección de la operación, el estado de enrolamiento del dispositivo y la posibilidad de obtener evidencia verificable. En particular, un dispositivo que ya fue enrolado no debe poder degradar unilateralmente su nivel de protección declarando que App Attest dejó de estar disponible.

4.4 O4 — Versiones antiguas de la app

Prioridad: alta.

Las versiones actuales de la aplicación no envían las cabeceras necesarias para realizar la validación de integridad. Por tanto, cuando APIM recibe una solicitud sin esta información, no puede distinguir si proviene de una versión antigua legítima o de un cliente que intenta evadir los controles de seguridad.

Esto genera un problema de transición:

- Si APIM permite continuar las solicitudes sin evidencia de integridad, se mantiene una vía de acceso que puede ser aprovechada por aplicaciones modificadas o automatizadas.
- Si APIM bloquea inmediatamente estas solicitudes, los usuarios que todavía utilizan versiones antiguas podrían recibir errores HTTP 403 sin que la aplicación tenga capacidad para interpretar correctamente el motivo o solicitar una actualización.

En consecuencia, mantener la compatibilidad indefinidamente debilita el control de seguridad, mientras que bloquear las versiones antiguas sin una estrategia de actualización puede afectar la continuidad del servicio.

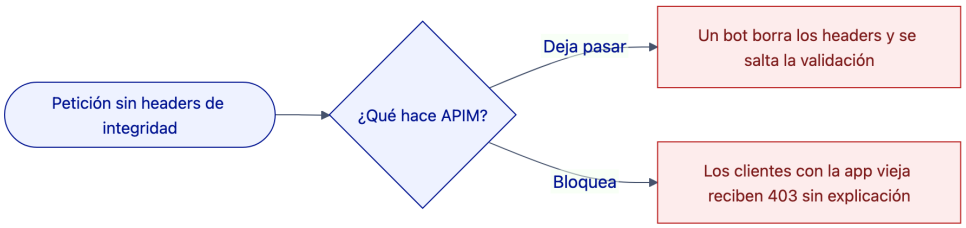


Figura 4.14 — Riesgo de permitir o bloquear versiones antiguas sin una estrategia de transición.

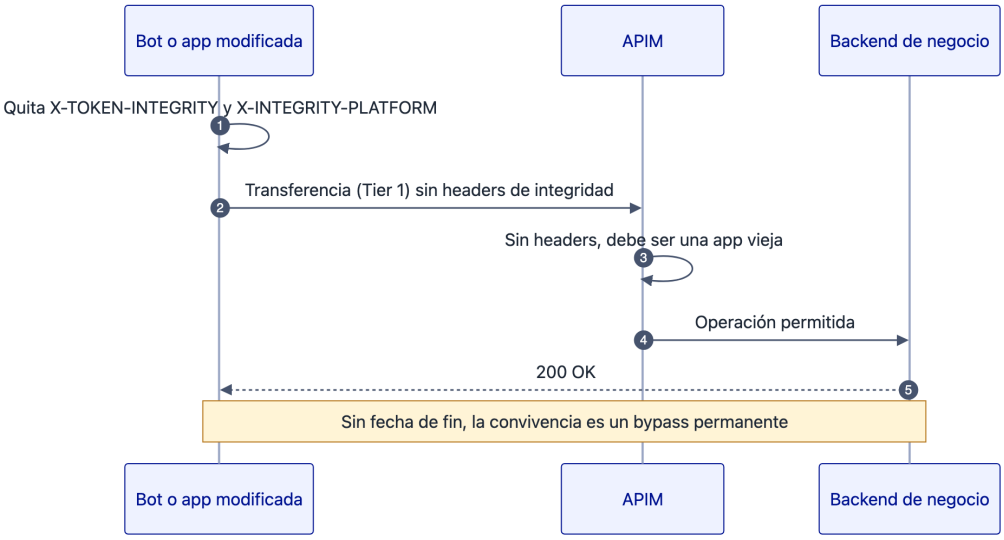


Figura 4.15 — Un cliente manipulado puede intentar identificarse como una versión antigua para evitar la validación de integridad.

Recomendación arquitectónica

Se recomienda implementar una transición controlada por fases, coordinada entre los equipos de arquitectura, APIM y desarrollo móvil.

Fase 0 — Publicación de la nueva versión

Publicar la versión N de la aplicación incorporando los mecanismos de generación y envío de las cabeceras de integridad correspondientes a cada plataforma.

Fase 1 — Convivencia en MONITOR

Permitir temporalmente la convivencia entre versiones antiguas y nuevas. Las solicitudes sin cabeceras de integridad deben identificarse como SKIPPED_LEGACY_CLIENT y registrarse para medir cuántos usuarios continúan utilizando versiones anteriores.

Esta etapa debe tener una fecha de finalización definida. No debe convertirse en una excepción permanente ni interpretarse como una validación satisfactoria.

Fase 2 — Actualización obligatoria

Establecer una versión mínima soportada y habilitar el mecanismo de actualización forzada al iniciar la aplicación.

Las solicitudes provenientes de versiones inferiores deben recibir una respuesta explícita, por ejemplo, HTTP 426 UPDATE_REQUIRED, acompañada de información que permita a la aplicación informar al usuario y dirigirlo a la actualización.

Fase 3 — Habilitación de ENFORCE

Una vez que la actualización obligatoria esté operativa y se haya reducido la presencia de versiones antiguas, habilitar ENFORCE progresivamente.

En esta etapa, las solicitudes protegidas que no presenten las cabeceras de integridad requeridas deben rechazarse con DENY_INTEGRITY_HEADERS_MISSING.

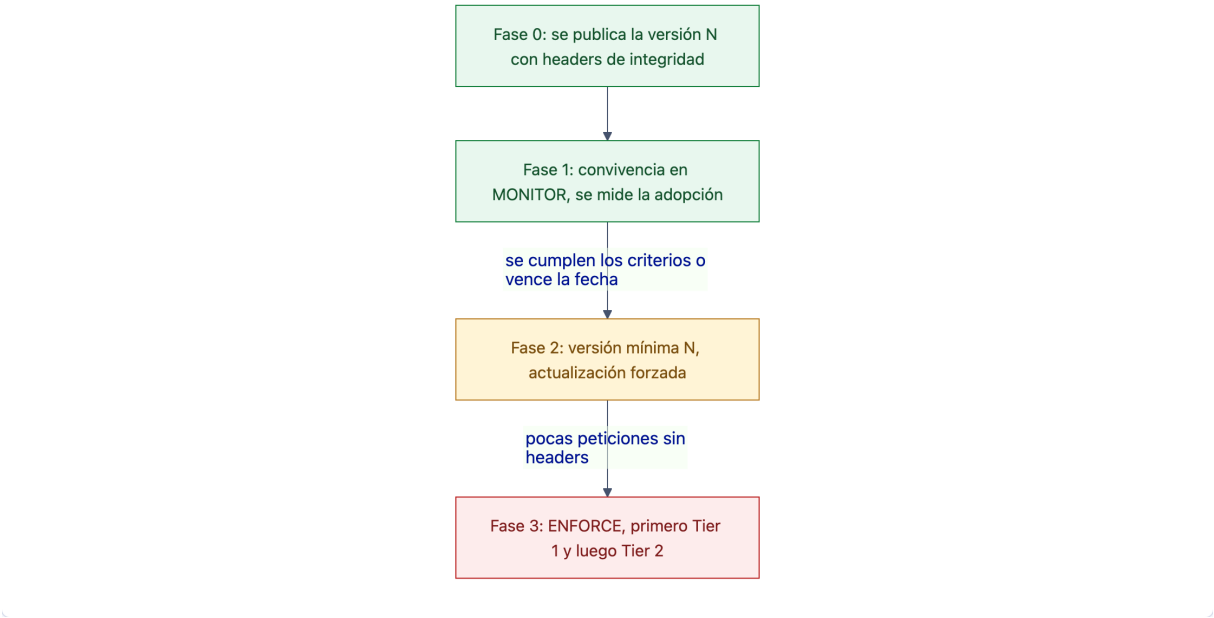


Figura 4.16 — Transición progresiva desde la publicación de la versión N hasta la habilitación de ENFORCE.

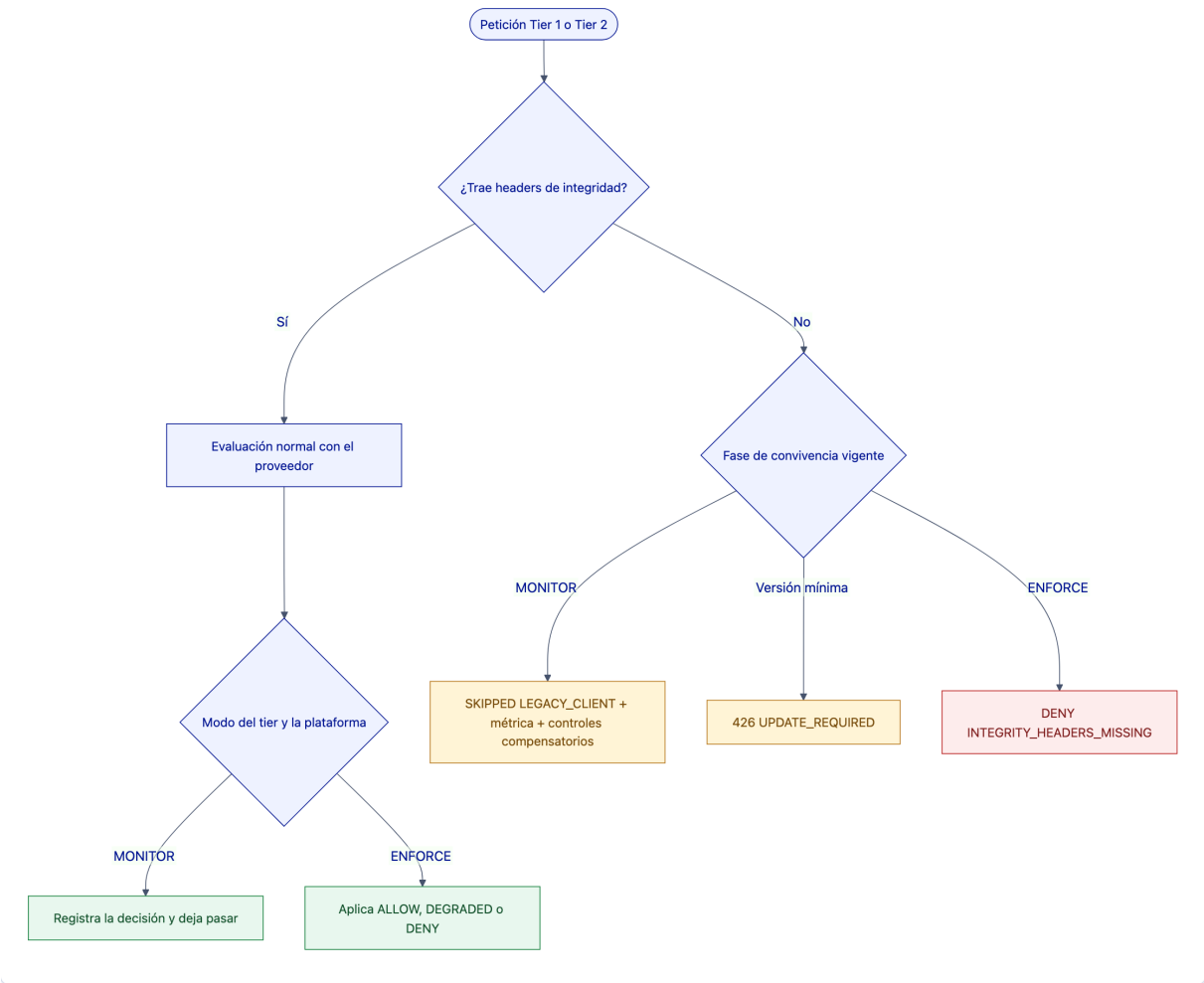


Figura 4.17 — Decisión de APIM según la fase de despliegue y la presencia de evidencia de integridad.

Consideraciones de seguridad

La cabecera X-APP-VERSION puede utilizarse para identificar versiones, medir su adopción y facilitar la transición. Sin embargo, es un dato proporcionado por el cliente y puede ser manipulado.

Por tanto, no debe utilizarse como evidencia de integridad ni como condición suficiente para conceder ALLOW. La decisión debe basarse en la presencia y validación de la evidencia de integridad, junto con la política de versiones mínimas definida por el banco.

Dependencia con el equipo móvil

Antes de habilitar ENFORCE, es necesario confirmar que la aplicación dispone de un mecanismo de actualización forzada al iniciar, que permita impedir el acceso a las operaciones protegidas cuando la versión instalada ya no sea compatible.

Sin este mecanismo, los usuarios de versiones antiguas podrían recibir errores que no pueden interpretar ni resolver desde la aplicación, incrementando las llamadas al centro de atención y afectando la experiencia del cliente.

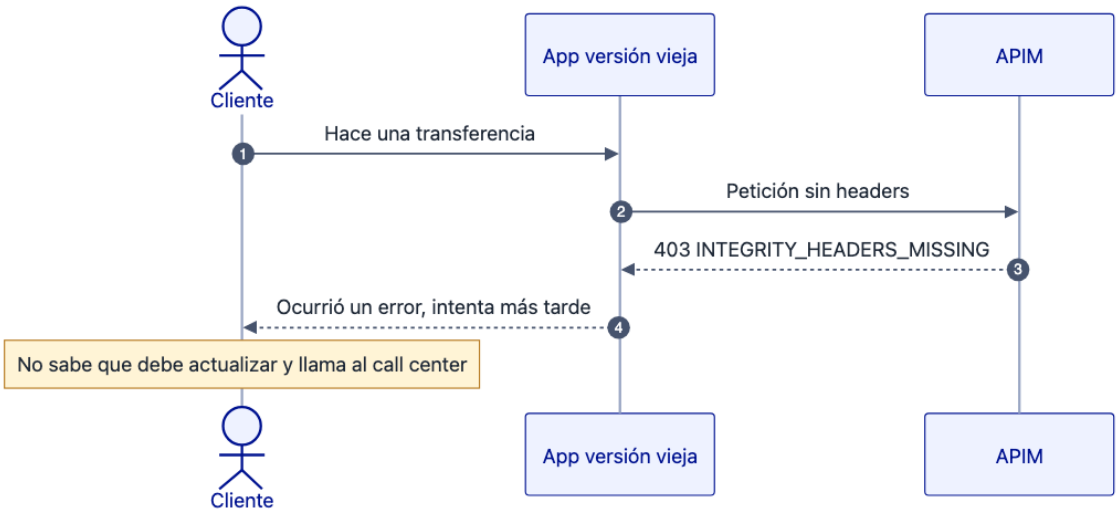


Figura 4.18 — Impacto de habilitar ENFORCE sin un mecanismo previo de actualización obligatoria

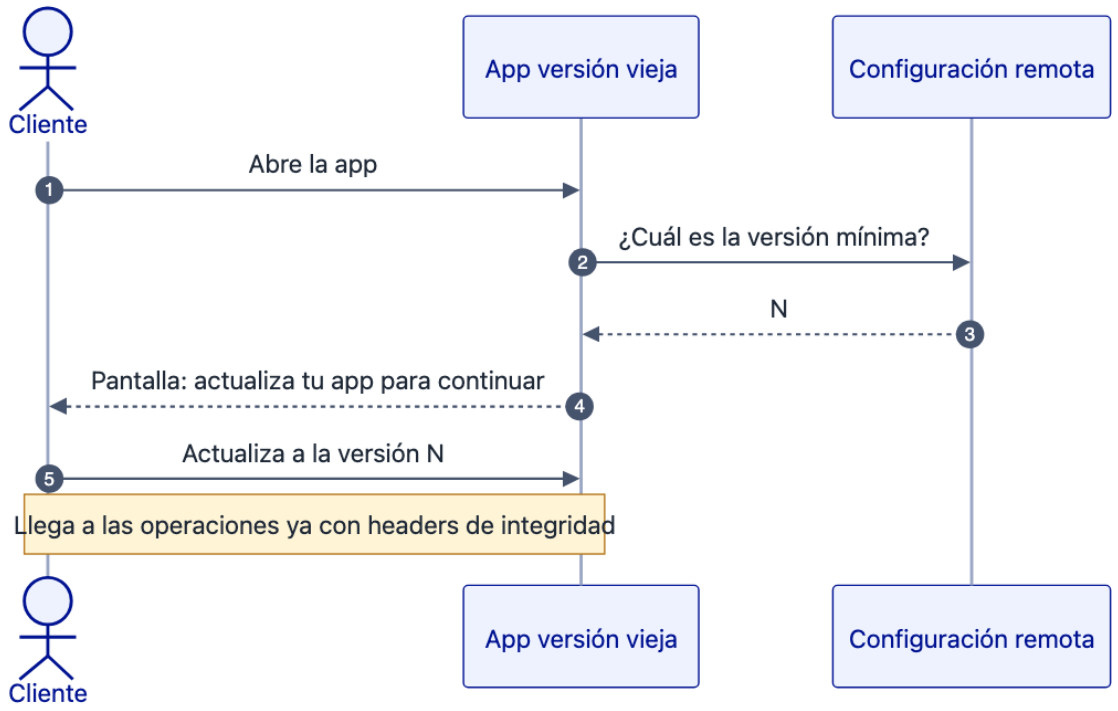


Figura 4.19 — Actualización forzada al iniciar la aplicación antes de habilitar ENFORCE.

Criterio de aceptación

La habilitación de ENFORCE debe depender de que la versión mínima soportada esté definida, la actualización forzada haya sido implementada y exista evidencia de adopción suficiente de la nueva

versión. La convivencia en MONITOR debe tener una fecha de término aprobada y no convertirse en una excepción permanente.

4.5 O5 — Rate limit de /deviceintegrity/initiate

Prioridad: alta.

El endpoint /deviceintegrity/initiate permite generar los challenges que se utilizan posteriormente para validar la integridad del dispositivo.

Cada solicitud genera una operación de escritura en Redis para almacenar el challenge y controlar su vigencia. Esto introduce un riesgo de consumo excesivo de memoria cuando el endpoint recibe un volumen elevado de solicitudes.

El problema aparece cuando cada solicitud genera una clave nueva, por ejemplo, utilizando un UUID. En ese escenario, Redis almacena una entrada por solicitud y su crecimiento depende de la cantidad de peticiones recibidas, no de la cantidad de sesiones activas.

Además, limitar las solicitudes únicamente por deviceId o sessionAppId no es suficiente si estos valores provienen de cabeceras controladas por el cliente. Un script automatizado puede generar identificadores diferentes en cada solicitud y evitar los límites establecidos.

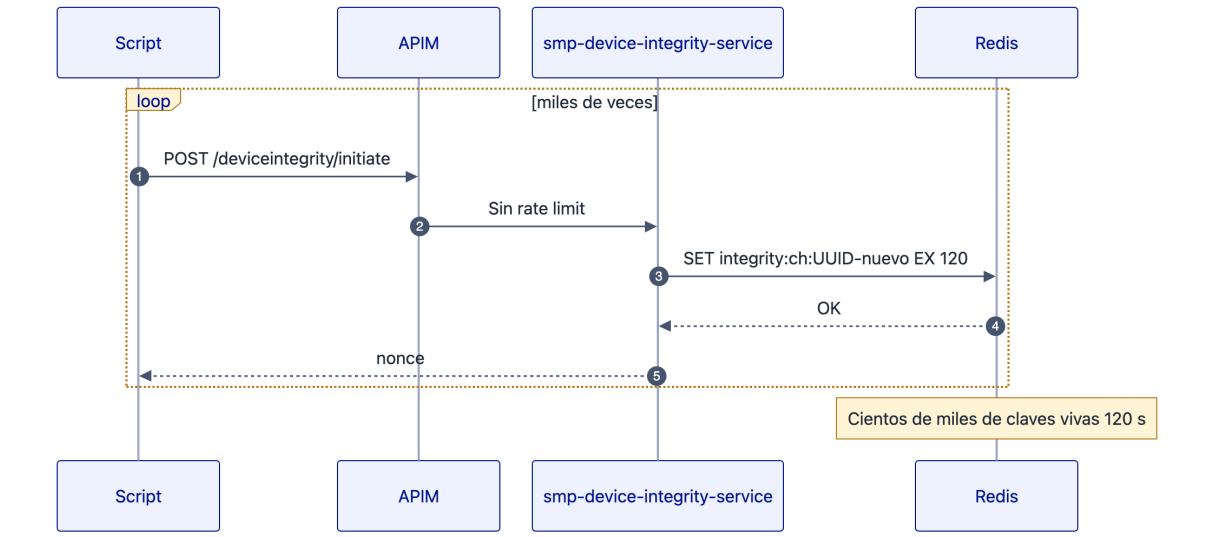


Figura 4.20 — Generación ilimitada de challenges y crecimiento descontrolado de Redis.

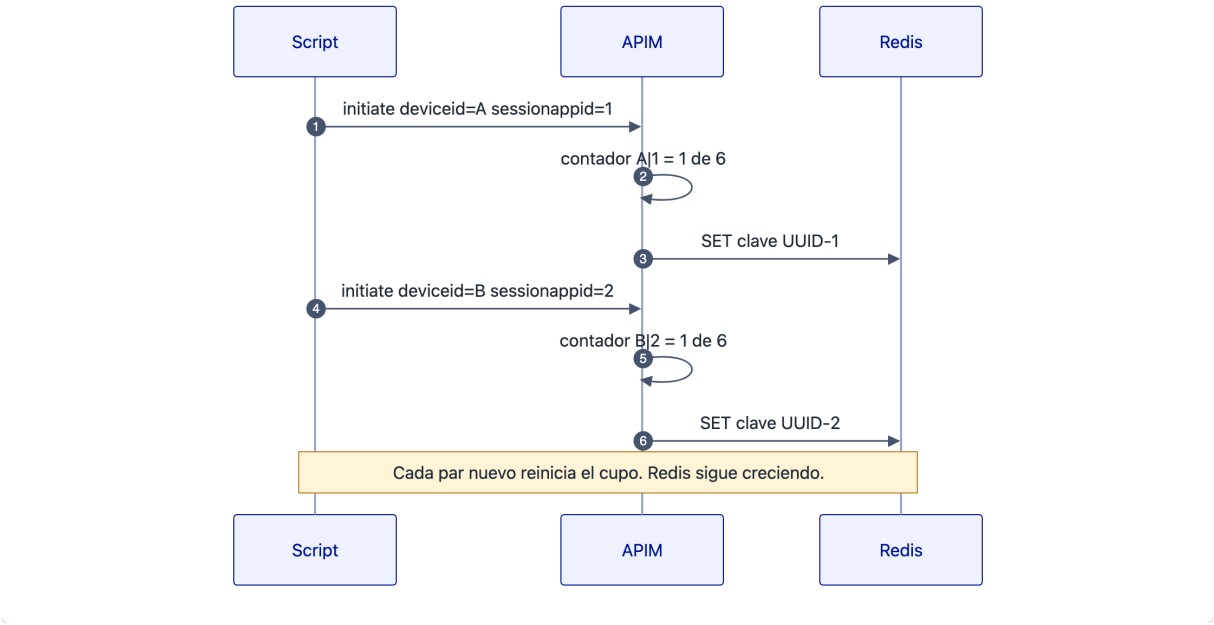


Figura 4.21 — Rotación de identificadores enviados por el cliente para evadir los límites de solicitudes.

Recomendación arquitectónica

El endpoint debe exigir autenticación mediante JWT y utilizar la identidad de sesión obtenida de un claim confiable del token, como sid, para controlar la generación de challenges.

Se recomienda implementar las siguientes medidas:

- **Identidad confiable:** exigir un JWT válido y obtener el identificador de sesión (sid) desde el token validado. No utilizar cabeceras proporcionadas por el cliente como identidad principal para aplicar límites.
- **Una clave por sesión:** almacenar el challenge utilizando una clave determinística, por ejemplo, integrity:ch:{sid}, en lugar de generar una nueva clave para cada solicitud.
- **Sobrescritura controlada:** utilizar una operación de escritura que reemplace el challenge anterior de la misma sesión y establezca un TTL de 120 segundos. De esta manera, las nuevas solicitudes no generan claves adicionales para una misma sesión.
- **Límites de solicitudes:** implementar controles en APIM y replicarlos en el servicio para evitar que la protección dependa exclusivamente de la capa de entrada.

Como referencia inicial, se propone establecer los siguientes límites:

Nivel de control	Límite propuesto
Por sesión	6 solicitudes cada 60 segundos
Por dirección IP	30 solicitudes cada 60 segundos
Global	Límite máximo de solicitudes por segundo, dimensionado según capacidad

Los valores deben validarse mediante pruebas de carga y ajustarse al comportamiento real de la aplicación.

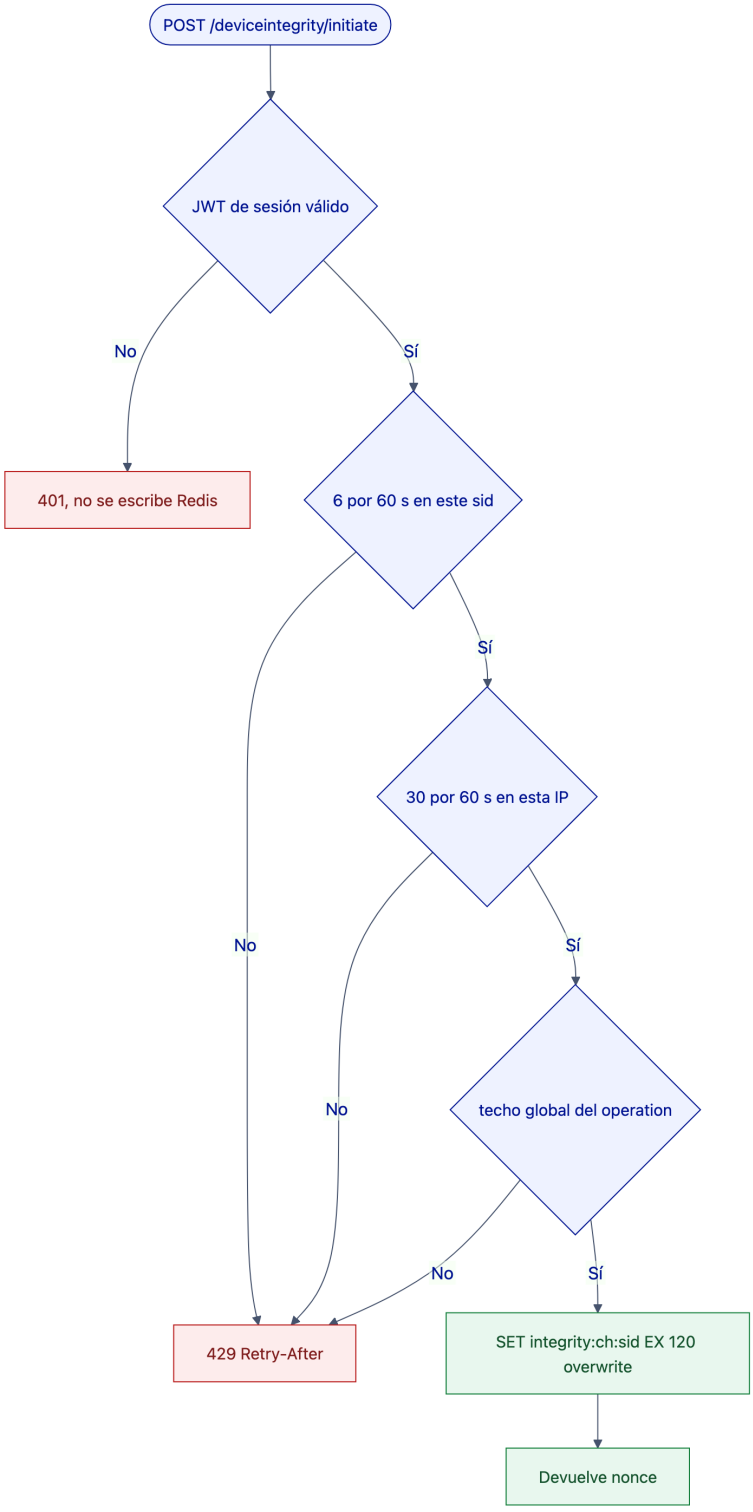


Figura 4.22 — Autenticación mediante JWT, límites por sesión, IP y capacidad global, y almacenamiento controlado del challenge.

Impacto en la capacidad de Redis

El almacenamiento debe diseñarse para que el consumo de memoria dependa principalmente de las sesiones activas y no del número acumulado de solicitudes.

Por ejemplo, con un volumen de 2 000 solicitudes por segundo y una vigencia de 120 segundos, un diseño que genere una clave por solicitud podría acumular aproximadamente 240 000 entradas simultáneas.

Considerando un tamaño estimado de 800 bytes por entrada, el consumo sería de aproximadamente 192 MB, sin incluir sobrecostos adicionales de Redis.

Con una clave determinística por sesión y sobrescritura controlada, el consumo se reduce a una entrada por sesión activa, evitando que las solicitudes repetidas incrementen indefinidamente la cantidad de claves.

Se recomienda, además, utilizar una instancia de Redis dedicada a los challenges, establecer límites de memoria mediante maxmemory y definir una política de expiración compatible con el TTL requerido. No se debe depender de una política de eviction que pueda eliminar challenges todavía vigentes y afectar la validación de integridad.

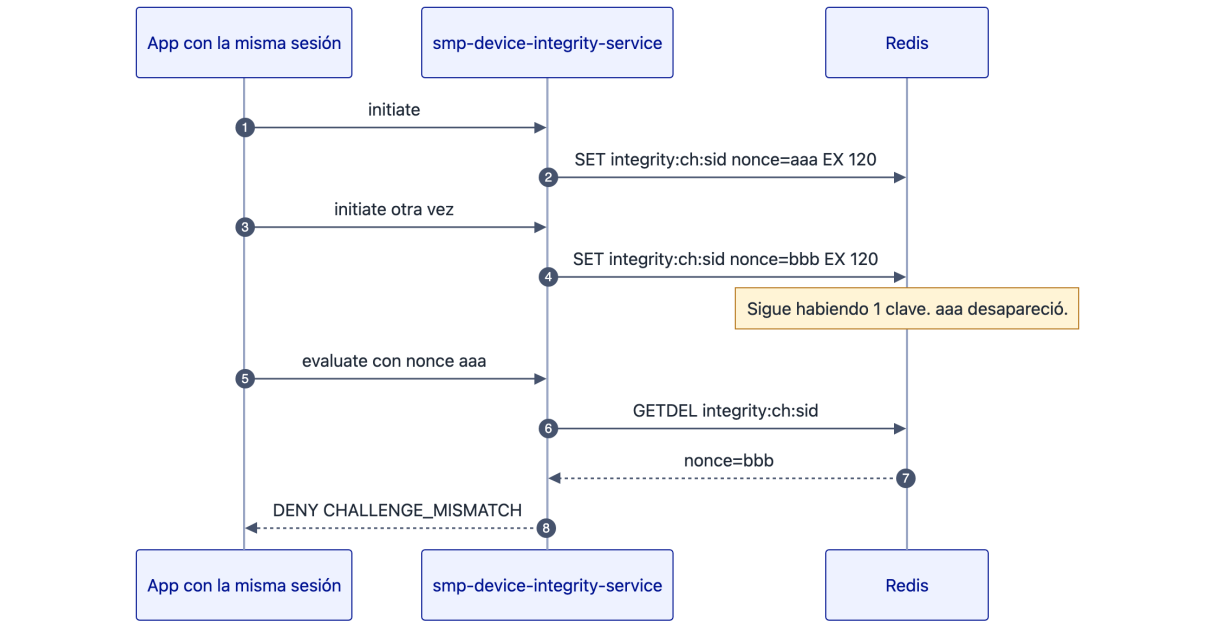


Figura 4.23 — Una nueva solicitud de la misma sesión reemplaza el challenge anterior, manteniendo una sola entrada activa.

Consideración de seguridad

La sobrescritura del challenge debe invalidar el valor anterior. De esta manera, si una sesión solicita un nuevo challenge, el anterior deja de ser válido y no puede utilizarse posteriormente.

También debe garantizarse que la generación y el consumo del challenge sean operaciones atómicas, para evitar condiciones de carrera cuando llegan solicitudes concurrentes desde una misma sesión.

Criterio de aceptación

El endpoint /deviceintegrity/initiate debe contar con autenticación obligatoria, límites de solicitudes por sesión, IP y capacidad global, y un mecanismo de almacenamiento que evite la generación ilimitada de claves en Redis. El consumo de memoria debe estar asociado a las sesiones activas y a la vigencia de sus challenges.

4.6 O6 — Huawei: validación local del JWS, no una API de descifrado

Prioridad: alta.

La tabla comparativa no deja claro dónde se realiza la validación de la evidencia de Huawei. Esto puede llevar a interpretar que smp-device-integrity-service debe invocar una API de Huawei para descifrar o validar el token.

A diferencia de Google Play Integrity, Huawei SysIntegrity no dispone de una API equivalente a decodeIntegrityToken. La aplicación obtiene el JWS mediante HMS Core y lo envía al banco. El servicio del banco debe verificar localmente su firma y contenido, utilizando la cadena de certificados de confianza (CBG) correspondiente.

Recomendación arquitectónica

- La aplicación obtiene el JWS desde HMS Core y lo envía al backend.
- smp-device-integrity-service verifica localmente la cadena de certificados, la firma del JWS y los campos requeridos: nonce, paquete, digest y basicIntegrity.
- La validación debe realizarse con certificados y reglas de confianza administrados por el banco, sin depender de una llamada a Huawei en cada operación.
- APIM mantiene la misma responsabilidad de orquestación y aplicación de la decisión de integridad que para las demás plataformas.

Riesgo aceptado

La señal basicIntegrity = true ofrece un nivel de garantía inferior a MEETS_DEVICE_INTEGRITY de Google. Además, Huawei no documenta un campo evaluationType equivalente en el payload. Estas diferencias deben quedar registradas como limitaciones de la señal de integridad de Huawei y aceptarse formalmente por Seguridad y el negocio.

El método checkSysIntegrityOnLocal, cuando no proporciona un JWS verificable, no debe utilizarse para autorizar operaciones en modo ENFORCE.

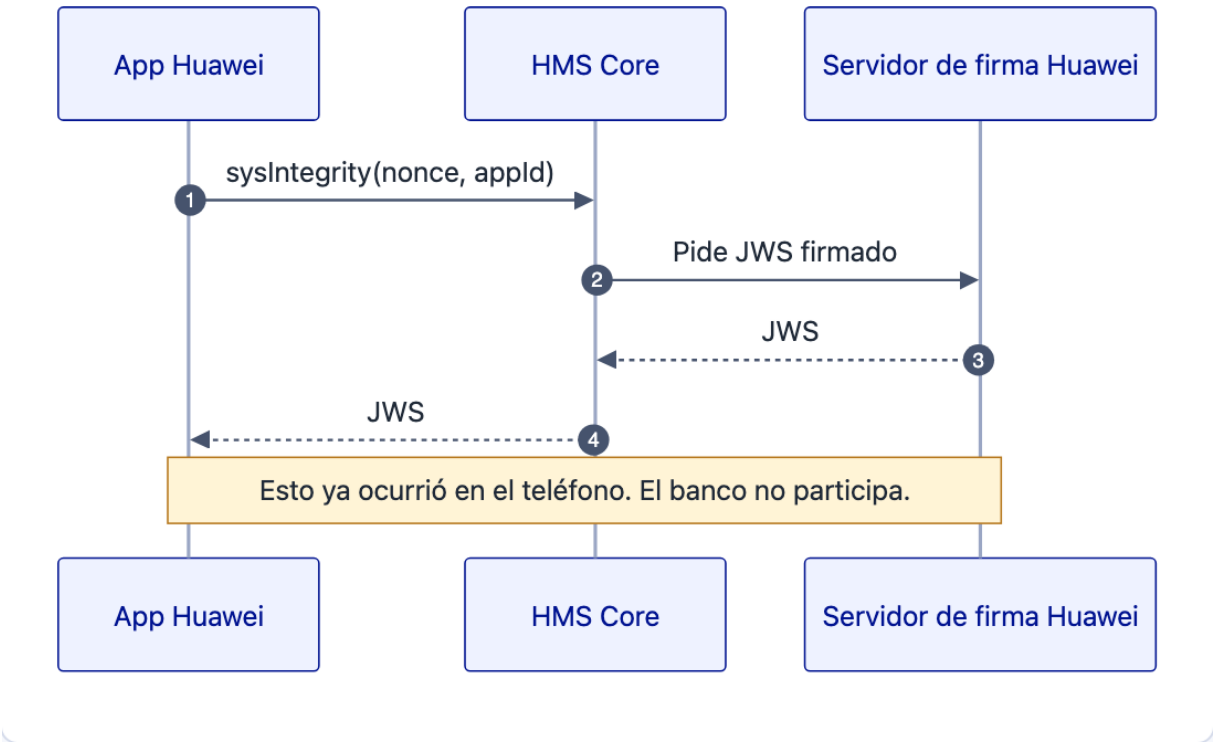


Figura 4.24 — La aplicación se comunica con Huawei para obtener el JWS.

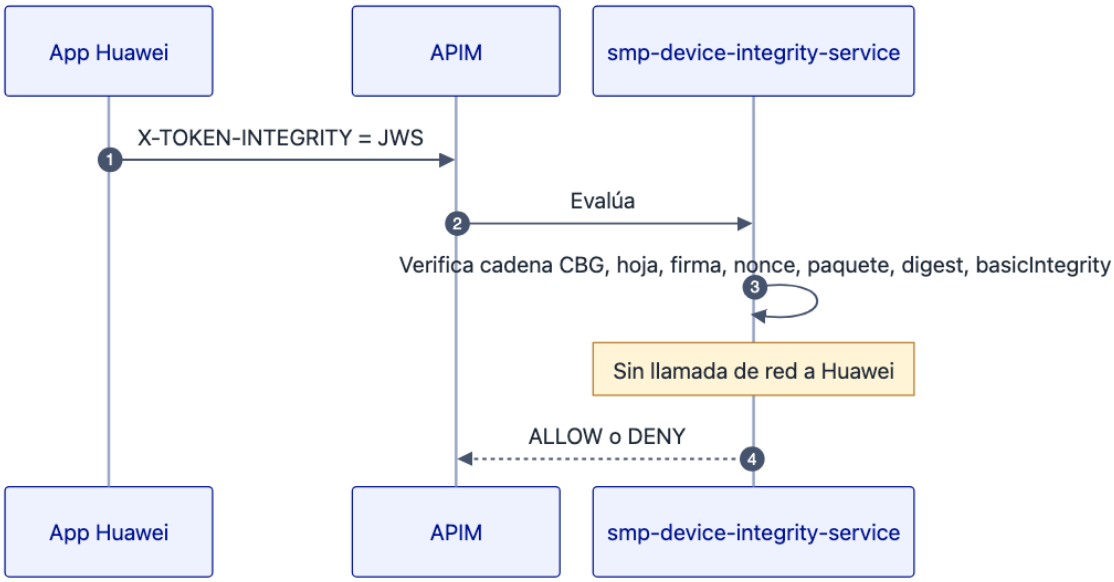


Figura 4.25 — El banco verifica localmente el JWS, sin llamada de red a Huawei.

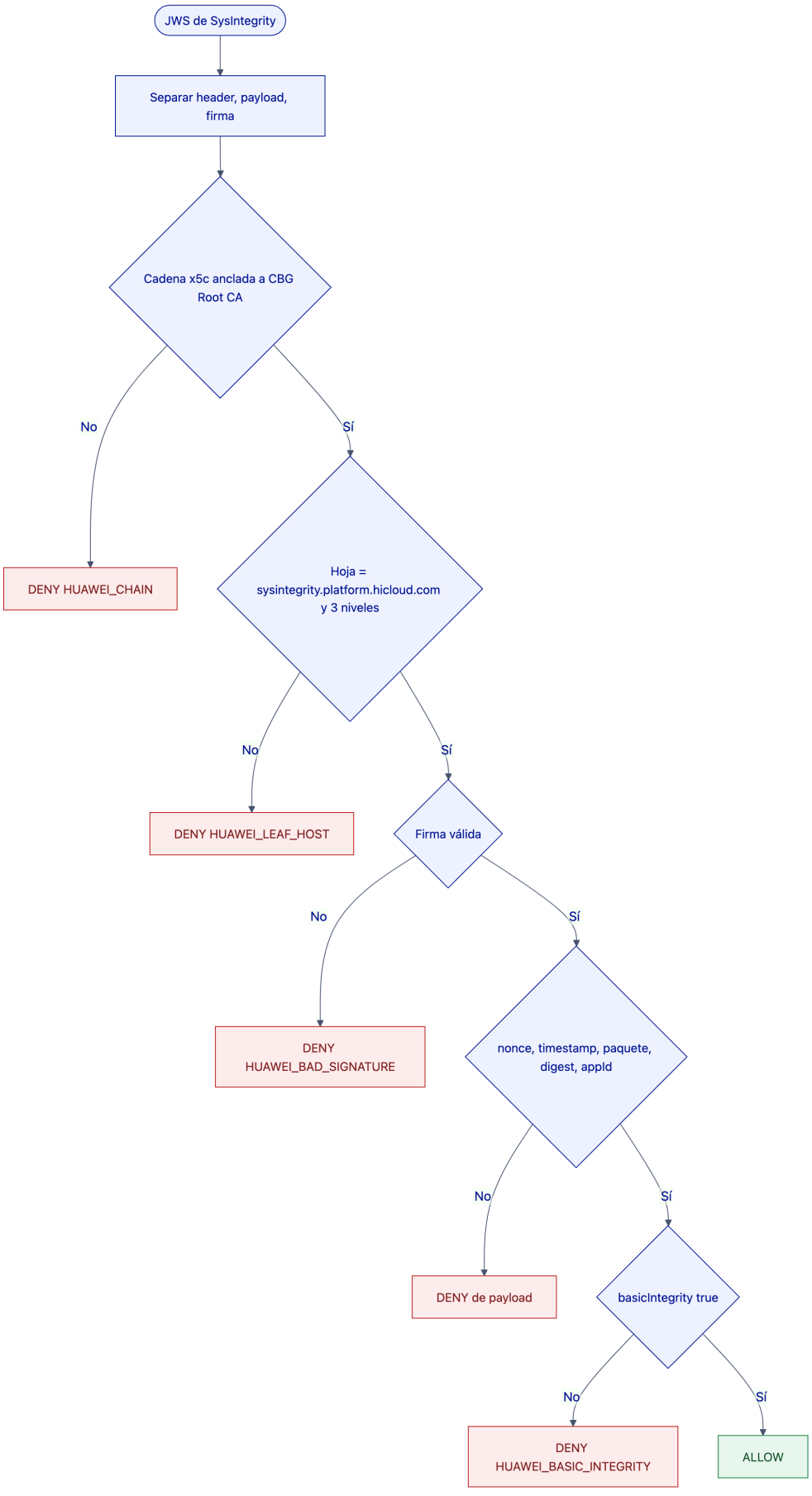


Figura 4.26 — Elementos de validación: cadena CBG, certificado hoja sysintegrity.platform.hicloud.com, firma, nonce, paquete, digest y basicIntegrity.



Figura 4.27 — Diferencia entre plataformas: Google descifra el token mediante su servicio; Huawei se verifica localmente en el banco.



Figura 4.28 — Eliminar la conexión de red entre smp-device-integrity-service y Huawei.

Criterio de aceptación

El flujo de despliegue y la documentación deben mostrar que la validación Huawei es local, que no existe una dependencia de red con Huawei durante cada validación y que las limitaciones de basicIntegrity han sido aceptadas formalmente.

5. Contradicciones internas del documento

Se identificaron inconsistencias entre las secciones, tablas y diagramas que deben corregirse antes de implementar la solución. Aunque algunas parecen diferencias de redacción, otras pueden provocar fallos de integración, especialmente en la identificación de la plataforma y en la generación y validación de los hashes.

La siguiente tabla establece el criterio único que debe mantenerse en todo el documento.

Dónde	Qué dice hoy	Qué debería decir
Títulos Tier 1 Android e iOS	«validación en el BFF»	APIM orquesta el flujo, smp-device-integrity-service valida la evidencia y el backend recibe únicamente la decisión de integridad.
Nota Huawei Tier 1	«el cliente y el BFF calculan nonce»	El nonce lo emite smp-device-integrity-service y lo persiste en Redis. La app lo recibe y lo usa al pedir el JWS. El servicio lo vuelve a leer de Redis y lo compara con el del JWS. El BFF no genera ni ve el nonce.
Despliegue	«solo lo invocan APIM y el BFF»	El único consumidor previsto es APIM, salvo que se apruebe expresamente otro llamante.
Headers	X-PLATFORM-ID en §1.3, X-INTEGRITY-PLATFORM en diagramas	Utilizar exclusivamente X-INTEGRITY-PLATFORM, con los valores android, huawei o ios.
Fórmula del hash	| en la parte 1, || en Huawei e iOS	Utilizar el separador ASCII 0x7C, codificación UTF-8 y una única regla de canonicalización compartida.
Payload canónico	destination_account | amount | currency	Definir un payload específico por operación. No todas las operaciones tienen cuenta, monto o moneda.
Nombres	CosmoDB, challengue	Utilizar Cosmos DB y challenge.

5.1 Responsabilidades y flujo de validación

Los títulos, diagramas y descripciones de Tier 1 deben reflejar una única distribución de responsabilidades:

- **Aplicación móvil:** obtiene la evidencia de integridad y construye los datos requeridos para vincularla con la operación.
- **APIM:** recibe la solicitud, identifica la operación, orquesta la validación y aplica la decisión antes de permitir el acceso al backend.
- **smp-device-integrity-service:** valida la evidencia criptográfica y los datos asociados a la petición.
- **Backend de negocio:** recibe la decisión confiable emitida por APIM; no realiza una segunda validación del proveedor de integridad.

5.2 Convención única de headers y canonicalización

Se debe utilizar un único nombre de header para identificar la plataforma:

X-INTEGRITY-PLATFORM

Valores permitidos: android, huawei e ios.

Este header no debe considerarse una fuente confiable por sí sola: la plataforma debe contrastarse con la identidad del dispositivo y los datos de enrolamiento disponibles en el servidor.

Para la construcción de hashes se debe establecer una regla común:

- Codificación UTF-8.
- Separador ASCII 0x7C (|).
- Mismo orden de campos en la aplicación y en el servicio.
- Mismas reglas de normalización para cada campo.
- El monto se representa con punto decimal y dos posiciones, sin separadores de miles. Ejemplo: 100.00.
- La moneda utiliza el código establecido para la operación, por ejemplo, PEN.

No se deben mezclar las representaciones | y ||, porque producen hashes diferentes y ocasionan que una evidencia legítima no pueda validarse.

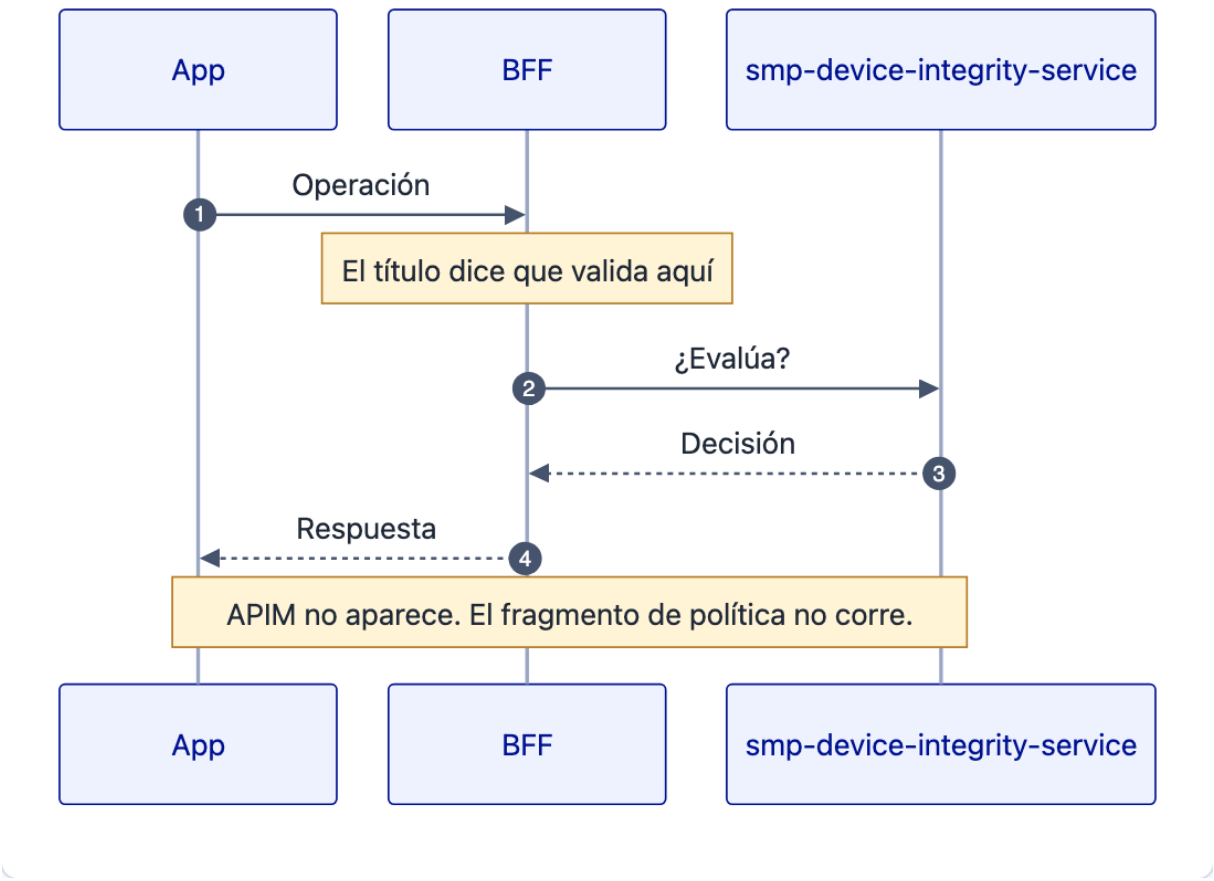


Figura 5.1 — Los titulos actuales sugieren que el BFF realiza la validación.

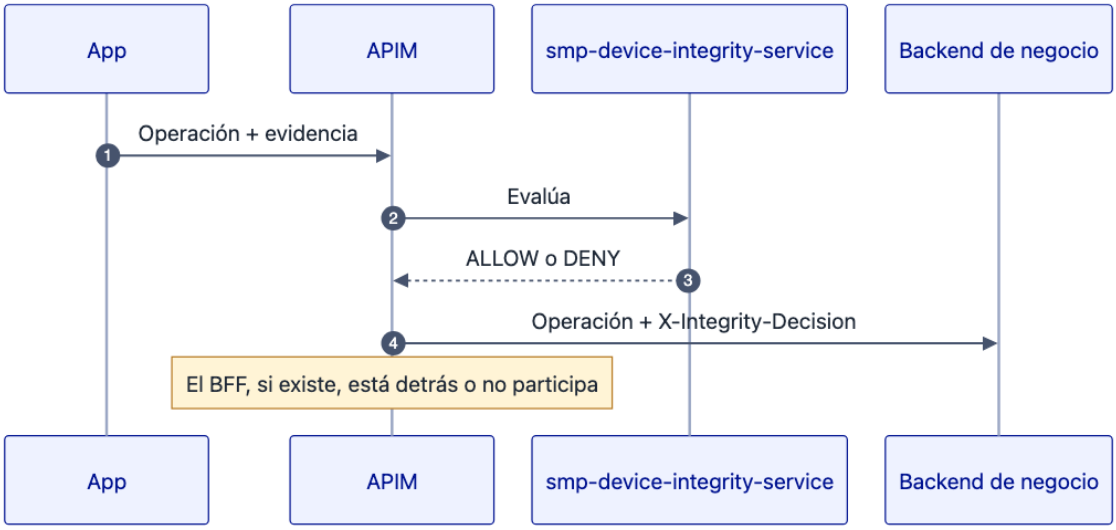


Figura 5.2 — Flujo corregido: APIM orquesta, el servicio valida y el backend recibe la decisión.

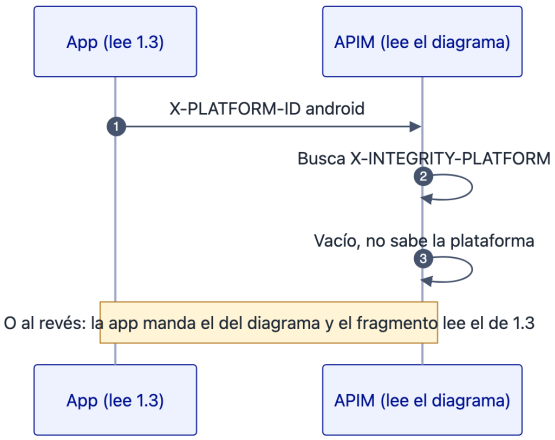


Figura 5.3 — Inconsistencia entre los nombres de headers utilizados para identificar la plataforma.

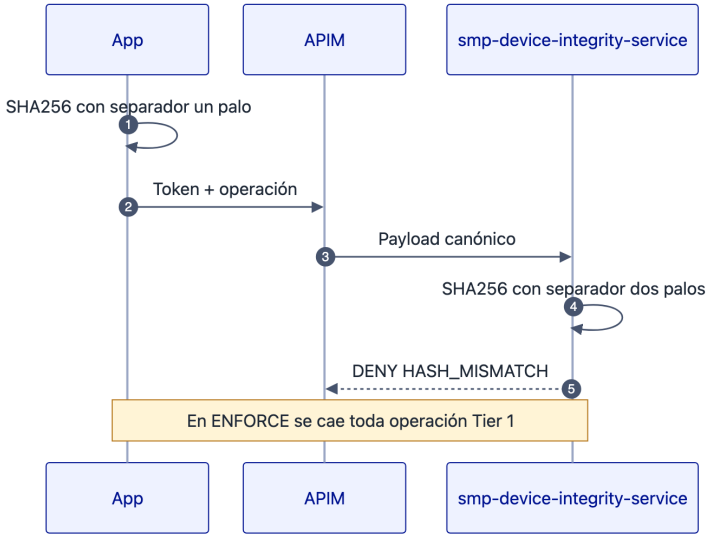


Figura 5.4 Diferencia en el separador del hash que provoca resultados distintos entre cliente y servidor.

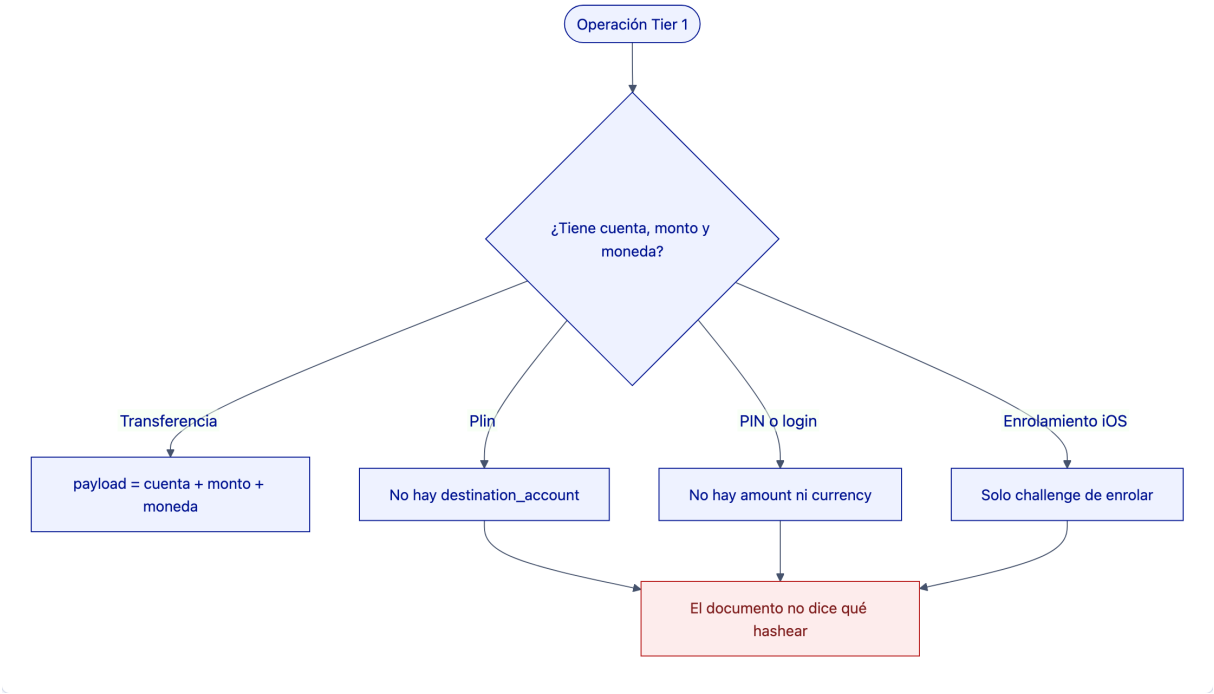


Figura 5.5 La estructura cuenta-monto-moneda no representa operaciones como PIN, login o Plin.

5.3 Catálogo de payloads canónicos por operación

El payload no puede asumir que todas las operaciones tienen una cuenta de destino, un monto y una moneda. Por ello, se propone un catálogo de estructuras canónicas específicas por operación.

Operación	payload_canonico (propuesta)
Transferencia a cuenta	xfer_account|{destination_account}|{amount}|{currency}
Plin	plin|{alias_or_phone}|{amount}|{currency}
Pago de servicios	billpay|{service_code}|{amount}|{currency}
Cambio de PIN	pin_change|{deviceid} (el PIN no entra)
Login	login|{deviceid}|{channel}
Enrolamiento iOS	ios_enroll|{deviceid}

El PIN nunca debe incluirse en el payload canónico. Para cada operación se deben definir expresamente los campos, su orden, formato y reglas de normalización.

Catálogo cerrado

Producto y AppSec deben aprobar y mantener una fila por cada operación Tier 1 antes de habilitar ENFORCE. Si APIM recibe una operación protegida que no está definida en el catálogo, debe responder HTTP 400 y no intentar construir un payload alternativo.

Criterio de aceptación: todas las secciones, tablas y diagramas deben utilizar las mismas responsabilidades, nombres de headers, valores de plataforma y reglas de canonicalización. El catálogo de payloads debe estar aprobado por Producto y AppSec, y contar con pruebas que demuestren que la aplicación y el servicio generan exactamente el mismo hash para cada operación Tier 1.

6. Pendientes de la sección 4.4

La sección 4.4 identifica cuatro políticas operativas que deben definirse antes de habilitar el modo ENFORCE. Sin estas decisiones, la solución no tiene un comportamiento controlado ante la indisponibilidad de los proveedores, errores de configuración o desactivaciones temporales de la validación.

Las políticas siguientes establecen cómo debe responder la solución ante estos escenarios, qué controles debe mantener el backend y quién está autorizado para modificar la configuración.

6.1 Caída del servicio o del proveedor

La indisponibilidad del servicio de integridad o de un proveedor externo no debe convertirse en un mecanismo para omitir la validación. Una política fail-open permitiría que una saturación o caída del servicio se transforme en un bypass de seguridad. Por otro lado, aplicar fail-closed en ambos tiers podría provocar una interrupción generalizada de las operaciones.

Recomendación arquitectónica

Aplicar una política diferenciada por nivel de riesgo:

Escenario de fallo	Tier 1	Tier 2	Código de motivo
Timeout o HTTP 5xx del servicio	DENY	DEGRADED	INTEGRITY_SERVICE_UNAVAILABLE
Timeout o HTTP 5xx de Google o Apple	DENY	DEGRADED	PROVIDER_UNAVAILABLE
Cuota de Play Integrity agotada	DENY	DEGRADED	PROVIDER_QUOTA
Redis o Cosmos DB no disponibles	DENY	DEGRADED	INTEGRITY_STORE_UNAVAILABLE

En Tier 1, la operación se rechaza porque no es posible demostrar la integridad requerida. En Tier 2, se permite continuar únicamente bajo una política DEGRADED, manteniendo controles adicionales del backend, como límites de monto, restricciones de sesión y mayor exigencia del motor antifraude.

DEGRADED no equivale a ALLOW ni elimina los controles de autenticación, autorización y prevención de fraude.

Timeouts iniciales para medir durante MONITOR

- APIM → smp-device-integrity-service: 1 segundo, con ignore-error habilitado para que APIM pueda aplicar la política de contingencia.
- Servicio → Google: 1.500 ms, con un único reintento controlado.
- Huawei: no requiere llamada de red al proveedor durante la validación si el JWS se verifica localmente.

- Apple: establecer el timeout según el mecanismo de validación implementado y medirlo durante MONITOR.

Los valores deben ajustarse con métricas reales de latencia, disponibilidad y volumen antes de habilitar ENFORCE.

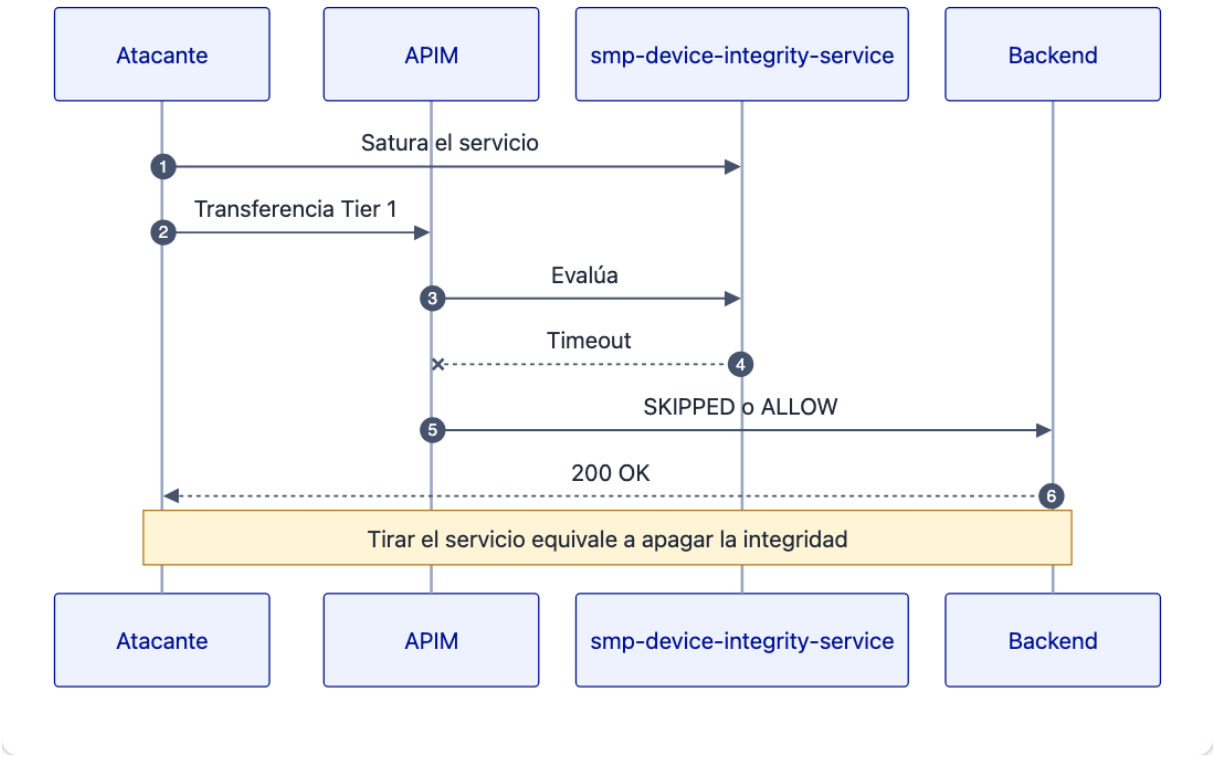


Figura 6.1 — Una politica fail-open convierte la indisponibilidad del servicio en un bypass de seguridad.

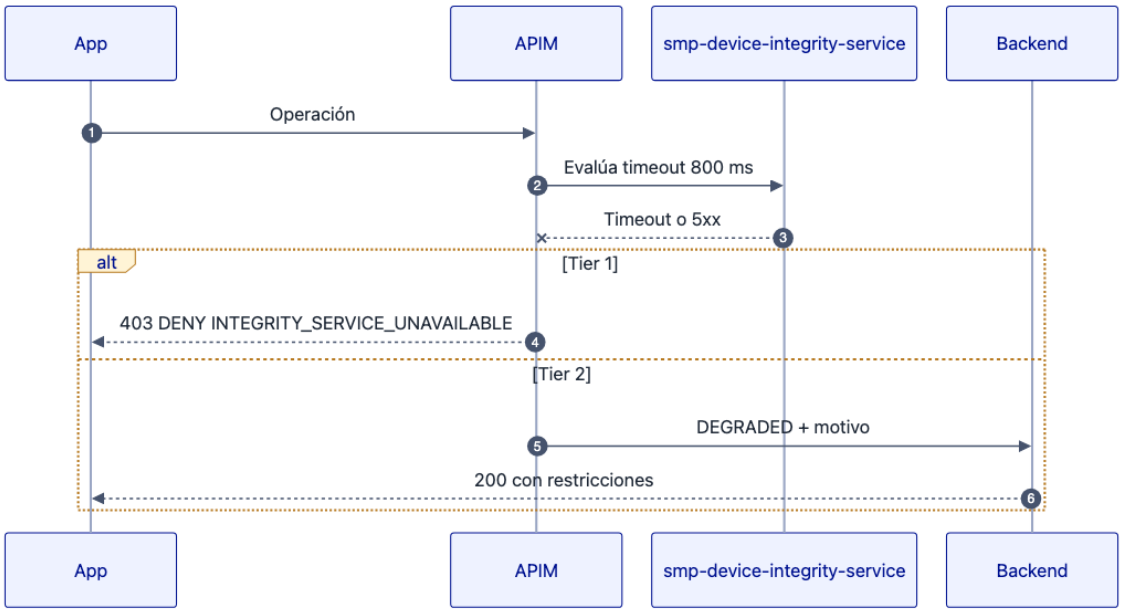


Figura 6.2 — Política recomendada: DENY en Tier 1, DEGRADED en Tier 2.

6.2 Named Value ausente o inválido

El mecanismo de desactivación temporal de la validación debe diferenciarse de la política de contingencia ante una caída técnica.

Un error de configuración no debe desactivar la seguridad. Tampoco debe provocar automáticamente el bloqueo de todo el canal. La solución debe continuar validando, salvo que exista una instrucción explícita y vigente para desactivar la evaluación.

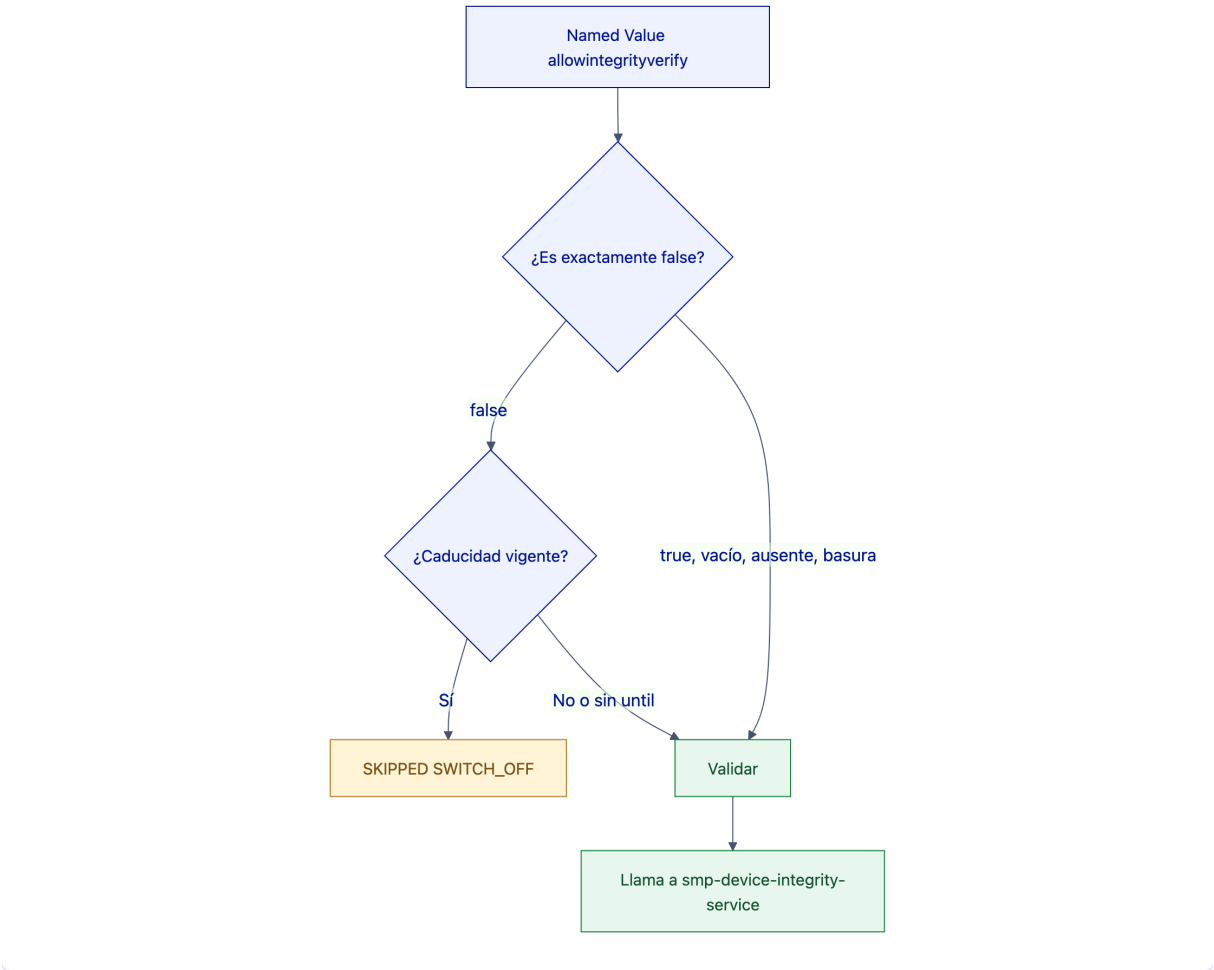


Figura 6.3 — Solo un false explícito y una fecha de caducidad vigente permiten desativar temporalmente la evaluación

Regla de una línea

El switch allowintegrityverify solo desactiva la evaluación cuando se cumplen simultáneamente estas condiciones:

- El valor, después de eliminar espacios y convertirlo a minúsculas, es exactamente false.
- Existe allowintegrityverify-until.
- La fecha y hora de caducidad son válidas y todavía están vigentes.

En ese único escenario, APIM devuelve SKIPPED con motivo SWITCH_OFF.

Cualquier otro valor (true, vacío, ausente, incorrecto o con fecha de caducidad inválida) mantiene activa la validación.

Importante: si el Named Value está ausente o tiene un formato inválido, se debe generar una alerta de configuración. No se debe interpretar como una autorización para omitir la integridad.

6.3 Tratamiento de SKIPPED en el backend

SKIPPED significa que APIM no realizó la evaluación de integridad. No significa que el dispositivo haya sido validado ni que su evidencia sea confiable.

Si el backend interpreta SKIPPED como ALLOW, el mecanismo de desactivación temporal y la convivencia con versiones antiguas de la aplicación pueden convertirse en autorizaciones implícitas.

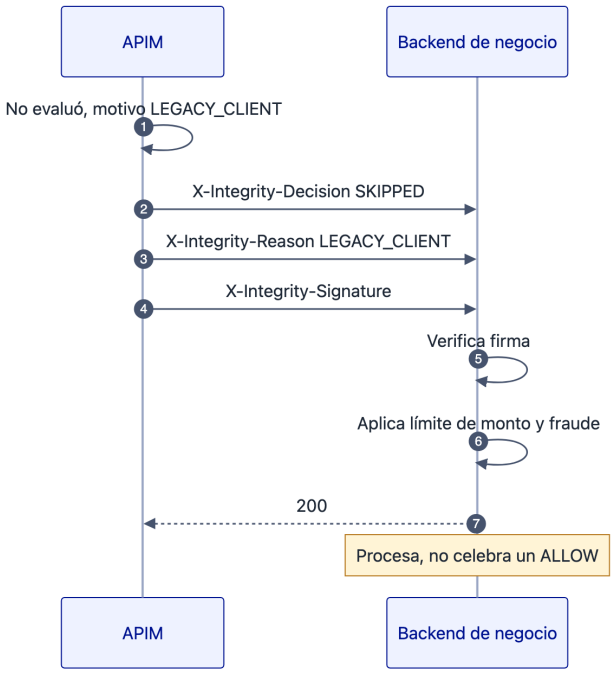


Figura 6.4 — APIM transmite la decisión SKIPPED y su motivo mediante un canal confiable; el backend aplica el tratamiento correspondiente

Recomendación arquitectónica

APIM debe transmitir la decisión y su motivo mediante información confiable, protegida por el mecanismo de autenticación e integridad entre APIM y el backend.

X-Integrity-Reason	Escenario	Tratamiento del backend
SWITCH_OFF	Validación desactivada temporalmente	Aplicar controles DEGRADED en Tier 1, con restricciones adicionales y alerta mientras dure la desactivación.
MONITOR	Validación en modo de observación	Procesar la operación y registrar la decisión hipotética, sin aplicar el bloqueo de integridad.
LEGACY_CLIENT	Aplicación dentro de la ventana de convivencia	Procesar con restricciones definidas para Tier 1, como límites de monto y controles antifraude adicionales.
TIER3	Operación fuera del alcance de integridad	Procesar según los controles habituales de negocio y seguridad.
Ausente o desconocido	Error de configuración, integración o contrato	Tratar como UNVERIFIED; DENY en Tier 1 y aplicar la política de contingencia aprobada en Tier 2.

Los valores MONITOR, LEGACY_CLIENT y SWITCH_OFF deben estar sujetos a las reglas de vigencia y alcance definidas para cada escenario. El backend no debe aceptar una decisión únicamente porque el cliente envíe estos valores.

6.4 Responsable del switch, aprobación y caducidad

La desactivación temporal de la validación es una excepción de seguridad. Sin responsables, aprobación y vencimiento, una medida adoptada durante un incidente puede permanecer activa indefinidamente.

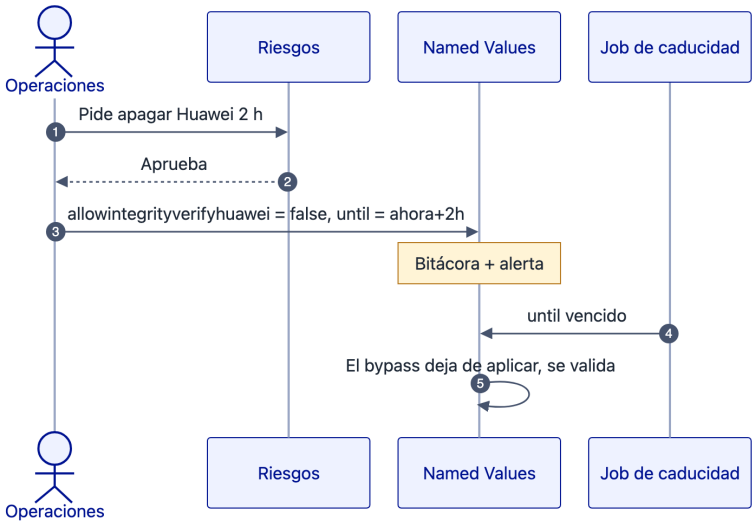


Figura 6.5 — La desactivación temporal es un cambio controlado: ticket, aprobación, bitácora y vencimiento automático.

Gobierno propuesto

Rol	Responsabilidad
Dueño	AppSec administra el Named Value o su modificación mediante pipeline.
Aprobador	Riesgos autoriza la excepción, indicando alcance, plataforma afectada y duración.
Ejecutor	Operaciones / SRE ejecuta el cambio, coordinando con AppSec.
Caducidad	Dos horas por defecto. Toda extensión requiere una nueva aprobación.
Auditoría	Registrar quién solicitó, aprobó y ejecutó el cambio, motivo, plataformas afectadas, hora de inicio y vencimiento.

Reglas de operación

- El switch debe generar una alerta desde el momento en que se activa.
- La desactivación debe tener un alcance definido: global o por plataforma.
- Al cumplirse la fecha de caducidad, la validación vuelve a estar activa automáticamente.
- Si no existe una fecha de caducidad válida, no se permite activar el bypass.
- Toda extensión debe quedar registrada y aprobada nuevamente.

Criterio de aceptación de la sección: las cuatro políticas deben estar aprobadas por Arquitectura, AppSec, Riesgos y Operaciones. Además, deben existir pruebas que demuestren el comportamiento esperado ante indisponibilidad de proveedores, errores de configuración, decisiones SKIPPED y activación o vencimiento del switch. ENFORCE no debe habilitarse mientras estas políticas no estén implementadas y verificadas.

7. Referencias usadas en la revisión

- Google Play Integrity: standard requests, requestHash, decodeIntegrityToken, cuotas diarias y formulario de ampliación.
- Apple DeviceCheck / App Attest: isSupported, attestation, assertion, counter, clientDataHash.
- Huawei Safety Detect SysIntegrity: JWS, cadena CBG, hoja sysintegrity.platform.hicloud.com, basicIntegrity. No hay API de descifrado equivalente a la de Google.
- Azure API Management: set-header exists-action, rate-limit-by-key, validate-jwt, send-request timeout.
- Azure Cosmos DB: ETag / If-Match, patch con filtro, 412 Precondition Failed, createItem 409.